KH정보교육원

서버프로그램 구현

CONTENTS

Part 1. 개발환경 구축하기

Chapter 1. 웹 애플리케이션의 실행 흐름 4
Chapter 2. 자바 웹 개발환경 구축하기 5
Chapter 3. 웹 서버 구축하기 11

Part 2. 공통 모듈 구현하기

Chapter 4. JAVA EE 와 웹 설계 규약 16
Chapter 5. MVC MODEL2 패턴의 이해 20
Chapter 6. 웹 애플리케이션 아키텍처 구축하기 48

Part 3. 서버 프로그램 구현하기

Chapter 7. 서블릿과 라이프 사이클 51
Chapter 8. 서블릿 구현 및 API 의 이해 54
Chapter 9. LISTENER, FILTER, WRAPPER 65
Chapter 10. JSP Element 83
Chapter 11. JSP 내장 객체 96
Chapter 12. 파일 업/다운로드 102
Chapter 13. Ajax 112

Part 4. 배치 프로그램 구현하기

Chapter 14. JSP 액션 태그 124
Chapert 15. EL 와 JSTL 128
Chapter 16. 공공데이터 활용 144

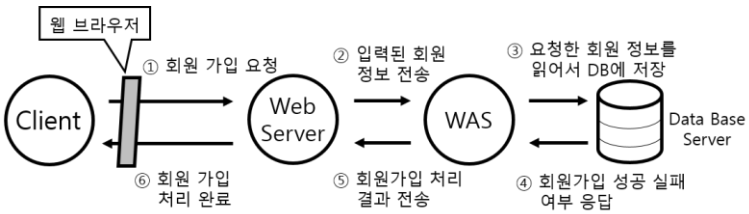
Part 1. 개발환경 구축하기

Chapter 1. 웹 애플리케이션의 실행 흐름

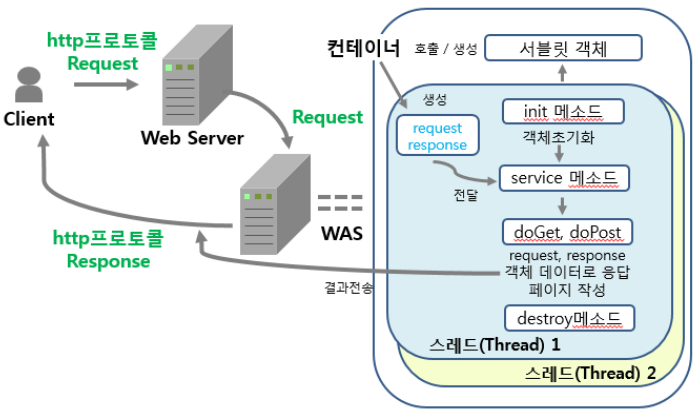
1 웹 어플리케이션의 실행 흐름

웹 어플리케이션이란 인터넷(Internet)을 기반으로 실행되는 어플리케이션을 뜻한다. 이 말은 즉, 웹 브라우저를 통해 요청한 정보를 처리하는 프로그램을 제작하는 것을 의미한다. 이는 단순히 HTML이나 정적인 페이지를 보여주는 웹 서버가 아닌 주어진 정보를 가지고 동적인 웹 페이지를 구현하는 것을 말하며, 이러한 서비스를 제공하는 서버를 WAS(Web Application Server)라고도 한다.

웹 어플리케이션의 실행 흐름을 회원가입 요청과 연관지어 보면 다음과 같다.



1. 사용자가 웹 브라우저를 통해 회원 가입 페이지의 기입란을 작성하고, 가입 요청 버튼을 클릭하여 웹 서버에 요청 메시지 전송.
2. 웹 서버는 입력된 정보를 확인 후 WAS에 해당 요청 데이터를 전송
3. WAS에서 입력된 정보를 기반으로 Data Base 서버에 저장
4. DataBase 서버에서 데이터 저장의 처리 결과를 WAS에 전송
5. WAS에서 회원 가입 요청에 대한 응답 처리 결과를 전송
6. 사용자는 웹 브라우저를 통해 회원 가입의 성공 / 실패 페이지를 조회



Chapter 2. 자바 웹 개발환경 구축하기

1 개발 환경 구축

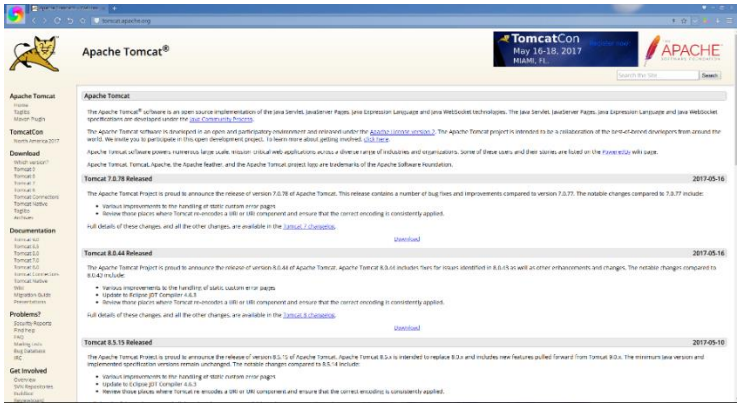
웹 어플리케이션 개발을 위해서는 JDK와 톰캣, 이클립스의 설치가 필요하다.

JDK와 이클립스의 설치는 생략한다.

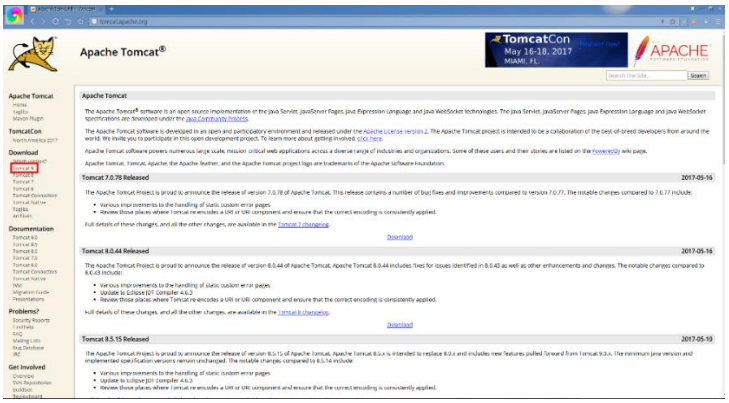
1.1 톰캣 설치하기

웹 어플리케이션을 작성한 후 웹페이지를 브라우저에 띄우기 위해서는 실질적으로 서블릿과 JSP를 구동시키는 WAS가 필요하며 다양한 WAS 중 오픈소스 프로젝트로 개발되어 무료로 제공되는 톰캣을 설치하여 사용하도록 한다.

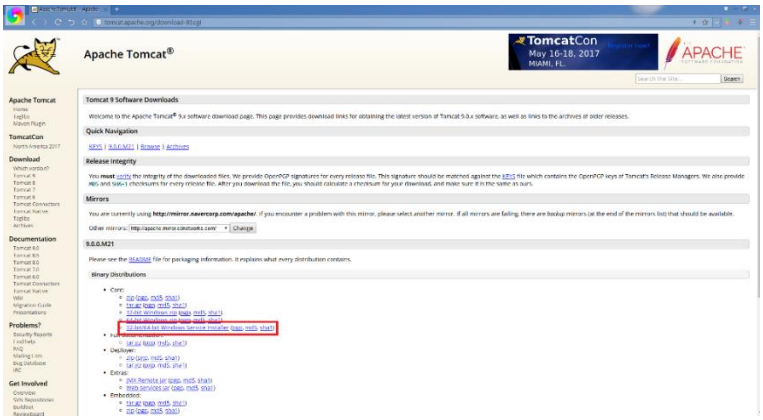
1. **http://tomcat.apache.org/ 접속**



2. **Download – Tomcat9 선택**

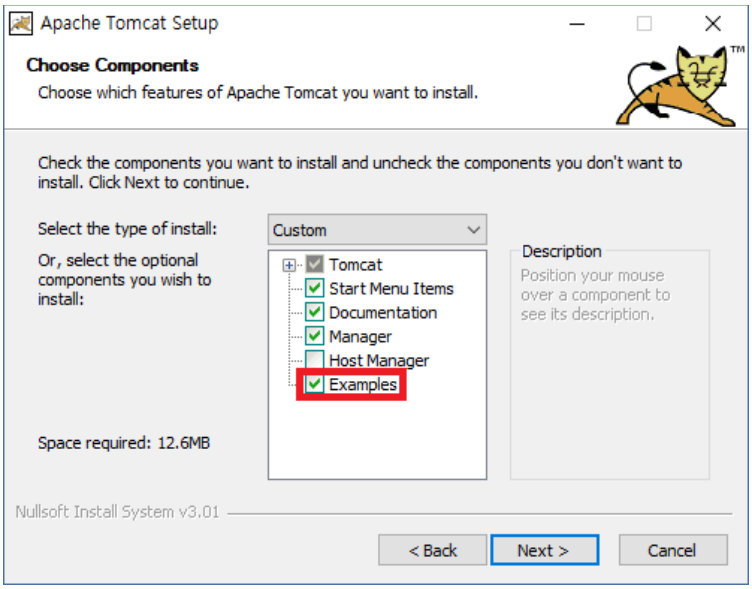


3. 32-bit/63-bit Windows Service Installer를 선택해 다운로드



4. 받은 파일을 이용하여 설치 시작

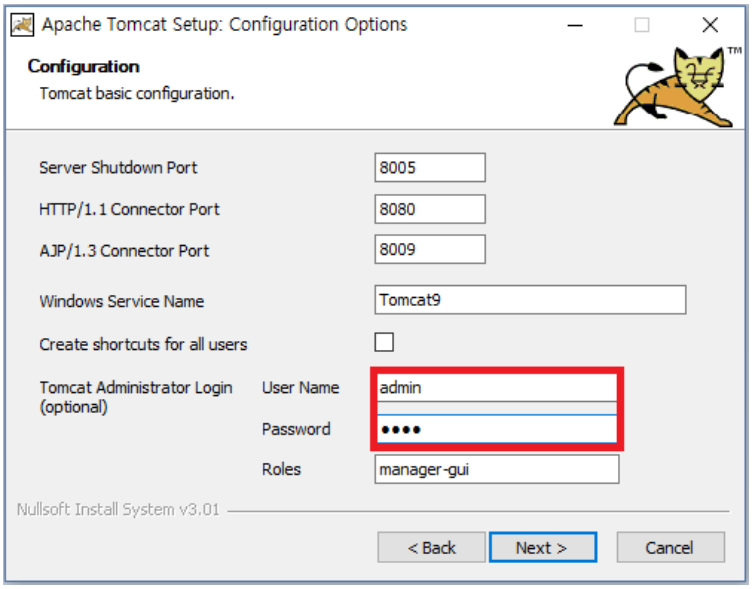
[Next] 버튼을 클릭하여 넘긴 후 환경설정부분에서 Examples를 추가로 체크한 후 [Next] 버튼을 클릭한다. Examples는 후에 확인을 위해 사용할 것이다.



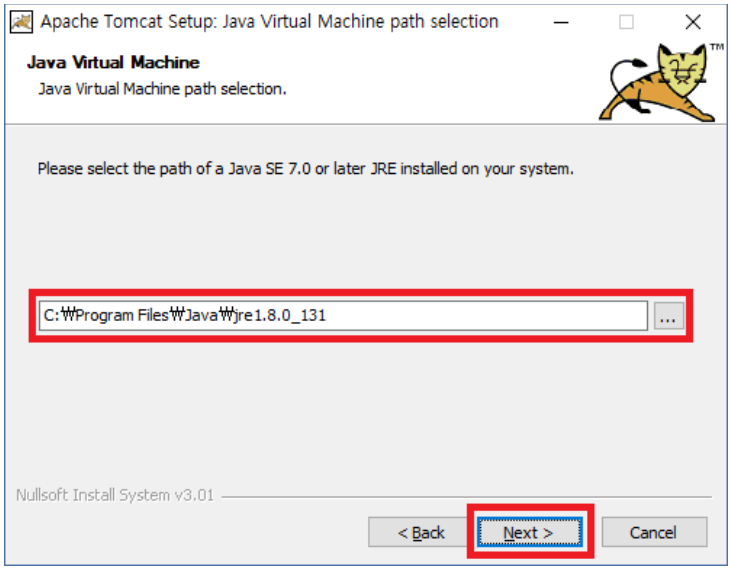
5. 환경 설정 중 Administrator Login 이름과 패스워드를 설정한다.

User Name : admin

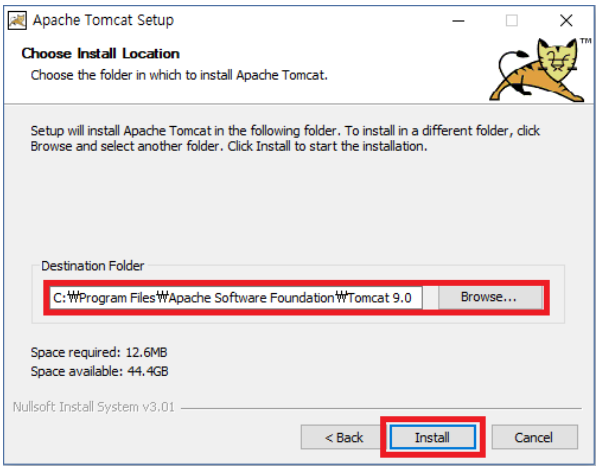
Password : 1234



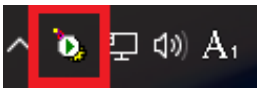
6. JRE 설치 경로를 입력한 후 [Next]



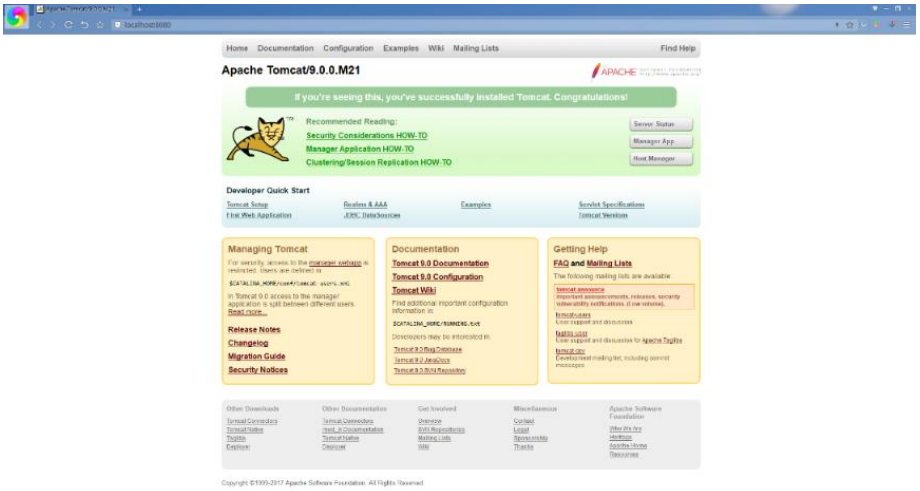
7. **톰캣이 설치될 경로를 지정한 후 [Install]**



8. 톰캣 설치가 완료되면 바로 실행이 되며 트레이 아이콘을 통해 확인할 수 있다



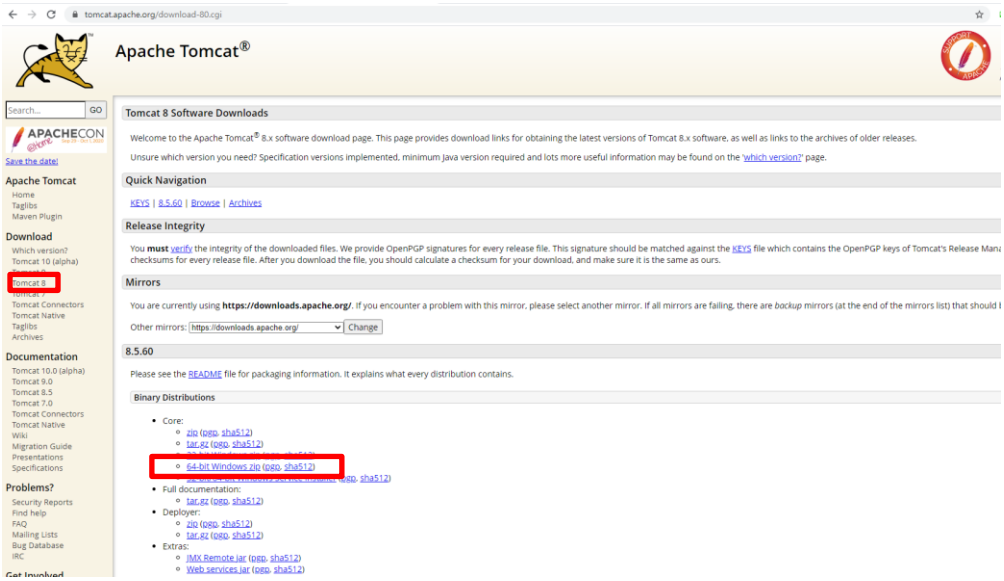
9. **정상적으로 설치가 완료되었다면 브라우저에 http://localhost:8080 을 입력하여 접속하면 톰캣 시작 페이지를 확인할 수 있다**



1.2 이클립스에 톰켓 연동하기

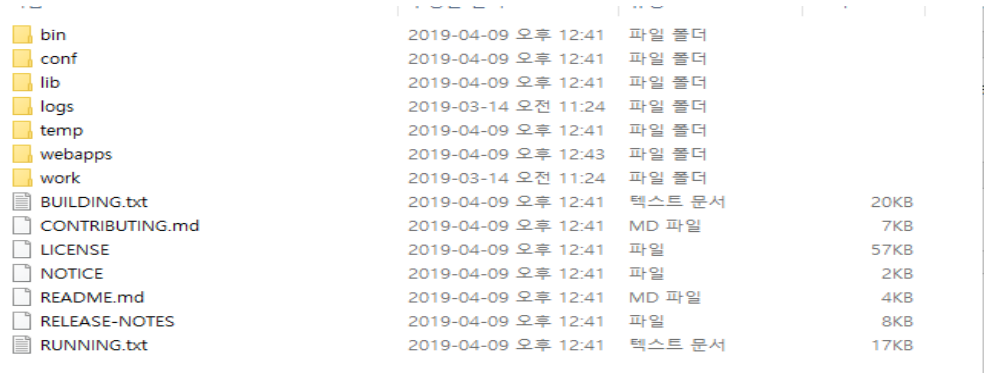
윈도우 설치용(instroller)이 아닌 tomcat실행파일을 다운받아서 통합개발도구인 이클립스에 연동한다.

1. http://tomcat.apache.org에서 tomcat파일 다운로드 사이트 사이드메뉴에서 Tomcat 8 선택 후 64-bit Windows zip (pgp, sha512) 클릭하여 다운로드 한다.



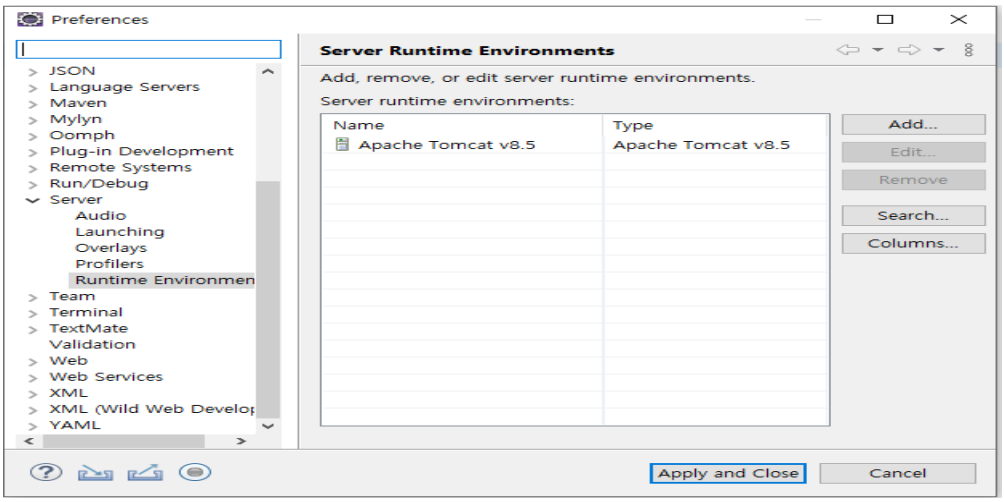
2. 다운로드 후 원하는 폴더에 압축해제한다. 다운로드 폴더에 가서 우클릭 -> 압축풀기 진행 한다. *압축을 해제하 폴더경로 반드시 확인 해야한다.

3. 압축을 해제하고 압축해제 폴더로 이동해보면 다음과 같은 파일들이 생성된다.

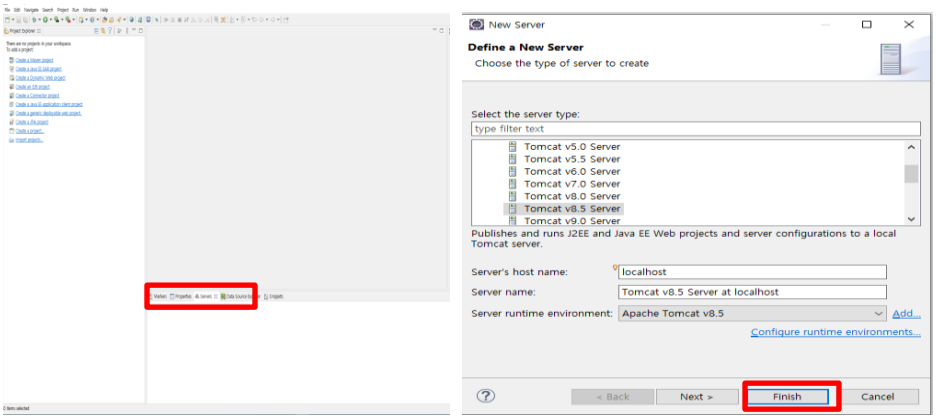


4. 이클립스를 열고 상단메뉴에서 window -> Preferences -> 팝업메뉴창 왼쪽 메뉴 -> Server -> Runtime Evironment -> add -> tomcat설치폴더 추가 폴더경로는 tomcat 폴더들이 있는경로까지만 설정

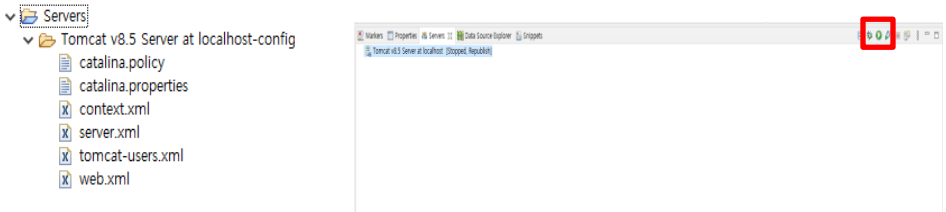
   예) 다운로드폴더₩apache-tomcat-8.5.39



5. 서버메뉴에서 serverproject를 생성하면 이클립스와 tomcat이 연동된다.



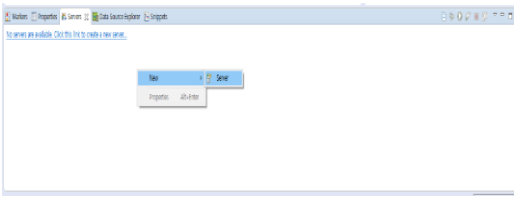
6. 프로젝트 생성 후 실행하기, 초록색 실행버튼 클릭



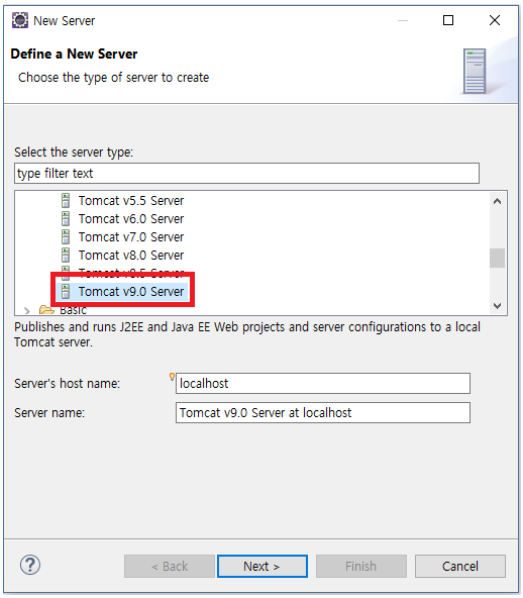
Chapter 3. 웹 서버 구축하기

1 이클립스에 톰캣 연동하기

① 이클립스의 하단부에 [Servers] 탭을 선택하여 우클릭 메뉴를 통해 [New – Server]를 선택

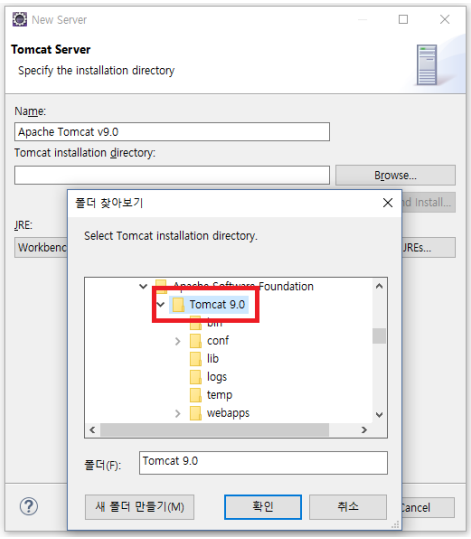


② [New Server] 창에서 [Define a New Server]에 설치한 버전과 맞는 버전의 톰캣서버를 선택한 후 [Next]

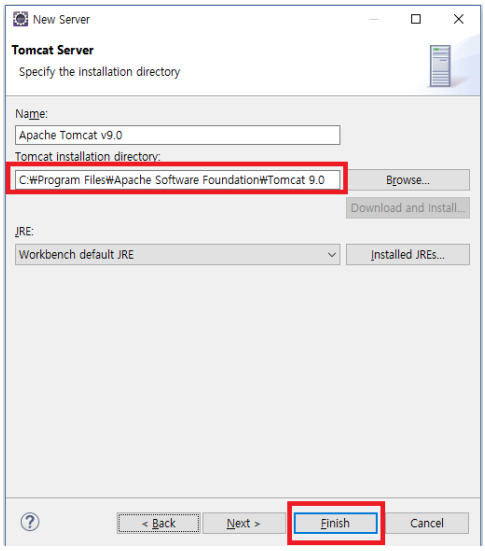


③ [Tomcat installation directory] 부분에 톰캣을 설치한 경로를 지정

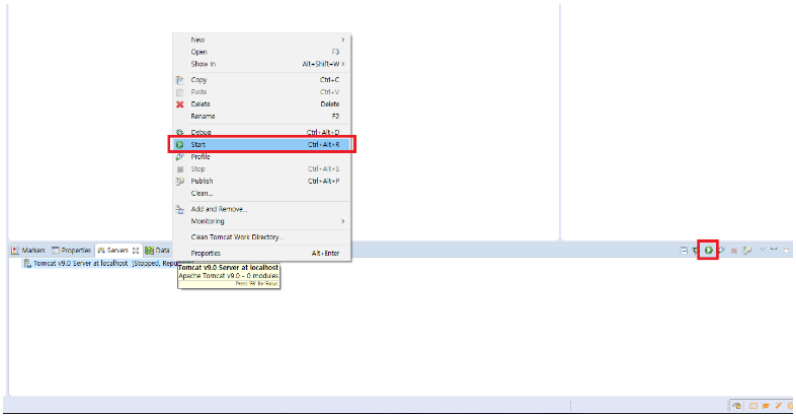
(기본 경로 : C:₩Program Files₩Apache Software Foundation₩Tomcat 9.0)



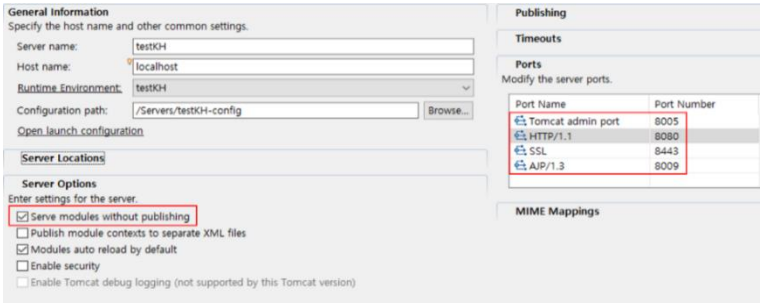
④ 경로가 잘 지정되었는지 확인 후 [FInish]



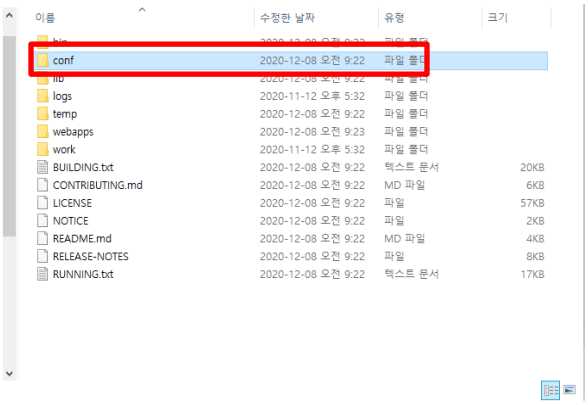
⑤ [Servers] 탭에 추가된 서버를 우클릭하여 [Start]하거나 아이콘을 이용해 서버를 시작시킨다



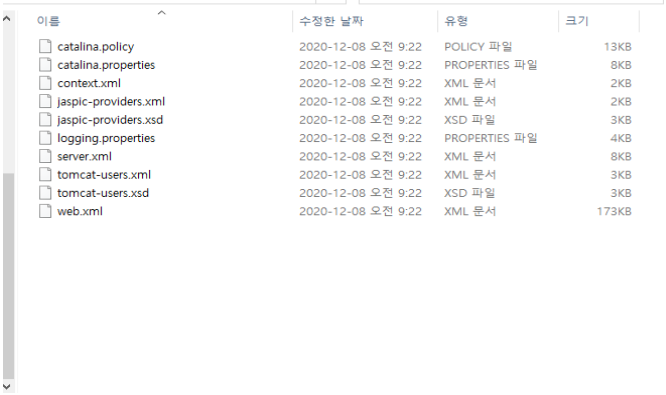
⑥ 만들어진 서버를 더블 클릭하면 서버의 설정을 확인하고 변경할 수 있다.



⑦ 포트관련 내용은 서버의 파일로 수정이 가능하다. tomcat 이 설치되어있는 폴더로 이동하여 conf 폴더 클릭한다.



⑧ conf 폴더내부의 server.xml 파일을 열기를 한다. 파일에서 우클릭하고 편집으로 파을일 열 수 있고, vscode, 문서편집기 애플리케이션을 이용해서 열기가 가능하다.
* vscode 를 이용해 보겠다.



⑨ 파일을 열고 69 번 라인을 보면 <Connector>태그가 나오는데 port="8080"부분을 port="9090"으로 변경하면 서버의 기본포트가 9090 으로 설정되 이클립스에서 서버를 올려도 포트를 변경하지 않아도 된다.

```
62    <!-- A "Connector" represents an endpoint by which requests are received
63         and responses are returned. Documentation at :
64         Java HTTP Connector: /docs/config/http.html
65         Java AJP  Connector: /docs/config/ajp.html
66         APR (HTTP/AJP) Connector: /docs/apr.html
67         Define a non-SSL/TLS HTTP/1.1 Connector on port 8080
68    -->
69    <Connector port="8080" protocol="HTTP/1.1"
70               connectionTimeout="20000"
71               redirectPort="8443" />
72    <!-- A "Connector" using the shared thread pool-->
73    <!--
74    <Connector executor="tomcatThreadPool"
75               port="8080" protocol="HTTP/1.1"
76               connectionTimeout="20000"
77               redirectPort="8443" />
78    -->
```

Part 2. 공통 모듈 구현하기

Chapter 4. JAVA EE와 웹 설계 규약

1 서블릿, Servlet

Server + Applet의 합성어

서버에서 실행되는 Applet이라는 의미를 가지고 있으며 자바를 이용하여 웹에서 실행되는 프로그램을 작성하는 기술을 뜻한다. 서블릿은 또한 자바 클래스 형태의 웹 어플리케션을 말하는데 서블릿 클래스를 통해 동작한다.

* 서블릿 클래스

브라우저를 통해 자바 클래스가 실행되도록 하기 위해서는 javax.servlet.http 패키지에서 제공하는 HttpServlet 클래스를 상속받아 구현해야한다. HttpServlet 클래스를 상속 받아 만든 서브 클래스를 서블릿 클래스라고 한다. 이를 간단히 서블릿이라고 부르기도 한다.

2 JSP, Java Server Page

자바로 서버 페이지를 작성하기 위한 언어

HTML과 JSP 태그(스크립트릿)로 구성되어 화면을 작성하는 데 유리한 웹 프로그래밍 기술

클라이언트가 요청한 페이지와 쿼리스트링을 서버로 전달하는 역할을 한다. 브라우저의 요청을받아 WAS(Web Application Server)에 의해 내부적으로 서블릿으로 만들어져 실행되게 된다.

3 서블릿과 JSP

서블릿과 JSP는 작성방법에 차이가 있을 뿐 결론적으로 동일한 역할을 하게 된다. 초기 자바의 웹 개발은 서블릿을 이용한 개발이었으며 서블릿을 이용한 웹 프로그래밍에서는 자바코드를 기반으로 하기 때문에 HTML 코드를 집어넣기가 상당히 불편했다. 이후 HTML에 자바 코드를 넣을 수 있게 하려는 발상으로 JSP가 나왔으며 JSP는 서블릿과 달리 HTML 코드에 자바코드를 집어넣는 방식으로 개발할 수 있다.

4 자바 빈

프로그램에서 사용될 여러 개의 정보를 모아놓는 데이터 저장소의 역할을 하는 클래스
정보 은닉(Data Hiding) 개념이 적용된다
자바의 클래스이며 필드와 getter/setter 메소드를 하나의 쌍으로 갖는 특별한 클래스를 뜻한다
JSP에서 <jsp:useBean>, <jsp:setProperty>, <jsp:getProperty> 액션 태그를 사용한다

5 자바 빈 설계 규약

JavaBean은 특별한 목적이 있는 클래스지만 일반적인 클래스의 형태를 가지고 있기 때문에 사용하다 보면 단순히 일반 클래스가 되는 경향이 있다. 이를 방지하기 위해 JavaBean만의 설계규약이 존재한다.

- 멤버 변수마다 별도의 getter/setter 메소드가 존재해야 한다
- getter 메소드는 파라미터가 존재하지 않는다
- setter 메소드는 파라미터가 반드시 하나 이상 존재한다
- 반드시 패키지화 한다
- 멤버 변수는 private 형으로 지정하여 선언한다(캡슐화 적용)

6 자바 빈을 이용한 간단한 웹 어플리케이션

myWeb/web/view/beanReq.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>Beans Insert Message</title>
08 </head>
09 <body>
10         <h1>간단한 자바빈즈 프로그램 </h1>
11         <hr color="red"/>
```

```
12              <form method="post" action="beanRes.jsp">
13              메세지 : <input type="text" name="message" />
14              <input type="submit" value="전송">
15              </form>
16  </body>
17  </html>
```

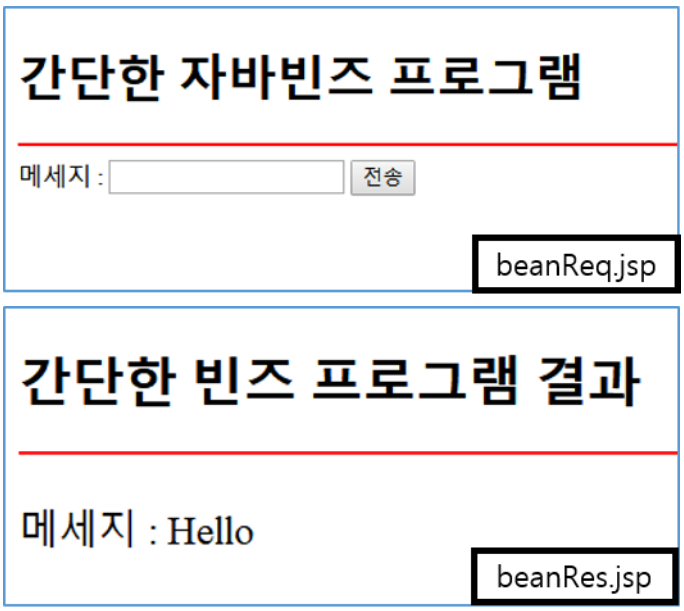
first.beanTest.model.vo.SimpleBeanData.java

```
01  package first.beanTest.model.vo;
02
03  public class SimpleBeanData {
04              private String message;
05
06              public String getMessage() {
07                      return message;
08              }
09
10              public void setMessage(String message) {
11                      this.message = message;
12              }
13  }
```

myWeb/web/view/beanRes.jsp

```
01  <%@ page language="java" contentType="text/html; charset=UTF-8"
02      pageEncoding="UTF-8"%>
03  <%
04  request.setCharacterEncoding("UTF-8");
05  //String message = request.getParameter("message");
06  %>
07  <jsp:useBean id="msg" class="first.beanTest.model.vo.SimpleBeanData" />
08  <%-- SimpleBeanData msg = new SimpleBeanData; --%>
09
10  <jsp:setProperty name="msg" property="message" />
```

```
11  <%-- msg.setMessage ("text"); --%>
12
13  <!DOCTYPE html>
14  <html>
15  <head>
16  <meta charset="UTF-8">
17  <title>Bean Response Check</title>
18  </head>
19  <body>
20          <h1>간단한 빈즈 프로그램 결과</h1>
21          <hr color="red"></hr><br>
22          <font size="5">
23                  메세지 : <jsp:getProperty name="msg" property="message" />
24                  <%-- <%=message%> --%>
25          </font>
26          <br><br>
27  <a href="../index.html">뒤로가기</a>
28  </body>
29  </html>
```



Chapter 5. MVC MODEL2 패턴의 이해

이제 전 챕터에서 설명한 MVC모델 2 패턴을 활용하여 웹 애플리케이션 아키텍처를 구축해보자.

DB는 JDBC에서 실습한 TEST_MEMBER 테이블을 사용하여 간단한 로그인 서비스를 만들어보자.

1 웹 페이지 기본 요소 작성

1.1 web.xml

myWeb/web/WEB-INF/web.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
03 xmlns="http://xmlns.jcp.org/xml/ns/javaee"
04 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
05 http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd" id="WebApp_ID" version="3.1">
06   <display-name>testMVC2</display-name>
07   <welcome-file-list>
08     <welcome-file>index.jsp</welcome-file>
09   </welcome-file-list>
10   <filter>
11     <filter-name>EncodingFilter</filter-name>
12     <filter-class>common.filter.EncodingFilter</filter-class>
13   </filter>
14   <filter-mapping>
15     <filter-name>EncodingFilter</filter-name>
16     <url-pattern>/*</url-pattern>
17   </filter-mapping>
18 </web-app>
```

1.2 EncodingFilter.java

common.filter.EncodingFilter.java

```
01  package common.filter;
02
03  import java.io.IOException;
04
05  import javax.servlet.*;
06  import javax.servlet.http.HttpServletRequest;
07
08  public class EncodingFilter implements Filter {
09     @Override
10     public void init(FilterConfig arg0) throws ServletException {}
11     @Override
12     public void doFilter(ServletRequest request, ServletResponse response, FilterChain
13  chain)
14           throws IOException, ServletException {
15
16        request.setCharacterEncoding("UTF-8");
17        response.setContentType("text/html;charset=UTF-8");
18
19        chain.doFilter(request, response);
20     }
21     @Override
22     public void destroy() {}
   }
```

1.3 JDBCTemplate.java

common.jdbcDriverJDBCTemplate.java

```
01 package common.jdbcDriver;
02
03 import java.io.*;
04 import java.sql.*;
05 import java.util.Properties;
06
07 public class JDBCTemplate {
08     public JDBCTemplate() {}
09     public static Connection getConnection() {
10       Connection con = null;
11       Properties prop = new Properties();
12       try {
13         String currentPath = JDBCTemplate.class.getResource("./").getPath();
14         prop.load(new BufferedReader(new
15 FileReader(currentPath+"driver.properties")));
16         Class.forName(prop.getProperty("driver"));
17         con = DriverManager.getConnection(prop.getProperty("url"),
18 prop.getProperty("user"),
19               prop.getProperty("pwd"));
20       } catch (Exception e) {
21         e.printStackTrace();
22       }
23       return con;
24     }
25     public static void close(Connection con) {
26       try {
27         if (con != null && !con.isClosed()) { con.close(); }
28       } catch (SQLException e) { e.printStackTrace(); }
29     }
30     public static void close(Statement stmt) {
31       try {
```

```
32            if (stmt != null && !stmt.isClosed()) { stmt.close(); }
33          } catch (SQLException e) { e.printStackTrace(); }
34        }
35        public static void close(ResultSet rset) {
36          try {
37            if (rset != null && !rset.isClosed()) { rset.close(); }
38          } catch (SQLException e) { e.printStackTrace(); }
39        }
40        public static void commit(Connection con) {
41          try {
42            if (con != null && !con.isClosed()) { .commit(); }
43          } catch (SQLException e) { e.printStackTrace(); }
44        }
45        public static void rollback(Connection con) {
46          try {
47            if (con != null && !con.isClosed()) { con.rollback();   }
48          } catch (SQLException e) { e.printStackTrace(); }
          }
      }
```

src/common/jdbcDriver/driver.properties

```
01  # Driver.properties
02  driver=oracle.jdbc.driver.OracleDriver
03  url=jdbc:oracle:thin:@localhost:1521:xe
04  user=student
05  pwd=student
```

1.4 myStyle.css

myWeb/web/css/myStyle.css

```
01  @charset "UTF-8";
02  ul li a {
03              text-decoration:none;
04              color:#000080;
05              display: block;
06              width: 120px;
07              height: 35px;
08              background: #96c7f3;
09              padding-top: 5px;
10  }
11  ul li a:hover {
12              background: #260679;
13              color: white;
14              text-shadow: 2px 2px 5px black;
15  }
16  ul li {
17              float:left;
18              list-style: none;
19              margin-top:60px;
20              margin-right: 5px;
21              padding-top: 10px;
22              text-align: center;
23              font-size: 16pt;
24              font-weight: bold;
25  }
26  .box {  ## 로그인 박스를 위한 style sheet
27              padding: 15px;
28              width: 300px;
29              height: 120px;
30              text-align: center;
31              background-color: #e3ffec;
```

```
32              float: right;
33  }
34  #myinfo {  ## 회원 정보 페이지를 위한 style sheet
35              background-color:#e8e8ff;
36              width: 300px;
37              left: 0;
38              right: 0;
39              margin-left: auto;
40              margin-right: auto;
41              padding:15px 20px 15px 20px;
42  }
```

2 Model : Member VO (Value Object) 요소 작성

프로젝트에서 사용될 VO/DTO 클래스는 웹 어플리케이션에서 오가는 객체를 클래스로 구현한 것이다. 일반적으로 VO/DTO 클래스의 필드 변수는 DataBase에서 설계된 테이블의 컬럼 이름과 동일하게 선언하여 차후 작업시 헷갈리지 않도록 하자.

2.1 Member.java

member.model.vo.Member.java

```
01 package member.model.vo;
02
03 public class Member implements java.io.Serializable {
04   private static final long serialVersionUID = 1L;
05   private String id;
06   private String passwd;
07   private String name;
08   private String email;
09
10   public Member() {}
11   public Member(String id, String passwd, String name, String email) {
12     super();
13     this.id = id;
14     this.passwd = passwd;
15     this.name = name;
16     this.email = email;
17   }
18   @Override  // toString() 재정의
19   public String toString() {
20     return "Member [id=" + id + ", passwd=" + passwd + ", name=" + name + ",
21 email=" + email + "]";
22   }
23 /* ..... Getter & Setter 선언부 ..... */
24 }
```

3 Controller 요소 작성

3.1 LoginServlet.java

member.controller.LoginServlet.java

```
01  package member.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05
06  import javax.servlet.ServletException;
07  import javax.servlet.annotation.WebServlet;
08  import javax.servlet.http.*;
09
10  import org.json.simple.JSONObject;
11
12  import member.model.service.MemberService;
13  import member.model.vo.Member;
14
15  @WebServlet("/login.lo")
16  public class LoginServlet extends HttpServlet {
17      private static final long serialVersionUID = 1L;
18
19      public LoginServlet() { }
20
21      protected void doPost(HttpServletRequest request, HttpServletResponse response)
22  throws ServletException, IOException {
23       // 사용자의 정보를 JSON 형식으로 전달하기 위해 ContentType 변경
24         response.setContentType("application/json;charset=UTF-8");
25         MemberService mservice = new MemberService();
26         String id = request.getParameter("id");
27         String passwd = request.getParameter("passwd");
28         JSONObject job = new JSONObject();
29
```

```
30      // 입력받은 사용자의 ID와 비밀번호를 인자로 하여 Service의 loginMember()
31  호출
32          Member m = mservice.loginMember(id, passwd);
33          if(m != null) {
34            HttpSession session = request.getSession();
35            session.setAttribute("member", m);
36            job.put("result", "ok");
37            job.put("name", m.getName());
38          } else {
39            job.put("result", "fail");
40           }
41
42          PrintWriter out = response.getWriter();
43          out.println(job.toJSONString());
44          out.flush();
45          out.close();
46      }
    }
```

3.2 LogoutServlet.java

member.controller.LogoutServlet.java

```
01  package member.controller;
02
03  import java.io.IOException;
04
05  import javax.servlet.*;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.*;
08
09  @WebServlet("/logout.lo")
10  public class LogoutServlet extends HttpServlet {
11      private static final long serialVersionUID = 1L;
12
13      public LogoutServlet() { }
14      protected void doGet(HttpServletRequest request, HttpServletResponse response)
15  throws ServletException, IOException {
16          // 세션을 request를 통해 받아올 때, 기존에 생성 되어 있지 않다면
17          // 새로 만드는 것을 원치 않기 때문에 false 값을 인자로 한다.
18          HttpSession session = request.getSession(false);
19
20          // 만약 세션이 null이 아니라면 해당 세션을 만료 시켜 세션 정보를 없앤다.
21          if(session != null) session.invalidate();
22          response.sendRedirect("index.jsp");
23      }
24  }
```

3.3 DupIdChkServlet.java

member.controller.DupIdChkServlet.java

```
01  package member.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import javax.servlet.ServletException;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.*;
08  import member.model.service.MemberService;
09
10  @WebServlet("/dupid.lo")
11  public class DupIdChkServlet extends HttpServlet {
12     private static final long serialVersionUID = 1L;
13
14        public DupIdChkServlet() { }
15
16     protected void doPost(HttpServletRequest request, HttpServletResponse response)
17  throws ServletException, IOException {
18        MemberService mservice = new MemberService();
19        int result = mservice.dupIdChk(request.getParameter("id"));
20        PrintWriter out = response.getWriter();
21
22        if(result > 0) {
23           out.append("fail");  // 만약 dupIdChk()의 결과값이 0 이상이면 'fail'
24        } else {
25           out.append("ok");  // 결과값이 0 보다 크지 않으면, 'ok'를 담아서 보낸다.
26        }
27        out.flush();
28        out.close();
29     }
30  }
```

3.4 MemberInsertServlet.java

member.controller.MemberInsertServlet.java

```
01  package member.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import javax.servlet.ServletException;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.*;
08
09  import member.model.service.MemberService;
10  import member.model.vo.Member;
11
12  @WebServlet("/minsert.lo")
13  public class MemberInsertServlet extends HttpServlet {
14      private static final long serialVersionUID = 1L;
15      public MemberInsertServlet() { }
16
17      protected void doPost(HttpServletRequest request, HttpServletResponse response)
18          throws ServletException, IOException {
19        MemberService mservice = new MemberService();
20        // 전달 받은 파라미터 값을 변수에 담아 새로운 객체를 생성한다
21        String id = request.getParameter("id");
22        String passwd = request.getParameter("passwd");
23        String name = request.getParameter("name");
24        String email = request.getParameter("email");
25
26        int result = mservice.insertMember(new Member(id, passwd, name, email));
27        if (result > 0) {
28          response.sendRedirect("index.jsp");
29        } else { }
30      }
31  }
```

3.5 MemberUpdateServlet.java

member.controller.MemberUpdateServlet.java

```
01  package member.controller;
02
03  import java.io.*;
04
05  import javax.servlet.*;
06  import javax.servlet.http.*;
07  import javax.servlet.annotation.WebServlet;
08
09  import member.model.service.MemberService;
10  import member.model.vo.Member;
11
12  @WebServlet("/mupdate.lo")
13  public class MemberUpdateServlet extends HttpServlet {
14    private static final long serialVersionUID = 1L;
15
16    public MemberUpdateServlet() { }
17
18    protected void doPost(HttpServletRequest request, HttpServletResponse response)
19  throws ServletException, IOException {
20       // 기존의 생성된 세션과 세션에 담겨있던 "member" 객체를 불러온다.
21       HttpSession session = request.getSession(false);
22       Member m = (Member) session.getAttribute("member");
23
24        MemberService mservice = new MemberService();
25
26       String id = request.getParameter("id");
27       String passwd = request.getParameter("passwd");
28       String name = request.getParameter("name");
29       String email = request.getParameter("email");
30       PrintWriter out = response.getWriter();
31
```

```
32      if(m != null && m.getId().equals(id)) {  // 만약 ID 값이 기존값과 일치한다면
33  수정 실행
34          m.setPasswd(passwd);
35          m.setName(name);
36          m.setEmail(email);
37          if(mservice.updateMember(m) > 0) {
38            session.setAttribute("member", m);
39            response.sendRedirect("index.jsp");
40          } else {
41            out.append("<script>alert('회원 정보 수정 오류!₩n'+
42                  '관리자에게 문의하세요!');</script>");
43          }
44      } else {
45          RequestDispatcher                                  view                                =
46  request.getRequestDispatcher("views/error/errorPage.jsp");
47          request.setAttribute("msg", "회원 정보 수정 오류 발생!!");
48          view.forward(request, response);
49      }
50      out.flush();
51      out.close();
    }
}
```

3.6 MemberDeleteServlet.java

member.controller.MemberDeleteServlet.java

```
01  package member.controller;
02
03  import java.io.IOException;
04
05  import javax.servlet.ServletException;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.*;
08
09  import member.model.service.MemberService;
10
11  @WebServlet("/mdelete.lo")
12  public class MemberDeleteServlet extends HttpServlet {
13      private static final long serialVersionUID = 1L;
14
15      public MemberDeleteServlet() { }
16    protected void doGet(HttpServletRequest request, HttpServletResponse response)
17  throws ServletException, IOException {
18      MemberService mservice = new MemberService();
19      int result = mservice.deleteMember(request.getParameter("id"));
20      if(result > 0) {
21     // 기존에 생성되어 있던 세션을 만료
22        HttpSession session = request.getSession(false);
23        session.invalidate();
24      }
25      // index 페이지로 이동
26      response.sendRedirect("index.jsp");
27    }
28  }
```

4 Model : Service 요소 작성

4.1 MemberService.java

member.model.service.MemberService.java

```
01  package member.model.service;
02
03  import static common.jdbcDriver.JDBCTemplate.*;
04  import java.sql.Connection;
05
06  import member.model.dao.MemberDao;
07  import member.model.vo.Member;
08
09  public class MemberService {
10    public MemberService() {}
11
12    // 로그인 시 Member 객체를 받아오는 메소드
13    public Member loginMember(String id, String passwd) {
14      Connection con = getConnection();
15      Member m = new MemberDao().loginMember(con, id, passwd);
16      close(con);
17      return m;
18    }
19    // ID 중복 체크를 위한 메소드
20    public int dupIdChk(String id) {
21      Connection con = getConnection();
22      int result = new MemberDao().dupIdChk(con, id);
23      return result;
24    }
25    // Member 객체를 추가하는 메소드
26    public int insertMember(Member m) {
27      Connection con = getConnection();
28      int result = new MemberDao().insertMember(con, m);
29
```

```
30        if(result > 0) commit(con);
31        else rollback(con);
32        close(con);
33
34        return result;
35    }
36    // 기존 Member 객체의 정보를 수정하는 메소드
37    public int updateMember(Member m) {
38        Connection con = getConnection();
39        int result = new MemberDao().updateMember(con, m);
40
41        if(result > 0) commit(con);
42        else rollback(con);
43        close(con);
44
45        return result;
46    }
47    // 멤버 객체의 삭제를 요청하는 메소드
48    public int deleteMember(String id) {
49        Connection con = getConnection();
50        int result = new MemberDao().deleteMember(con, id);
51
52        if(result > 0) commit(con);
53        else rollback(con);
54        close(con);
55
56        return result;
57    }
58 }
```

5 Model : Member DAO(Data Access Object) 요소 작성

5.1 MemberDao.java

member.model.dao.MemberDao.java

```
01  package member.model.dao;
02
03  import static common.jdbcDriverJDBCTemplate.*;
04  import java.sql.*;
05
06  import member.model.vo.Member;
07
08  public class MemberDao {
09    public MemberDao() {}
10    // 사용자 로그인 메소드
11    // 매개변수로 받은 ID와 PASSWD로 DataBase에 접속하여 일치하는 데이터 조
12  회
13    public Member loginMember(Connection con, String id, String passwd) {
14      Member m = null;
15      PreparedStatement pstmt = null;
16      ResultSet rset = null;
17      String query = "SELECT * FROM TEST_MEMBER WHERE ID = ? AND PASSWD
18  = ?";
19      try {
20        pstmt = con.prepareStatement(query);
21        pstmt.setString(1, id);  // 첫번째 '?' 에 id 값 대입
22        pstmt.setString(2, passwd);  // 두번째 '?' 에 passwd 값 대입
23        rset = pstmt.executeQuery();
24        if(rset.next()) {  // 첫 열은 head 컬럼이므로 next() 로 실제 컬럼값을 가져
25  온다.
26          m = new Member();
27          m.setId(rset.getString("ID"));  // 받아온 ID 컬럼 값을 member변수에 대
28  입
29          m.setPasswd(rset.getString("passwd"));
```

```
30            m.setName(rset.getString("NAME"));
31            m.setEmail(rset.getString("EMAIL"));
32          }
33        } catch (SQLException e) {
34          e.printStackTrace();
35        } finally {  // ResultSet과 PreparedStatement 리소스를 반환
36          close(rset);
37          close(pstmt);
38        }
39        return m;  // 조회하여 가져온 Member 객체를 반환한다
40      }
41      // ID 값의 중복을 조회하는 메소드
42      public int dupIdChk(Connection con, String id) {
43        PreparedStatement pstmt = null;
44        ResultSet rset = null;
45        int result = 0;
46       // id로 테이블을 조회하여 있으면 1 이상, 없으면 0인 쿼리 작성
47        String query = "SELECT COUNT(*) FROM TEST_MEMBER WHERE ID = ?";
48        try {
49          pstmt = con.prepareStatement(query);
50          pstmt.setString(1, id);
51          rset = pstmt.executeQuery();
52          if(rset.next()) {
53            result = rset.getInt(1);    // rset의 첫 컬럼의 숫자값을 가져온다
54          }
55        } catch (SQLException e) {
56          e.printStackTrace();
57        } finally {
58          close(rset);
59          close(pstmt);
60        }
61        return result;
62      }
```

```
63    // DataBase에 Member 객체를 추가하는 메소드
64    public int insertMember(Connection con, Member m) {
65      PreparedStatement pstmt = null;
66      int result = 0;
67      String query = "INSERT INTO TEST_MEMBER VALUES(?, ?, ?, ?)";
68      try {
69        pstmt = con.prepareStatement(query);
70        pstmt.setString(1, m.getId());
71        pstmt.setString(2, m.getPasswd());
72        pstmt.setString(3, m.getName());
73        pstmt.setString(4, m.getEmail());
74        // executeupdate() 는 실행 결과를 반영된 행의 개수로 리턴하므로
75        // 1 이상은 실행 성공, 0 이하(구문 에러 포함)는 실패이다.
76        result = pstmt.executeUpdate();
77      } catch (SQLException e) {
78        e.printStackTrace();
79      } finally {
80        close(pstmt);
81      }
82      return result;
83    }
84    // 기존 사용자의 정보를 수정하여 DataBase의 데이터를 수정하는 메소드
85    public int updateMember(Connection con, Member m) {
86      PreparedStatement pstmt = null;
87      int result = 0;
88      String query = "UPDATE TEST_MEMBER SET "+
89          "PASSWD = ?, NAME = ?, EMAIL = ? WHERE ID = ?";
90      try {
91        pstmt = con.prepareStatement(query);
92        pstmt.setString(1, m.getPasswd());
93        pstmt.setString(2, m.getName());
94        pstmt.setString(3, m.getEmail());
95        pstmt.setString(4, m.getId());
```

```
96          result = pstmt.executeUpdate();
97        } catch (SQLException e) {
98          e.printStackTrace();
99        } finally {
100         close(pstmt);
101       }
102       return result;
103     }
104     // 기존 사용자를 DataBase에서 삭제하는 메소드
105     public int deleteMember(Connection con, String id) {
106       PreparedStatement pstmt = null;
107       int result = 0;
108       String query = "DELETE FROM TEST_MEMBER WHERE ID = ?";
109       try {
110         pstmt = con.prepareStatement(query);
111         pstmt.setString(1, id);
112         result = pstmt.executeUpdate();
113       } catch (SQLException e) {
114         e.printStackTrace();
115       } finally {
116         close(pstmt);
        }
        return result;
      }
    }
```

6 View 요소 작성

6.1 index.jsp 와 header.jsp

myWeb/web/index.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <%@ taglib prefix="c" uri="http://java.sun.com/jstl/core"%>
04 <!DOCTYPE html>
05 <html>
06 <head>
07 <title>MVC2 패턴</title>
08 <link rel="stylesheet" href="css/myStyle.css"/>
09 </head>
10 <body>
11 <h1 align="center">MVC2 Web Project</h1>
12 <c:import url="header.jsp"/>  <!—header.jsp를 index.jsp에 포함시킨다 -->
13 <br><br><br>
14 <h2 align="center">index page 입니다.</h2>
15 </body>
16 </html>
```

myWeb/web/header.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8" import="member.model.vo.*" %>
03 <% Member m = (Member)session.getAttribute("member"); %>
04 <nav>  <!— 메뉴 선택을 위한 nav 태그 작성 -->
05         <ul>
06                 <li><a href="#">메뉴1</a></li>
07                 <li><a href="#">메뉴2</a></li>
08                 <li><a href="#">메뉴3</a></li>
09                 <li><a href="#">메뉴4</a></li>
10         </ul>
11 </nav>
12 <%if (m == null){ %>
```

```
13  <div class="box">   <!— 로그인 From -->
14              <p>     아이디   :   <input   type="text"   name="userid"
15  id="userid"></p>
16              <p>비밀번호   :   <input   type="password"   name="userpwd"   id   =
17  "userpwd"></p>
18              <button          type="button"          id="btnLogin">로그인하기</button>
19    
20              <button type="button" id="btnEnroll" onclick=
21               "location.href='views/member/enroll.html';">회원가입</button>
22  </div>
23  <% } else { %>
24  <div class="box">
25              <h3>환영합니다, <%= m.getName() %>님!!!</h3>
26              <p>오늘도 좋은 하루 되세요~!!</p>
27              <button  onclick="location.href='views/member/myInfo.jsp'">회원정보보기
28  </button>
29              <button onclick="location.href='logout.lo'">로그아웃</button>
30  </div>
31  <% } %>
32  <script type="text/javascript"
33    src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
34  <script type="text/javascript">
35    $(function() {
36      $("#btnLogin").on('click', function() {
37        $.ajax({   // JQuery 를 통한 ajax 호출 방식 사용
38          type : "POST",
39          url : "login.lo",
40          data : {
41              id : $("#userid").val(),
42              passwd : $("#userpwd").val()
43          },
44          dataType : "json",   // 전달받을 객체는 JSON 이다.
45          success : function(data) {
```

```
46              if(data.result == "ok"){
47                      var text = "<h3>환영합니다, "+data.name+"님!!!</h3>"
48                      +"<p>오늘도 좋은 하루 되세요~!!</p>₩
49                      <button  onclick='location.href=₩"views/member/myInfo.jsp₩"'>
50  회원정보보기</button>₩
51                      <button          onclick=₩"location.href='logout.lo'₩">로그아웃
52  </button>";
53                      $(".box").html(text);
54              } else {
55                      alert("로그인 실패!₩nID와 비밀번호를 다시 확인하세요.");
56              }
57          },
58          error : function(request,status,error) {
59              alert("code:"+request.status+"₩n"+"message:"+request.responseText+
60              "₩n"+"error:"+error);
61          }
        });
      });
    });
    </script>
    <hr style="clear:both;">  <!— 이전 float 속성을 지우고 구분선을 표시한다. -->
```

6.2 enroll.html

myWeb/web/views/member/enroll.html

```
01  <!DOCTYPE html>
02  <html>
03  <head>
04  <meta charset="UTF-8">
05  <title>Enroll Member</title>
06  <link rel="stylesheet" href="../../css/myStyle.css" />
07  </head>
08  <body>
```

```
09  <h2 align="center">회원 가입 하기</h2>
10  <hr>
11    <section id="myinfo">
12      <form action="../../minsert.lo" id="enrollForm" method="post">
13        <table>
14          <tr>
15            <td>ID :</td>
16            <td><input type="text" name="id" id="userid"></td>
17          </tr>
18          <tr>
19            <td>Password :</td>
20            <td><input type="password" name="passwd"></td>
21          </tr>
22          <tr>
23            <td>이름 :</td>
24            <td><input type="text" name="name"></td>
25          </tr>
26          <tr>
27            <td>E-Mail :</td>
28            <td><input type="email" name="email"></td>
29          </tr>
30        </table>
31        <p align="center">
32          <button type="submit">회원 가입하기</button>    
33          <button type="reset">작성 양식 초기화</button>
34        </p>
35      </form>
36    </section>
37    <br> <br>
38    <p align="center">
39      <button type="button" onclick="location.href='../../index.jsp';">메인으로
40  가기</button>
41    </p>
```

```
42    <script type="text/javascript"
43    src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
44    <script type="text/javascript">
45    $("#enrollForm").submit(function(event){
46      var regex = /^[A-Za-z0-9]{5,14}$/;  // 정규식을 통한 문자열 패턴 분석
47      var chk = 0;
48      if($('#userid').val().length < 5) {
49        alert("아이디는 5자 이상이어야 합니다.");
50      } else if(!regex.test($('#userid').val())) {
51        alert("아이디는 영문자와 숫자만 가능합니다.");
52      } else {
53        $.ajax({     // ajax를 통한 아이디 중복 값 체크
54          url : "../../dupid.lo",
55          type : "post",
56          async : false,
57          data : {
58            id : $('#userid').val()
59          },
60          dataType : "text",
61          success : function(value) {
62            if(value === "fail"){
63              alert("이미 존재하는 아이디입니다.₩n 다른 아이디로 정하십시오.");
64            } else { alert("정상 가입이 되었습니다.");  chk++; }
65          },
66          error : function(request,status,error) {
67                alert("code:"+request.status+"₩n"+"message:"+
68                request.responseText+"₩n"+"error:"+error);  }
69        });
70      }
71      if(chk == 0) event.preventDefault();
72    });
73    </script>
74  </body>
```

```
</html>
```

6.3 myInfo.jsp

myWeb/web/views/member/myInfo.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
  pageEncoding="UTF-8" import="member.model.vo.*"%>
<% Member m = (Member) session.getAttribute("member"); %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>My Info View</title>
<link rel="stylesheet" href="../../css/myStyle.css" />
</head>
<body>
  <h2 align="center">내 정보 보기</h2>
  <hr>
  <% if(m != null){ %>
  <section id="myinfo">
    <form action="../../mupdate.lo" method="post">
      <table>
        <tr>
          <td>ID :</td>
          <td><input type="text" name="id" value="<%=m.getId()%>"
readonly="readonly"></td>
        </tr>
        <tr>
          <td>Password :</td>
          <td><input type="password" name="passwd"
value="<%=m.getPasswd()%>"></td>
        </tr>
        <tr>
```

```
29              <td>이름 :</td>
30              <td><input type="text" name="name"
31  value="<%=m.getName()%>"></td>
32            </tr>
33            <tr>
34              <td>E-Mail :</td>
35              <td><input type="email" name="email"
36  value="<%=m.getEmail()%>"></td>
37            </tr>
38          </table>
39          <p align="center">
40            <button type="submit">회원 정보 수정</button>
41               
42            <button type="button"
43  onclick="location.href='../../mdelete.lo?id=<%=m.getId()%>';">회원 탈퇴</button>
44          </p>
45        </form>
46      </section>
47      <%} else { %>
48        <jsp:forward page="enroll.html"/>
49      <!-- 가입하지 않은 사용자가 접속할 경우 enroll.html로 forwarding -->
50      <% } %>
51      <br><br>
52      <p align="center">
53        <button type="button" onclick="location.href='../../index.jsp';">메인으로
54  가기</button>
      </p>
    </body>
    </html>
```

Chapter 6. 웹 애플리케이션 아키텍처 구축하기

1 MODEL 1 방식의 웹

디자인코드인 HTML과 비즈니스 로직인 자바코드를 따로 구분하지 않고 하나의 JSP파일 내에 함께 기술하는 방식을 말한다. 모델 1 방식의 개발 형태를 보면 JSP/Servlet만으로 개발 된 경우, JSP + Java Bean을 이용한 경우, JSP + Custom Tag를 이용한 경우, 이 모든 것을 섞어 개발한 경우가 있다. 과거에 많이 사용되었던 웹 개발 아키텍처이고 또한 현재에도 많은 개발자들이 사용하고 있으며 간단한 페이지를 구성할 때 어울리는 방식이다.

하지만 JSP페이지가 너무 복잡해지는 경우고 있고 개발자와 디자이너 간의 의사소통이 어려운 점 등 때문에 프로젝트 규모가 커지는 경우에는 쓰이지 않으며, 모델 2 개발 방식을 많이 취하게 된다.

2 MODEL 2 방식의 웹

웹 어플리케이션을 개발할 때 MVC패턴을 적용하여 웹 어플리케이션의 개발이 가능하도록 구현한 것이다. MVC는 Model, View, Controller로 각각의 역할을 나누어 작업하고자 하는 일을 분담시키는 것을 말한다.

Model은 어플리케이션 로직을 담당하는 부분으로 데이터베이스와의 로직을 담당하는 부분을 말한다. javaBean이나 서비스클래스가 담당한다.

View 는 client에게 보여지는 부분이며 Model에서 생성된 Data를 client에게 제공하는 역할을 담당한다. JSP가 담당을 한다.

Controller는 사용자의 요청을 받아서 요청에 해당하는 비즈니스 로직을 수행하도록 하고, 작업결과에 따라 응답을 결정하는 역할을 한다. Model과 View사이의 데이터 전달 역할을 한다. Servlet이 담당한다.

모델 2 웹 개발 방식은 MVC 구조를 웹에 적용하여 개발하는 방식을 뜻한다.

2.1 모델 2 방식의 장점 / 단점

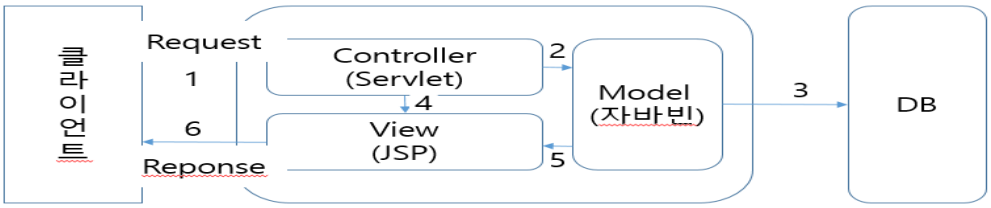
장점	단점
뷰(view)와 로직처리를 위한 자바 코드를 분리되어 코드 복잡도 낮아짐	구조가 복잡해져 기술 습득이 어렵고 작업량이 많아진다
뷰, 로직에 대한 명확한 역할 분담이 가능하다	자바 코드에 대한 비교적 높은 이해도가 요구된다

UI 레이어(화면 구현)을 단순화 시킴으로서 디자이너의 작업을 수월하게 한다	구조 설계를 위한 시간이 많이 소요된다 (프로젝트 초반 개발속도의 저하 초래)
유지보수가 쉬워지고, 비즈니스 로직의 재사용성이 높아지고 확장성이 용이하다	

2.2 모델 1과 모델2

모델2의 웹 개발 방식은 큰 규모의 프로젝트나 업데이트가 빈번한 프로젝트에는 장점이 크게 적용되지만 소규모 프로젝트나 규모가 작은 프로젝트일 경우 복잡한 형태 때문에 작업량만 늘어날 수 있다. 이럴 경우 모델1 방식으로 개발하는 것이 더 나을 수도 있다.

2.3 모델 2방식의 구조



3 모델2 방식의 구성 및 흐름

Controller는 중앙에 위치하여 View와 Model사이의 연동을 담당한다. 사용자의 요청은 Controller에서 받으며 Model을 통한 비즈니스 로직을 처리하고 수행 결과를 View를 통해 사용자에게 보여지게 한다. Controller 역할을 하는 서블릿이 매우 중요하며, 서블릿을 잘못 개발할 경우 웹 어플리케이션에 미치는 영향이 매우 크다.

3.1 Controller 역할

- 사용자의 요청을 받고 분석한다
- 사용자의 요청을 처리할 자바 빈을 생성하고 , 비즈니스 로직(메소드) 실행한다.
- 비즈니스 로직 수행 후 사용자의 요청을 JSP페이지나 혹은 특정 URL로 이동시킨다

3.2 Model 역할

- 비즈니스 로직이 구현되어 있다.,
- Database에 데이터를 추가, 조회, 변경, 조회하는 작업결과를 View에 제공한다.

3.3 View 역할

- 클라이언트에게 최종적으로 보여지는 영역으로 JSP와 JSTL을 이용하여 구현한다.

Part 3. 서버 프로그램 구현하기

Chapter 7. 서블릿과 라이프 사이클

1 서블릿이란?

Server + Applet의 합성어, JAVA 언어를 이용하여 사용자의 요청을 받아 처리하고 그 결과를 다시 사용자에게 전송하는 역할의 Class 파일을 말한다. 즉 웹에서 동적인 페이지를 java로 구현한 서버측 프로그램이라고 보면 된다.

2 서블릿의 역사

Java 언어의 창시자인 제임스 고슬링(James Gosling)은 1995년 자바를 발표하며 자바로 구현할 수 있는 서버 프로그래밍 기술에 대해서도 염두에 두고 있었지만, 해당 개념이 실제 구현이 가능할 정도의 제품화가 되어 있지 않은 상태였다. 당시 Sun사에서는 이를 서버로 구현할 수 있는 제품이 없어 잠시 미룰 수 밖에 없었다. 즉, Java EE Platform이 제품화되어 있지 않은 시기의 이야기인 것이다.

얼마 뒤, 자바 팀의 일원이었던 파바니 디완지(Pavni Diwanji)는 자바 서버 기술에 대한 필요성을 느껴 서블릿의 개념을 고안하였고, 이 개념을 토대로 프로젝트를 진행하여 서블릿 구현 및 제품화에 성공하였다. 그리고 이 기술은 1997년 6월에 Servlet 1.0을 공식 발표하면서 Java EE의 제품화에 포함되었다.

3 서블릿 버전 변천사

서블릿 버전	발표일	지원 Platform	주요 변경 내용
Servlet 1.0	1997.06	.	
Servlet 2.1	1998.11	JDK 1.1	RequestDispatcher, ServletContext 요소를 처음으로 정의
Servlet 2.3	2001.08	J2EE 1.3 J2SE 1.2	Filter 클래스 추가
Servlet 2.4	2003.11	J2EE 1.4 J2SE 1.3	web.xml문서를 통한 XML 스키마 사용
Servlet 2.5	2005.09	Java EE 5	어노테이션 지원 기능 추가

		Java SE 5	
Servlet 3.0	2009.12	Java EE 6 Java SE 6	어노테이션 지원 범위 확대, 서블릿 보안 강화, 비동기 서블릿 지원
Servlet 3.1	2013.05	Java EE 7	Non-blocking I/O 지원, HTTP1.1 지원

4 서블릿 설계 규약

I. 모든 서블릿은 **javax.servlet.Servlet** 인터페이스를 상속 받아 구현한다.

II. 서블릿을 구현 시 Servlet 인터페이스와 **ServletConfig** 인터페이스를 javax.servlet.**GenericServlet**에 구현한다.

III. HTTP 프로토콜을 사용하는 서블릿은 **javax.servlet.http.HttpServlet** 클래스는 javax.servlet.**GenericServlet**를 상속한 클래스로 서블릿은 httpServlet클래스를 상속받는다.

IV. 서블릿의 Exception을 처리하기 위해서는 **javax.servlet.ServletException** 을 상속 받아야 한다.

5 서블릿 동작 원리

톰캣이 구동되면 자바가상머신(JVM)이 구동되어 자바 문법을 따르는 서븟릿을 처리할 수 있는 환경이 제공된다. 이러한 환경을 서블릿 컨테이너라고 부른다. 서블릿의 실행은 서블릿 컨테이너에 의해 이루어진다.

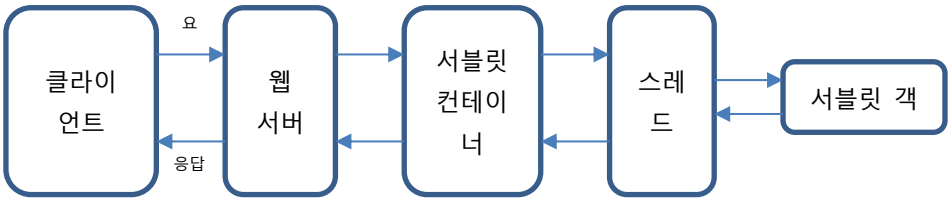
5.1 서블릿 컨테이너

웹서버와 서블릿 사이의 통신을 지원

서블릿의 생명주기를 관리

새로운 요청이 들어올 때 마다 자바 스레드를 생성해서 사용자의 요청을 처리

컨테이너는 선언적인 방법으로 보안관련 내용을 설정할 수 있다



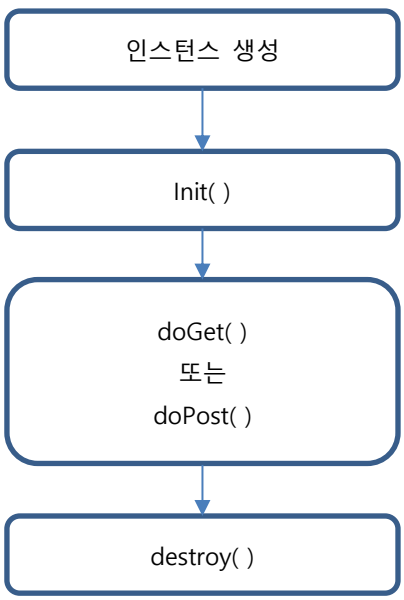
5.2 서블릿 라이프 사이클

서블릿은 클라이언트의 첫 번째 요청인 경우 서블릿 클래스를 찾아 메모리에 객체를 생성한다. 인스턴스화된 서블릿 객체는 메모리에 계속 남아 있게 되고 이후 서블릿이 호출되어도 서블릿 인스턴스가 다시 생성되지 않고 이미 로딩된 서블릿의 서비스를 받게 된다.

첫 번째 요청일 경우, 객체가 생성되면서 init( ) 메소드를 한 번 호출하며 초기화 작업을 진행한다.

그 후 클라이언트의 요청이 있을 때마다 doGet( ) 또는 doPost( ) 메소드가 실행되며, 이후에는 초기화 작업을 하지 않는다. 또한 여러 클라이언트가 동시에 요청하더라도 스레드가 생성되며 처리가 되어 doGet( ), doPost( )의 수행 속도가 빠른 편이다.

최종적으로 서블릿이 서비스되지 않을 때 destroy( ) 메소드가 호출되는데,
톰캣이 재시작되거나 종료되었을 때, 그리고 서블릿 내용이 변경되어 다시 컴파일했을 경우이다.

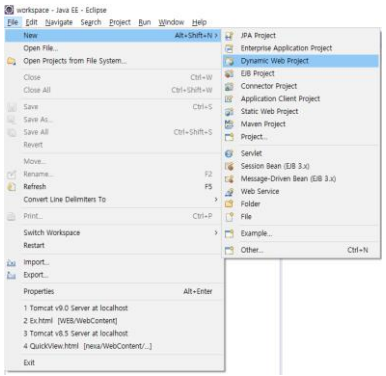


Chapter 8. 서블릿 구현 및 API의 이해

1 기본 서블릿 프로그램 만들기

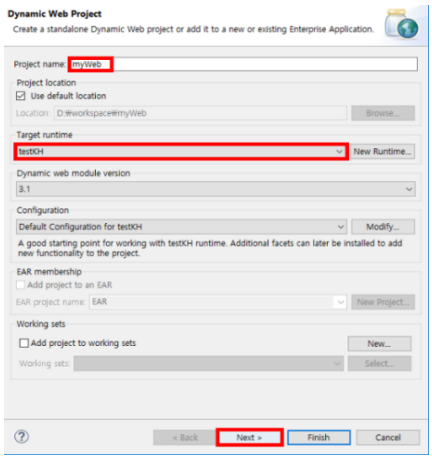
우선 서블릿을 이용해 기본적인 페이지를 확인할 수 있는 프로그램을 작성해 보자

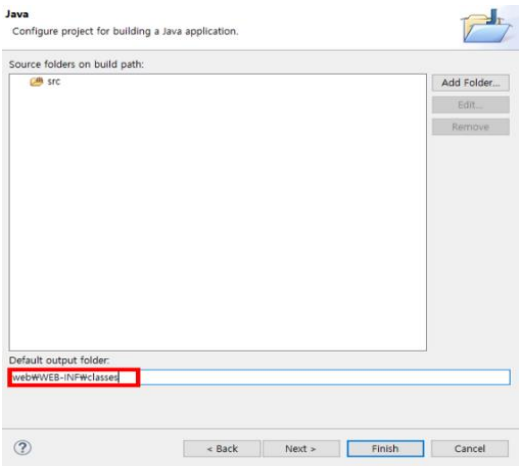
① 이클립스에서 [File – New – Dynamic Web Project 선택]



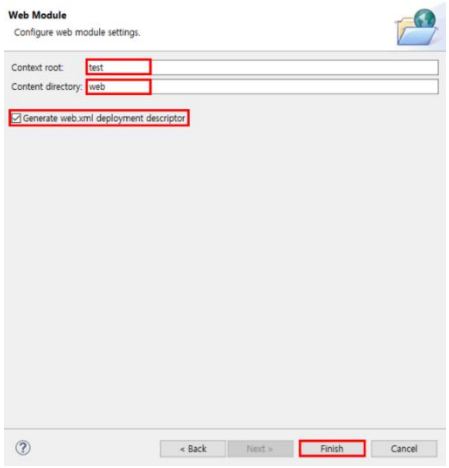
② [Project name]을 myWeb으로 입력하여 프로젝트 이름 설정 및 구동 환경이 되는 서버 설정 후 Next

③ 컴퍼일 되는 class 파일들이 생성될 경로를 web₩WEB-INF₩classes 로 변경 후 Next





④ Web Module 설정 창에서 Context root와 Content Directory 값을 확인 후 Generate web.xml 체크 박스를 체크한 뒤 Finish를 눌러 완료하자.

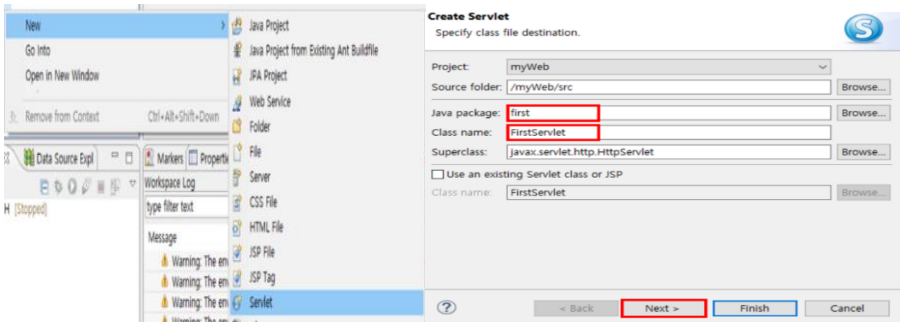


** **Context root :** 서버에 접근하기 위한 웹의 루트 디렉토리를 의미하며, 별다른 설정이 없으면 프로젝트 명으로 표시된다. 하지만 보안 상 웹의 최상위 경로를 그대로 노출하는 것은 좋지 않으므로 변경할 것을 권장한다.

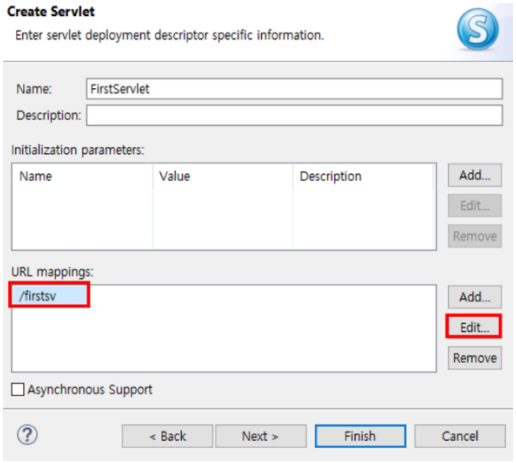
** **Content directory :** 이클립스 기준 WebContent가 기본이나 웹 서버로 사용하게 될 톰캣의 기본 루트인 web으로 변경해두면, 후에 웹 서버 등록 시에 편리하게 사용할 수 있다.

** **web.xml :** 웹 서버의 기본 설정을 다루는 파일이므로 반드시 생성해 두자.

⑤ 프로젝트 생성 후 [File – New – Servlet] 선택하여 package를 first, class를 firstServlet 으로 지정하고 Next



⑥ 하단의 URL mapping을 통해 사용될 servlet의 가명을 지어준다.

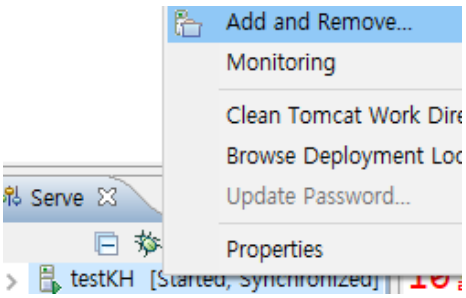


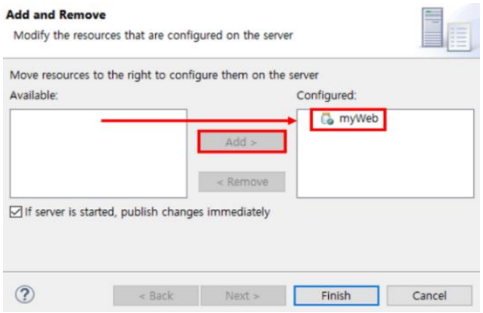
⑦ myWeb 프로젝트 내에 Java Resources – src 디렉토리에 first 패키지와 FirstServlet.java 서블릿 파일이 생성된 것을 확인할 수 있다.



⑧ 만들어 둔 서버에 마우스 우클릭 후 Add and Remove를 클릭하여 만든 웹 프로젝트

를 추가하고 Finish





그럼 서버에 프로젝트를 올리는 과정은 끝났다.

서버를 실행 시키면 프로젝트를 확인 할 수 있다.

- FirstServlet.java를 다음과 같이 작성한 후 브라우저를 통해 http://localhost:8080/test/firstsv 으로 접속

** 서버 설정에 따라 localhost 뒤의 포트번호가 다를 수도 있다.

first.FirstServlet.java

```
01  package first;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05
06  import javax.servlet.ServletException;
07  import javax.servlet.annotation.WebServlet;
08  import javax.servlet.http.HttpServlet;
09  import javax.servlet.http.HttpServletRequest;
10  import javax.servlet.http.HttpServletResponse;
11
12  @WebServlet("/firstsv")
13  public class FirstServlet extends HttpServlet {
14     private static final long serialVersionUID = 1L;
15     public FirstServlet() { super(); }
16
17     protected void doGet(HttpServletRequest request, HttpServletResponse response)
18           throws ServletException, IOException {
```

```
19      response.setContentType("text/html;charset=euc-kr");
20      PrintWriter out = response.getWriter();
21      out.println("<html>");
22      out.println("<body>");
23      out.println("My First Servlet Program");
24      out.println("<br>");
25      out.println("</body>");
26      out.println("</html>");
27    }
28    protected void doPost(HttpServletRequest request, HttpServletResponse response)
29        throws ServletException, IOException {
30       doGet(request, response);
31    }
32  }
```

- 브라우저화면에서 다음과 같이 결과를 확인할 수 있다



- 서블릿은 클라이언트(브라우저)의 요청에 응답하기 위해 HTML문서 형식을 제공해야 한다. response로부터 얻은 출력스트림 객체인 out의 print 메소드를 이용해 21-26 라인처럼 HTML 코드를 직접 작성해야 한다.

2 컨텍스트 패스, Context Path

위에서 작성한 서블릿을 확인하기 위해 접속한 URL의 형식은 다음과 같다



http://localhost:8080 까지는 웹 서버를 지칭하며 뒤의 Context root는 웹 서버에서 어떠한 서비스를 받을 것인지 지정하는 문자열이다. 웹 서버에서는 다양한 서비스를 제공할 수 있으며 이를 구분하기 위한 문자열을 컨텍스트 패스(Context Path)라고 부른다.

톰캣 서버에서 웹 어플리케이션의 서비스를 제공할 수 있게 해주려면 톰켓 서버에 웹 어플리케이션을 등록해야 하며 톰캣 서버의 server.xml 파일의 <Context> 태그를 사용한다. 이클립스를 사용하지 않을 때에는 일일이 개발자가 기술해야 하지만 이클립스는 프로젝트 단위로 자동 생성하게 된다.

톰캣 서버의 server.xml을 열어보면 이클립스에서 자동으로 추가해준 내용을 확인할 수 있다.

```
<Context docBase="myWeb" path="/test"
  reloadable="true"
  source="org.eclipse.jst.jee.server:myWeb"/>
```

3 서블릿 클래스 구성

서블릿 클래스는 클라이언트를 통해 서비스된다는 사용되는 목적에 따라 그 형태가 정형화된 모습을 보인다. javax.servlet.http 패키지에서 제공하는 HttpServlet클래스를 상속받아 구현해야 하며, 브라우저를 통해 외부에서 실행되도록 하기 위해 접근 제한자를 public으로 지정해야 한다.

```
public class [서블릿 클래스명] extends HttpServlet {
  // 접근제한자는 반드시 public
  // HttpServlet 클래스를 상속
}
```

3.1 GET 방식, POST 방식의 클라이언트 요청

클라이언트는 서버에 get과 post로 불리는 두 가지 방식 중 하나로 요청을 한다.

GET 방식은 서버에 있는 정보를 요청하여 가져오기 위해 설계되었다
URL에 쿼리 스트링을 추가하여 데이터를 전송하는 방식이며 형식은 다음과 같다
형식 : http://xxx.xxx.co.kr/servlet/login?id=tommy&name=arizona
240바이트까지 전달할 수 있으며 QUERY.STRING 환경변수를 통해 전달된다
URL노출로 보안성이 요구되는 경우에는 사용하면 안되며 검색엔진에서 검색단어 전송에 많이 이용한다

POST 방식은 서버로 정보를 올리기 위해 설계되었다
html header에 포함되어 전송되며 데이터크기의 제한은 없다
URL에 파라미터가 표시되지 않는다

* 쿼리 스트링

입력한 데이터를 서버로 전달하는 방식으로 '?'로 시작한다. 데이터는 '이름=값'의 형식을 취하며 여러 데이터들이 있을 경우 '&'로 구분한다.

3.2 doGet(), doPost()

서블릿은 클라이언트의 요청 방식에 따라 doGet( ) 혹은 doPost( ) 메소드가 호출된다

클라이언트의 요청 방식에 맞추어 HttpServlet 클래스의 doGet( ) 혹은 doPost( ) 메소드를 오버라이딩해야 한다.

- doGet( ) 메소드

```
public void doGet(HttpServletRequest req,
        HttpServletResponse resp
) throws ServletException, IOException
{
        // 내용 작성 부
}
```

req는 요청을 처리하기 위한 매개변수

resp는 응답을 처리하기 위한 매개변수

- doPost( ) 메소드

```
public void doPost(HttpServletRequest req,
        HttpServletResponse resp
) throws ServletException, IOException
{
        // 내용 작성 부
}
```

req는 요청을 처리하기 위한 매개변수

resp는 응답을 처리하기 위한 매개변수

하나의 서블릿은 get과 post방식을 모두 처리할 수도 있으며 방식에 따라 다른 기능을 제공하는 경우가 많아 doGet( ) 메소드와 doPost( ) 메소드 둘 다 오버라이딩을 하기도 한다.

3.3 출력 스트림

서블릿의 결과로 HTML 문서를 제공하기 위해 출력 스트림을 얻어와야 한다.

출력 스트림을 얻어오기 전에 HTML 문서에 한글이 포함되어 있다면 한글이 깨지지 않도록 캐릭터 셋을 UTF-8로 지정해 주어야 한다.

```
response.setContentType("text/html;charset=UTF-8"); // UTF-8 지정
PrintWriter out = response.getWriter(); // 출력 스트림 객체 얻기
```

4. 서블릿 API

4.1 HttpServletRequest (interface)

Javax.servlet.ServletRequest 인터페이스를 상속받은 후손 인터페이스로 HTTP Servlets에 대한 요청정보(request information)를 처리하는 메소드를 제공한다.

메소드명	내 용
getParameter(String)	client가 보내준 값이 저장된 명칭이 매개변수와 같은 명칭에 저장된 값을 불러오는 매소드
getParameterNames()	client가 보내준 값을 저장한 명칭을 불러오는 매소드
getParameterValues(String)	client가 보내준 값이 여러개일 경우 그 값을 배열로 불러오는 메소드
getParameterMap()	client가 보내준 값이 전체를 Map방식으로 불러오는 매소드
setAttribute(String, object)	request 객체에 전달하고 싶은 값을 String 이름으로 Object 저장하는 메소드
getAttribute(String)	매개변수와 동일한 객체 속성값 불어오는 메소드

removeAttribute(String)	request 객체에 저장되어 매개변수와 동일한 속성값 삭제하는 매소드
setCharacterEncoding(String)	전송받은 reques 객체의 값들의 CharaterSet 을 설정해주는 매소드
getRequestDispatcher(String)	컨테이너내에서 request, response 객체를 전송하여 처리할 컨포넌트(jsp 파일 등)를 불러오는 매소드로 forward()메소드와 같이 사용

4.2 HttpServletResponse (interface)

: javax.servlet.ServletResponse 인터페이스를 상속받은 후손 인터페이스로 요청에 대한 처리결과를 요청한 클라이언트에게 응답하기 위해 사용하는 객체이다.

메소드명	내 용
setContentType(String)	응답으로 작성하는 페이지의 MIME type 을 정하는 매소드
setCharacterEncoding(String)	응답하는 데이터의 CharacterSet 을 지정해주는 매소드
getWriter()	문자를 페이지에 전송을 위한 Stream 을 가져오는 매소드
getOutputStream()	byte 단위로 페이지에 전송을 위한 Stream 을 가져오는 매소드
sendResdirect(String)	client 가 매개변수의 페이지를 다시 서버에 요청하게 하는 매소드
getAttribute(String)	매개변수와 동일한 객체 속성값 불어오는 메소드
removeAttribute(String)	request 객체에 저장되어 매개변수와 동일한 속성값 삭제하는 매소드

setCharacterEncoding(String)	전송받은 reques 객체의 값들의 CharaterSet 을 설정해주는 매소드
getRequestDispatcher(String)	컨테이너내에서 request, response 객체를 전송하여 처리할 컨포넌트(jsp 파일 등)를 불러오는 매소드로 forward()메소드와 같이 사용

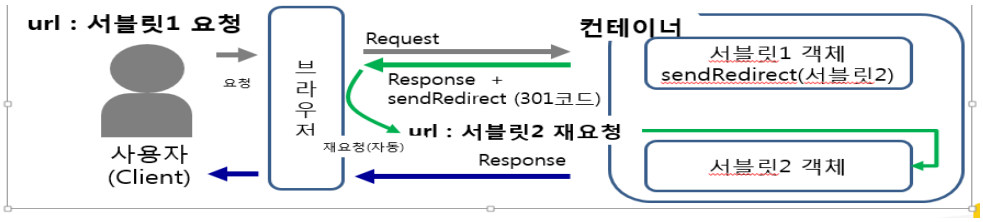
4.3 sendRedirect VS RequestDispatcher

sendRedirect

client 의 브라우저에게 매개변수로 등록한 페이지를 재요청하라고 응답해주는 메소드이다.(301/302 코드 전송)

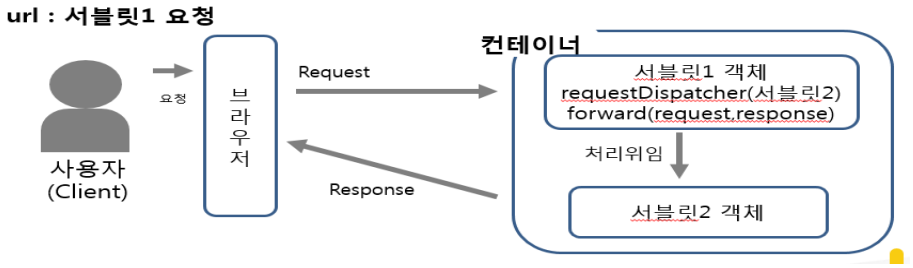
encodeRedirectURL

encodeRedirectURL 은 매개변수(URL)에 세션 ID 정보를 추가 재요청 처리하는 메소드이다. client 가 별도로 다른 페이지 요청을 안해도 url 주소(페이지)가 변경된다는 것이 특징이다..



RequestDispatcher() ~ forward()

컨테이너 내에서 처음 요청받은 페이지가 요청데이터(HttpServletRequest, HttpServletResponse)를 다른 페이지에 전송하여 처리를 요청을 하고 자신이 처리한 것처럼 하는 객체와 메소드이다. url 주소(페이지)가 변경되지 않는 것이 특징이다.



Chapter 9. LISTENER, FILTER, WRAPPER

1 서블릿 필터와 래퍼

서블릿은 웹에서 실행되는 프로그램이기 때문에 네트워크 통신의 사이 사이에서 특별한 동작을 만들어 낼 수 있다.

예를 들면, 데이터를 입력한 후 실제 저장하는 페이지로 넘어가기 전에 넘겨지는 데이터들에 대하여 charset을 UFT-8로 변경하는 작업이 가능하다. 또한, 세션이 만들어지거나 삭제될 때 이것을 감지하는 작업도 할 수 있다. 이렇게 여러가지 동작에 있어서 사이 사이에 끼워져서 실행되는 서블릿의 클래스를 필터라 부르고 동작이 발생할 때 감지하는 것을 이벤트라 부른다.

1.1 서블릿 필터

서블릿 필터란 말 그대로 여과기능을 수행하는 클래스를 말한다.

일반적으로 웹 프로그램이 메인 화면에서 여러 서비스를 연결하는데, 반드시 로그인이 필요하다고 한다면 각각의 페이지에 로그인 여부를 확인하는 코드를 일일히 넣어 주어야 한다. 하지만 필터 클래스를 사용하면, 메인 페이지에서 서비스 페이지로 넘어 가기 이전에 로그인 여부를 체크할 수 있어 코드의 중복이나 복잡성을 해소할 수 있다.

이러한 필터를 사용하려면 먼저 필터 클래스를 작성해야 하는데, 필터 클래스의 기본적인 구조는 다음과 같다.

```
public class MyFilter implements Filter {
  // 필터를 사용하려면 javax.servlet.Filter 인터페이스를 상속 받아 구현해야 한다.
  @Override
  public void init(FilterConfig arg0) throws ServletException {
    // 필터의 초기화 작업
  }
  @Override
  public void doFilter(ServletRequest request, ServletResponse response, FilterChain
chain)
      throws IOException, ServletException {
```

```
    // 필터가 수행될 때 해당 메소드를 실행한다.
    // 매개변수인 FilterChain은 다음에 존재하는 필터로 이동 시키는 역할을 한다.
    // 1. Request 파라미터를 이용하여 요청의 필터 작업 수행
    // 2. 체인의 다음 필터 처리
    chain.doFilter(request, response);
    /*
     * 3. response 를 이용하여 응답의 필터링 작업 수행 필터 메서드 내용부의 마지막
     *  코드는 현재까지 작업한 내용을 적용하고 연결된 페이지로 이동하도록 만들어 준다.
     *  이런 역할을 하는 메서드가 chain 객체의 doFilter()이다.
     *  세번째 매개변수인 FilterChain 클래스의 객체인 chain 을 이용해서
     *  다른 필터나 서블릿과 연결하는 코드를 반드시 작성해야 한다
     */
  }
  @Override
  public void destroy() {
    // 4. 필터가 없어질 때 호출된다
 }
}
```

1.1.1 서블릿 필터 사용하기

각 서비스를 수행하기 이전에 처리해야할 문제 중에는 문자셋 인코딩을 하나로 통일하는 문제도 존재한다. 이번 장에서는 이러한 문제를 예제를 통해 다루어 보도록 하겠다.

현재 myWeb 프로젝트에 인코딩 역할을 할 first.common.EncodingFilter.java 파일을 작성해보자.

first.common.filter.EncodingFilter.java

```
01 package first.common.filter;
02
03 import java.io.IOException;
04 import javax.servlet.Filter;
05 import javax.servlet.FilterChain;
```

```
06 import javax.servlet.FilterConfig;
07 import javax.servlet.ServletException;
08 import javax.servlet.ServletRequest;
09 import javax.servlet.ServletResponse;
10
11 public class EncodingFilter implements Filter {
12
13    @Override
14    public void init(FilterConfig arg0) throws ServletException {
15       System.out.println("EncodingFilter.init() 호출~!");
16    }
17
18    @Override
19    public void doFilter(ServletRequest request, ServletResponse response, FilterChain
20 chain)
21          throws IOException, ServletException {
22
23       request.setCharacterEncoding("UTF-8");
24       response.setContentType("text/html;charset=UTF-8");
25       System.out.println("----- EncodingFilter Work to Setting 'Charset UTF-8' -----");
26
27       chain.doFilter(request, response);
28    }
29
30    @Override
31    public void destroy() {
32       System.out.println("EncodingFilter.destroy() 호출~!!");
33    }
34 }
```

필터 클래스의 작성이 완료 되었다면 이를 web.xml 에 등록하여 웹 서버에서 필터를 초기화하고, 작업할 수 있게 해주어야 한다.

web.xml에 필터 클래스를 등록하는 작업은 다음과 같이 수행한다.

```
<filter>     <!-- 웹 어플리케이션에서 사용될 필터를 지정하는 역할 -->
    <filter-name> 필터 명 </filter-name>
    <filter-class> 필터 클래스 </filter-class>
    <init-param>    <!—가지고 시작할 파라미터 선언 -->
      <param-name> 파라미터 명 </param-name>
       <param-value>value</param-value>
      <!—param-name 은 param의 Key 역할, param-value 는 value 역할 -->
    </init-param>
  </filter>
  <filter-mapping> <!-- 특정 자원에 대해 어떤 필터를 사용할지를 지정 -->
    <filter-name> 필터 명 </filter-name>
    <url-pattern> *.jsp </url-pattern>  <!-- 특정 url 패턴 -->
    <!-- <servlet-name>servlet 명</servlet-name> -->
    <dispatcher>INCLUDE</dispatcher>
  </filter-mapping>
```

** param-name/value : 필터가 초기화될 때, 즉 필터의 init() 메소드가 호출될 때 전달되는 파라미터 값, 이는 서블릿의 초기화 파라미터와 비슷한 역할을 하며 주로 필터를 사용하기 전에 초기화해야 하는 객체나 자원을 할당할 때 필요한 정보를 제공하기 위해 사용된다.

** url-pattern : 클라이언트가 jsp 확장자를 갖는 자원을 요청할 경우 FilterName이 사용되도록 지정한다.

** servlet-name : 클라이언트가 요청한 특정 URI 에 대해서 필터링을 할 때 사용한다

** dispatcher : 실행되는 자원을 클라이언트가 요청한 것인지, 아니면 RequestDispather 의 forward() 를 통해서 이동한 것인지 아니면 include() 통해서 포함되는 것인지에 따라서 필터를 적용하도록 지정할 수 있다.

dispatcher의 값 분류	
REQUEST	클라이언트의 요청인 경우 필터 적용 (기본값)
INCLUDE	Include()를 통해서 포함하는 경우 필터를 적용
FORWARD	RequestDispatcher의 forward()를 통해서 제어를 이동하는 경우 필터를 적용
ERROR	error핸들러를 호출했을 경우 필터를 적용

위 항목을 참고하여 작성한 myWeb의 web.xml 이다.

myWeb/web/WEB-INF/web.xml	
01	<?xml version="1.0" encoding="UTF-8"?>

```
02  <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
03    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
04    xsi:schemaLocation=http://xmlns.jcp.org/xml/ns/javaee
05  http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd " id="WebApp_ID" version="3.1">
06   ... 중략 ...
07    <filter>
08      <filter-name>EncodingFilter</filter-name>
09      <filter-class>first.common.filter.EncodingFilter</filter-class>
10    </filter>
11    <filter-mapping>
12      <filter-name>EncodingFilter</filter-name>
13      <url-pattern>/*</url-pattern>
14    </filter-mapping>
15  </web-app>
```

이클립스에서 서버를 재실행하여 Console 창을 보면 해당 로그가 찍히는 것을 확인할 수 있다.

```
EncodingFilter.init() 호출~!
정보: Server startup in 5126 ms
----- EncodingFilter Work to Setting 'Charset UTF-8' -----
----- EncodingFilter Work to Setting 'Charset UTF-8' -----
정보: Stopping service Catalina
EncodingFilter.destroy() 호출~!!
```

1.2 서블릿 래퍼

먼저 설명했던 서블릿 필터를 자세히 살펴 보자. 필터는 클라이언트에게서 받은 request와 내보낼response에 대해서 UTF-8로 인코딩을 변경하는 작업은 있었지만, 실제 데이터를 가공하지는 못했다. 단순히 response로 받은 데이터를 **전달**만 했다는 뜻이다.

즉 서블릿 필터는 사용자의 요청을 대신 받을 수 있으나, 이에 대한 데이터 가공은 하지 못한다. 그렇기 때문에 만약 필터로 전달받은 데이터의 변경이 필요하다면 ServletRequestWrapper와 ServletResponseWrapper 클래스를 사용하여 데이터를 변경하는 작업을 해 주어야 한다.

HttpServletRequestWrapper 클래스와 HttpServletResponseWrapper 클래스는 모두 javax.servlet.http
패키지에 정의되어 있으며, 이 두 클래스는 각각 HttpServletRequest 인터페이스와 HttpServletResponse 인터페이스에 정의되어 있는 모든 메소드가 구현되어 있다. 따라서 필터를 통해 변경하고 싶은 정보가 있을 경우 그 정보를 추출하는 메소드를 쓰임에 맞게 오버라이딩하여 필터의 doFilter() 메소드에 넘겨주기만 하면 된다.

- ServletRequestWrapper 기본 구조

```
public class ParameterWrapper extends HttpServletRequestWrapper {
      public ParameterWrapper(HttpServletRequest wrapper) {
         super(wrapper);
        // 생성자는 전달받은 request 객체를 사용하여 만든다.
    }
      public String getParameter(String name) {
        // .... 원하는 작업 수행 ....
          return super.getParameter(name);
        }
      }
  }
```

- ServletResponseWrapper 기본 구조

```
public class ResWrapper extends HttpServletResponseWrapper {
            private HttpServletResponse response;
            public ResWrapper(HttpServletResponse response) {
                        super(response);
                        this.response = response;
            }
}
```

- 래퍼를 사용한 필터 클래스의 구조

```
public class MyFilter extends Filter {
  ... 중략 ...
  public void dofilter(ServletRequest request, ServletResponse response, FilterChain chain) {
```

```
    // 기존의 request/response로 Wrapper 를 생성하고, doFilter 메소드에 사용하면 된다.
    MyRequestWrapper requestWrapper =
    new MyRequestWrapper((HttpServletRequest) request);
    chain.doFilter(requestWrapper, response);
  }
}
```

1.2.1 서블릿 래퍼 사용하기

위의 구조를 참고하여 로그인 시 비밀번호를 암호화 하는 필터-래퍼 클래스를 작성해 보자.

first.loginSv.controller.LoginServlet.java

```
01  package first.loginSv.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import javax.servlet.ServletException;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.HttpServlet;
08  import javax.servlet.http.HttpServletRequest;
09  import javax.servlet.http.HttpServletResponse;
10
11  @WebServlet("/login.do") // 웹에서 접근 가능한 맵핑 주소 설정
12  public class LoginServlet extends HttpServlet {
13    private static final long serialVersionUID = 1L;
14
15      public LoginServlet() {
16          super();
17      }
18      protected void doGet(HttpServletRequest request, HttpServletResponse response)
19  throws ServletException, IOException {
20  // 암호화를 위한 페이지를 PrintWriter에 담아 구성한다.
21      PrintWriter out = response.getWriter();
```

```
22      out.println("<html><head>");
23      out.println("<title>필터 & 래퍼 테스트 </title></head>");
24      out.println("<body><h2>필터 & 래퍼 테스트하기</h2>");
25      out.println("<form action='loginAck.do' method='post'>");
26      out.println("ID : <input type='text' name='id'><br>");
27      out.println("비밀번호 : <input type='password' name='pwd'><br>");
28      out.println("<input type='submit' value='확인하기'>");
29      out.println("</form>");
30      out.println("</body></html>");
31
32      out.flush();
33      out.close();
34    }
35    protected void doPost(HttpServletRequest request, HttpServletResponse response)
36  throws ServletException, IOException {
37      doGet(request, response);
38    }
39  }
```

first.loginSv.controller.LoginAckServlet.java

```
01  package first.loginSv.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import javax.servlet.ServletException;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.HttpServlet;
08  import javax.servlet.http.HttpServletRequest;
09  import javax.servlet.http.HttpServletResponse;
10
11  /**
12   * Servlet implementation class LoginAckServlet
13   */
14  @WebServlet("/loginAck.do") // 웹에서 접근 가능한 맵핑 주소 설정
```

```
15  public class LoginAckServlet extends HttpServlet {
16      private static final long serialVersionUID = 1L;
17
18      public LoginAckServlet() {
19          super();
20      }
21      protected void doPost(HttpServletRequest request, HttpServletResponse response)
22  throws ServletException, IOException {
23          //doGet(request, response);
24          PrintWriter out = response.getWriter();
25          String id = request.getParameter("id");
26          String pwd = request.getParameter("pwd");
27          out.println("<html>₩r₩n<head>₩r₩n"
28              + "<meta charset=₩"UTF-8₩">₩r₩n" +
29              "<title>필터와 래퍼 확인 Page</title>₩r₩n" +
30              "</head>₩r₩n<body>"
31              + "<h1>필터 & 래퍼 확인하기</h1><hr><br>");
32          out.println("전달된 ID : "+id+"<br>");
33          out.println("전달된                                비밀번호                                :
34  "+(String)request.getAttribute("pwdChk")+"<br>");
35          out.println("암호화된 비밀번호 : "+pwd+"<br>");
36          out.println("<a href='index.html'>이전으로 가기</a>");
37          out.println("</body></html>");
38          out.flush();
39          out.close();
40      }
41  }
42
```

first.loginSv.filter.MyFilter.java

```
01  package first.loginSv.filter;
02
03  import java.io.IOException;
04  import first.loginSv.wrapper.LoginWrapper;
```

```
05 import javax.servlet.Filter;
06 import javax.servlet.FilterChain;
07 import javax.servlet.FilterConfig;
08 import javax.servlet.ServletException;
09 import javax.servlet.ServletRequest;
10 import javax.servlet.ServletResponse;
11 import javax.servlet.http.HttpServletRequest;
12
13 public class MyFilter implements Filter {
14     @Override
15     public void init(FilterConfig arg0) throws ServletException {
16       System.out.println("MyFilter.init() 호출~!");
17     }
18     @Override
19     public void doFilter(ServletRequest request, ServletResponse response, FilterChain
20 chain)
21         throws IOException, ServletException {
22
23       HttpServletRequest h_request = (HttpServletRequest) request;
24       h_request.setAttribute("pwdChk", h_request.getParameter("pwd"));
25       LoginWrapper lw = new LoginWrapper(h_request);
26       chain.doFilter(lw, response);
27     }
28     @Override
29     public void destroy() {
30       System.out.println("Myfilter.destroy() 호출!!");
31     }
32 }
```

first.loginSv.wrapper.LoginWrapper.java

```
01 package first.loginSv.wrapper;
02
03 import java.nio.charset.Charset;
04 import java.security.MessageDigest;
```

```
05  import java.util.Base64;
06  import javax.servlet.http.HttpServletRequest;
07  import javax.servlet.http.HttpServletRequestWrapper;
08
09  public class LoginWrapper extends HttpServletRequestWrapper {
10    public LoginWrapper(HttpServletRequest request) {
11      super(request);
12    }
13    @Override
14    public String getParameter(String name) {
15      if(name != null && name.equals("pwd"))
16        name = getSha512(super.getParameter(name)); // SHA-512 암호화 메소드
17      else name = super.getParameter(name);
18      return name;
19    }
20    private static String getSha512(String pwd) {
21          try {  // SHA-512 암호화를 위한 로직 구현
22              MessageDigest md = MessageDigest.getInstance("SHA-512");
23              byte[] bytes = pwd.getBytes(Charset.forName("UTF-8"));
24              md.update(bytes);
25              return Base64.getEncoder().encodeToString(md.digest());
26          } catch (Exception e) {
27              System.out.println("Sha512 error.");
28              e.printStackTrace();
29              return null;
30          }
31      }
32  }
```

myWeb/web/WEB-INF/web.xml

```
01  <?xml version="1.0" encoding="UTF-8"?>
02  <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
03  ... 중략 ...
04    <filter>
05      <filter-name>MyFilter</filter-name>
```

```
06      <filter-class>first.loginSv.filter.MyFilter</filter-class>
07    </filter>
08    <filter-mapping>
09      <filter-name>MyFilter</filter-name>
10      <url-pattern>*.do</url-pattern>
11    </filter-mapping>
12  </web-app>
```

'LoginServlet' 에서 작성된 Form을 통해 전달되는 'id'와 'pwd'는 MyFilter를 거쳐 'loginAckServlet'으로 이동 된다. 또한 전달 과정에서 pwd는 래퍼 클래스에서 SHA-512 암호화 알고리즘을 통해 데이터가 가공되어 결과 값을 찍게 된다.

1.3 서블릿 이벤트

서블릿은 다양한 시점에 발생되는 이벤트와 이벤트를 처리하기 위한 인터페이스를 정의하고 있다. 이들 이벤트와 인터페이스를 이용하면 웹 어플리케이션에서 필요로 하는 데이터의 초기화나 요청 처리 등을 추적할 수 있으며, 데이터 저장 영역(application, session, request)에 데이터가 들어가고 나가는 혹은 그 객체가 생성되고 소멸되는 일련의 작업들에 대해 컨트롤할 수 있다.

이런 이벤트에 대해서 미리 web.xml 파일에 등록해두면 웹 서버는 초기 구동 시에 해당 이벤트를 대기 상태로 두었다 이벤트 발생 시 자동으로 이벤트를 감지하고 우리가 지정한 클래스 내의 메서드를 실행해준다. 때문에 서블릿에서 이벤트 클래스를 실행되게 하려면 무엇보다 web.xml 파일의 설정이 중요하다. 이러한 이벤트는 외부의 변화보다는 내부의 변화를 처리하는 경우가 많다.

- web.xml에 이벤트 리스너 추가하기

<listener>태그는 WAS가 시작되거나 종료될 때 실행되어야 하므로, web.xml 파일의 최상위에 작성해야 한다.

```
<web-app>
  <listener>
      <display-name>이벤트 리스너 명</display-name>
      <listener-class>리스너 클래스</listener-class>
  </listener>
  ... 중략 ...
</web-app>
```

그럼 우리가 설정하여 제어할 수 있는 이벤트 리스너의 종류에 대해서 알아보자. 이벤트는 발생 시점에 따라 종류가 나뉘는데, 세부사항은 다음과 같다.

이벤트 리스너	ServletContextListener
발생시점	웹 어플리케이션의 시작과 종료시 자동으로 발생
메소드 종류	contextInitialized(ServletContextEvent sce) : void 웹 컨테이너가 처음 구동될 때 실행되는 메소드
	contextDestoryed(ServletContextEvent sce) : void 웹 컨테이너가 종료될 때 실행되는 메소드

이벤트 리스너	ServletContextAttributeListener
발생시점	ServletContext 객체에 속성이 추가, 삭제, 수정될 때 발생
메소드 종류	attributeAdded(ServletContextAttributeEvent scae) : void 새로운 속성 값이 추가될 때 실행되는 메소드
	attributeRemoved(ServletContextAttributeEvent scae) : void 속성 값이 제거될 때 실행되는 메소드
	attributeReplaced(ServletContextAttributeEvent scae) : void 속성 값이 변경될 때 실행되는 메소드

이벤트 리스너	HttpSessionListener
발생시점	HttpSession 객체가 생성되거나 소멸될 때 발생
메소드 종류	sessionCreated(HttpSession se) : void 세션이 생성되었을 경우 실행되는 메소드
	sessionDestoryed(HttpSession se) : void 세션이 무효화 되었을 경우 실행되는 메소드

이벤트 리스너	HttpSessionAttributeListener
발생시점	HttpSession 객체에 속성이 추가, 삭제, 수정될 때 발생
메소드 종류	attributeAdded(HttpSessionBindingEvent e) : void 세션에 새로운 속성 값이 추가될 때 실행되는 메소드
	attributeRemoved(HttpSessionBindingEvent e) : void 세션의 속성 값이 제거될 때 실행되는 메소드
	attributeReplaced(HttpSessionBindingEvent e) : void 세션의 속성 값이 변경될 때 실행되는 메소드

web.xml에 한 개이상의 리스너가 등록되어 있는 경우, contextInitialized() 메서드는 등록된 순서대로 실행되고 contextDestoryed() 메서드는 등록된 반대 순서대로 실행된다.

1.3.1 서블릿 이벤트 리스너 사용하기

웹서버를 운영하다보면, 유저가 로그인하거나 로그아웃하면서 세션을 생성, 만료하는 시점을 파악해야하는 상황이 종종 발생하는데, 이번 예제에서는 이러한 세션을 감지할 수 있는 간단한 이벤트를 만들어 보도록 하자.

먼저 HttpSessionListener와 HttpSessionAttributeListener를 상속받는 Listener 클래스를 하나 만든다.

first.member.listener.SessionListener.java

```
01  package first.member.listener;
02
03  import javax.servlet.http.HttpSessionAttributeListener;
04  import javax.servlet.http.HttpSessionBindingEvent;
05  import javax.servlet.http.HttpSessionEvent;
06  import javax.servlet.http.HttpSessionListener;
07
08  public class SessionListener implements HttpSessionListener,
09  HttpSessionAttributeListener {
10    /* .... 구현해야할 메소드 중 사용하는 부분만 표현하였음 .....*/
11      @Override
12      public void attributeAdded(HttpSessionBindingEvent e) {
13        String name = (String)e.getSession().getAttribute("name");
14        System.out.println(name+" 님이 접속하셨습니다.");
15      }
16      @Override
17      public void sessionCreated(HttpSessionEvent e) {
18        System.out.println(e.getSession().getId()+"세션이 생성되었습니다.");
19      }
20      @Override
21      public void sessionDestroyed(HttpSessionEvent e) {
22        System.out.println(e.getSession().getId()+"세션이 파괴되었습니다.");
23      }
    }
```

essionListener를 만들었다면, 이제 web.xml에 이벤트를 등록하여야 한다.

myWeb/web/WEB-INF/web.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
03 <listener>
04         <listener-class> first.member.listener.SessionListener </listener-class>
05 </listener>
06 <display-name>myWeb</display-name>
07 <welcome-file-list>
08     <welcome-file>index.html</welcome-file>
09   </welcome-file-list>
10 </web-app>
```

web.xml에 등록을 완료하였다면, 이제 실제 사용할 서블릿과 시작 화면을 만들어보자.

first.member.controller.ListenerServlet.java

```
01 package first.listener.controller;
02
03 import java.io.IOException;
04 import java.io.PrintWriter;
05
06 import javax.servlet.ServletException;
07 import javax.servlet.annotation.WebServlet;
08 import javax.servlet.http.HttpServlet;
09 import javax.servlet.http.HttpServletRequest;
10 import javax.servlet.http.HttpServletResponse;
11 import javax.servlet.http.HttpSession;
12
13 /**
14  * Servlet implementation class ListenerServlet
15  */
16 @WebServlet("/listener.li")
17 public class ListenerServlet extends HttpServlet {
18     private static final long serialVersionUID = 1L;
19
20     public ListenerServlet() {
```

```
21          super();
22      }
23      protected void doGet(HttpServletRequest request, HttpServletResponse response)
24  throws ServletException, IOException {
25          HttpSession session = request.getSession(false);
26          PrintWriter out = response.getWriter();
27
28          // 기존에 세션이 생성되어 있다면 만료시킨다.
29          if(session != null){
30            session.invalidate();
31          }
32
33          out.println("<html>₩r₩n<head>₩r₩n"
34              + "<meta charset=₩"UTF-8₩">₩r₩n" +
35              "<title>Listener Test Page</title>₩r₩n" +
36              "</head>₩r₩n<body>"
37              + "<h1>리스너 테스트 하기</h1><hr><br>");
38          out.println("<form action='listenerAck.li' method='post'>");
39          out.println("이름 : <input type='text' name='name'><br>");
40          out.println("<input type='submit' value='확인하기'>");
41          out.println("</form>");
42          out.println("<hr>");
43          out.println("<a href='index.html'>메인으로 가기</a>");
44          out.println("</body></html>");
45          out.flush();
46          out.close();
47      }
48      protected void doPost(HttpServletRequest request, HttpServletResponse response)
49  throws ServletException, IOException {
50          doGet(request, response);
51      }
    }
```

first.member.controller.ListenerAckServlet.java

```
01  package first.listener.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05
06  import javax.servlet.ServletException;
07  import javax.servlet.annotation.WebServlet;
08  import javax.servlet.http.HttpServlet;
09  import javax.servlet.http.HttpServletRequest;
10  import javax.servlet.http.HttpServletResponse;
11  import javax.servlet.http.HttpSession;
12
13  /**
14   * Servlet implementation class listenerAckServlet
15   */
16  @WebServlet("/listenerAck.li")
17  public class ListenerAckServlet extends HttpServlet {
18      private static final long serialVersionUID = 1L;
19
20      public ListenerAckServlet() {
21          super();
22      }
23     protected void doPost(HttpServletRequest request, HttpServletResponse response)
24  throws ServletException, IOException {
25
26          String name = request.getParameter("name");
27
28          HttpSession session = request.getSession();
29          session.setAttribute("name", name);
30
31          PrintWriter out = response.getWriter();
32
33          out.println("<html>₩r₩n<head>₩r₩n"
```

```
34              + "<meta charset=₩"UTF-8₩">₩r₩n" +
35              "<title>Listener Test Page</title>₩r₩n" +
36              "</head>₩r₩n<body>"
37              + "<h1>리스너 테스트 확인</h1><hr><br>");
38          out.println("전달된 이름 : "+name+"<br>");
39          out.println("이벤트 확인을 위해 서버 콘솔을 확인해 보세요!<br>");
40          out.println("<a href='listener.li'>세션 종료</a>");
41          out.println("</body></html>");
42
43          out.flush();
44          out.close();
45      }
46  }
```

myWeb/web/index.html

```
01  <!DOCTYPE html>
02  <html>
03  <head>
04  <meta charset="UTF-8">
05  <title>First Servlet Project</title>
06  </head>
07  <body>
08  <h1>서블릿 테스트 하기</h1>
09  <hr>
10  <ol>
11              <li><a href="login.do">필터와 래퍼 테스트</a></li>
12              <li><a href="listener.li">리스너 테스트</a></li>
13  </ol>
14  </body>
15  </html>
```

이제 실행해서 간단한 이름 주고 받기를 통한 세션 연결과 만료를 확인해보고, 이클립스의 콘솔창에서 찍히는 로그를 확인해보자. 그럼 세션이 생성되거나 만료될 때, 속성값이 추가 될 때 각각 리스너가 실행되는 것을 볼 수 있다.

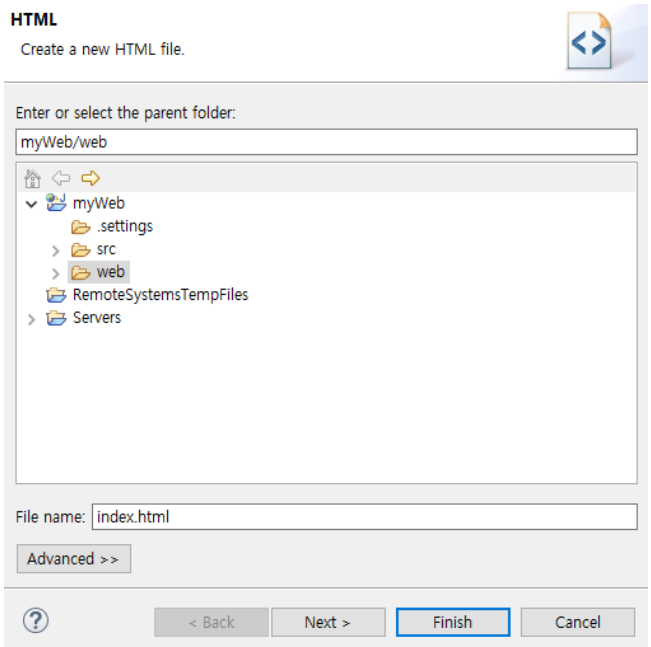
Chapter 10. JSP Element

1 기본 JSP 프로그램 만들기

JSP를 이용해 기본 JSP 프로그램을 만들어 보자.

먼저 index.html로 목차역할을 할 인트로 페이지를 만들자.

① web 폴더 밑에 index.html 파일을 하나 만든다.

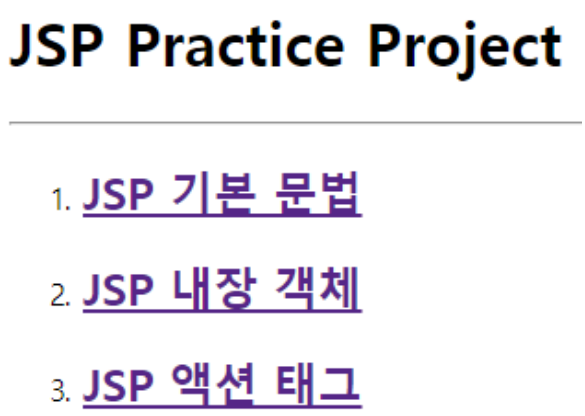


② 링크 연결을 위한 HTML 코드를 작성하자.

myWeb/web/index.html

```
01  <!DOCTYPE html>
02  <html>
03  <head>
04  <meta charset="UTF-8">
05  <title>First JSP Project</title>
06  </head>
07  <body>
08  <h1>JSP Practice Project</h1>
09  <hr>
10    <ol>
11      <li><h2><a href="">JSP 기본 문법</a></h2></li>
12      <li><h2><a href="">JSP 내장 객체</a></h2></li>
13      <li><h2><a href="">JSP 액션 태그</a></h2></li>
14    </ol>
15  </body>
16  </html>
```

③ 서버 실행 후 http://localhost:8080/test/ 로 접속하여 index.html에 정상적으로 접근이 가능한 지 확인.

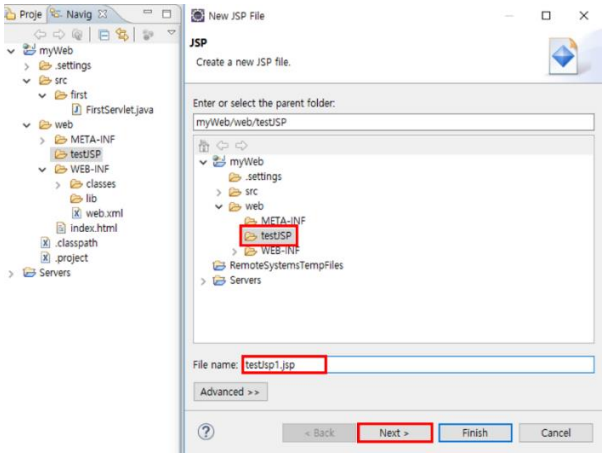


이제 JSP 파일을 하나씩 생성하여 링크를 연결해주면 되겠다.

① 만들어둔 프로젝트에 [File – New – JSP FIle] 선택

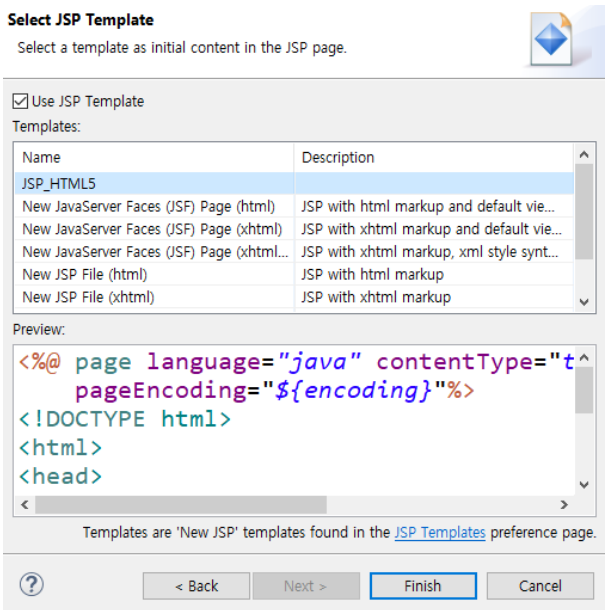
JSP File 이 안보인다면 Other 메뉴에서 jsp 검색

파일이름을 testJsp1.jsp 로 지정하여 생성



** 실습 코드 접근이 쉽도록 testJSP라는 별도 폴더를 만들어 정리하였다.

② JSP 기본 템플릿은 원하는 양식으로 설정한다. 만약 원하는 양식이 없다면, JSP Templates를 통해 개인 용도의 템플릿 제작도 가능하다.



③ 다음과 같이 testJsp1.jsp의 내부 코드를 작성하고 index.html 의 JSP 기본 문법 목차의 링크 경로를 수정 해주자.

myWeb/web/testJSP/testJsp1.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>Hello JSP!!</title>
08 </head>
09 <body>
10 <h3>Hello JSP!!</h3>
11 <a href="../index.html">뒤로가기</a>
12 </body>
13 </html>
```

```
    <a href="">JSP 기본 문법</a>
➔   ><a href="./testJSP/testJsp1.jsp">JSP 기본 문법</a>
```

④ 브라우저를 이용해 http://localhost:8080/test/testJSP/testJsp1.jsp로 접속하면 결과 확인이 가능하다



* JSP파일로 만들어진 페이지는 웹 서버 내부적으로 서블릿 코드를 생성한다
이클립스로 작업한 경우 (workspace폴더)₩.metadata₩.plugins₩org.eclipse.wst.server.core ₩tmp0₩work₩Catalina₩localhost₩test₩org₩apache₩jsp₩ 내에 생성된 것을 확인할 수 있다.

2 JSP 기본 문법

JSP만의 문법에 대해 알아보도록 한다

2.1 주석

JSP에는 HTML, JSP, JAVA 코드 3가지가 섞여 있으므로 주석 또한 3가지를 적용할 수 있다

2.1.1 HTML 주석

웹 브라우저가 해석하지 않도록 하는 주석

```
<!-- 주석 -->
```

2.1.2 JSP 주석

JSP가 서블릿 코드를 변환될 때 포함되지 않도록 하는 주석

```
<%-- 주석--%>
```

2.1.3 Java 주석

JSP내의 자바 코딩 영역에서 사용되는 주석

```
// 한 줄 주석
/* 문장 주석 */
```

2.2 JSP 기본 태그

종류	설명	형식
주석(Comments tag)	컴파일 시 주석으로 취급하기 위한 코드	<%-- --%>
지시자(directive tag)	JSP 페이지 전체에 사용하기 위한 설정을 작성	<%@ %>
선언(declaration)	변수와 메소드를 선언	<%! %>
스크립트릿(scriptlet)	자바 코드를 기술	<% %>
표현식(expression)	계산식이나 함수 리턴을 문자열 형태로 출력	<%= %>

2.2.1 스크립트릿, Scriptlet

JSP에는 자바 코드와 HTML 코드가 섞여 있게 된다. HTML 태그는 정적인 페이지를 디자인하게 되고 자바 코드는 웹 서버에서 처리해야 할 문장들을 기술하게 된다. 이 때 사용되는 자바 언어를 서버 스크립트 언어라고 하며, JSP문서 내부에서 자바 코드를 사용하기 위해 제공되는 것이 스크립트릿 태그이다.

스크립트릿 태그 내에 정의된 내용은 JSP코드가 서블릿으로 변환될 때 _jspService( )메소드 내부로 들어가게 된다.

- 스크립트릿 태그 기본 형식

```
<%
  웹 서버에서 실행되어야하는 자바 코드
%>
```

2.2.2 선언문

JSP 페이지에서 변수나 메소드를 정의하기 위해 선언문을 사용한다

서블릿으로 변환될 때 스크립트릿과는 달리 클래스 영역에 정의된다

- 선언문 태그 기본 형식

```
<%!
  변수 선언, 메소드 정의
%>
```

1.2.3 표현식

브라우저에 HTML 형태로 결과를 출력하기 위해서 간단한 방식으로 제공되는 태그

표현식으로 기술된 내용은 서블릿 컨테이너에서 스크립트 내의 out.print( ) 형식으로 변환된다

- 표현식 태그 기본 형식

```
<%=변수 %>
<%=수식 %>
<%=메소드호출 %>
```

2.3 JSP 지시자

JSP 페이지가 실행되면서 지시자에 설정된 내용을 JSP 페이지 전체에서 사용 가능하도록 한다

지시자에는 page, include, taglib 3가지가 있으며, JSP 문서의 제일 위 부분에 작성한다

- 지시자 태그 기본 형식

```
<%@ 지시자 속성="값" ... %>
```

- 지시자 종류와 사용 목적

종류	목적
page	해당 JSP 페이지 전반적인 환경 설정
include	페이지에 다른 파일의 내용을 삽입
taglib	태그 라이브러리에서 기능을 사용할 수 있도록 제공

2.3.1 page 지시자

JSP 페이지에 여러 정보를 나타내기 위해 사용되는 JSP 문법

- language 속성

JSP에서 사용할 언어를 결정

```
<%@ page language="java" %>
```

JSP에서는 서버 스크립트 언어로 Java만 사용 가능하다. 따라서 language 속성으로 java만 기술할 수 있으며 디폴트로 java가 설정되어 있다.

- extends 속성

jsp페이지가 상속받을 상위 클래스를 기술

일반적으로 서블릿 컨테이너가 알아서 처리하며 개발자가 지정할 필요가 없다

```
<%@ page extends="javax.servlet.jsp.HttpJspBase" %>
```

- import 속성

java 소스와 마찬가지로 import할 클래스의 풀네임을 기술한다

JSP는 기본적으로 javax.servlet.*, javax.servlet.hhtp.*, javax.servlet.jsp.* 3개의 패키지 내용을 import하고 있다

```
<%@ page extends="java.util.Calendar" %>
```

- session 속성

true / false 로 설정하며 세션을 사용할지 말지를 결정하는 속성

HttpSession 객체를 사용할 것인지 결정된다

기본적으로 true로 설정되어 있으며 세션을 사용하지 않으려면 false를 지정한다

```
<%@ page session="false" %>
```

- buffer 속성

JSP페이지의 출력 버퍼 크기를 설정, 디폴트는 8kb

none으로 설정될 경우 JspWriter 객체를 이용한 출력 시 버퍼를 사용하지 않는다

```
<%@ page buffer="8kb" %> <%-- 디폴트--%>
```

- autoFlush 속성

자동으로 버퍼를 비울지 설정

autoFlush를 false로 지정할 경우 버퍼가 가득 찰 경우 에러가 발생할 수 있다

autoFlush를 false로 지정하려면 buffer속성이 none이어서는 안 된다

```
<%@ page autoFlush="false" %>
```

autoFlush를 이용하지 않고 out 객체의 flush( ) 메소드를 이용할 수도 있다

- isThreadSafe 속성

프로세스를 스레드로 처리하여 수행속도와 효율성을 개선할 수 있지만 하나의 자원에 스레드들이 동시에 접근할 경우 무결성을 해칠 수 있게 된다

이런 상황을 대비하여 스레드들이 각각의 페이지에서 안전하게 동작(Thread Safe)할 수 있도록 설정한다

```
<%@ page isThreadSafe="false" %>
```

false로 지정할 경우 클라이언트의 요청에 한꺼번에 스레드를 만들어 처리하므로 객체들을 따로 동기화 시켜야한다

- info 속성

JSP 페이지에 대한 간략한 설명을 기록하는 용도

페이지에 대한 정보를 알려주는 역할을 한다

```
<%@ page info="페이지 정보" %>
```

- errorPage 속성

JSP 페이지에서 에러가 발생했을 때 보여줄 에러 페이지를 설정

```
<%@ page errorPage="error.jsp" %>
```

- isErrorPage 속성

현재 페이지가 에러페이지인지 아닌지 설정하기 위한 속성

true로 설정되면 exception 객체를 사용하여 예외의 원인을 확인할 수 있게 된다

기본값은 false

```
<%@ page isErrorPage="true" %>
```

간단한 에러 확인 예제

myWeb/web/testJSP/errorTest.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>에러 확인 하기</title>
08 </head>
09 <body>
10             <form action="errorConfirm.jsp" method="post">
11           사원명 : <input type="text" name="sname"><br>
12           나이 : <input type="text" name="sage"><br><br>
13           <input type="submit" value="제출하기">    
14           <input type="reset" value="재작성하기">
15             </form>
16 </body>
17 </html>
```

myWeb/web/testJSP/errorConfirm.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8" errorPage="../view/error/errorPage.jsp" %>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>사원 정보 입력 완료</title>
08 </head>
09 <%
10 String name = request.getParameter("sname");
11 int age = Integer.parseInt(request.getParameter("sage"));
12 %>
13 <body>
14 <table>
15         <tr>
16           <td>사원명 :</td>
17           <td><%= name %></td>
18         </tr>
19         <tr>
20           <td>나 이 :</td>
21           <td><%= age %></td>
22         </tr>
23 </table>
24 <a href="../index.html">뒤로가기</a>
25 </body>
26 </html>
```

myWeb/web/view/error/errorPage.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8" isErrorPage="true" %>
03 <!DOCTYPE html>
04 <html>
05 <head>
```

```
06  <meta charset="UTF-8">
07  <title>Error Page</title>
08  </head>
09  <body>
10      Error 발생!!<br>
11      입력 사항을 확인해주세요!! <hr>
12      ERROR Info :: <%= exception.getClass().getName() %>
13  <a href="../index.html">뒤로가기</a>
14  </body>
15  </html>
```

- contentType 속성

JSP페이지의 MIME(Multipurpose Internet Mail Extensions) 타입을 결정한다

MIME는 서버가 클라이언트에게 보낼 내용에 대하여 미리 보내는 내용이며 헤더에 포함된다

웹 서버가 브라우저에 전송할 페이지가 html 일 경우 "text/html"을 지정

```
<%@ page contentType="text/html" %>
```

한글 HTML 문서로 전달할 때는 인코딩 방식 또한 UTF-8로 지정한다

```
<%@ page contentType="text/html; charset=UTF-8" %>
```

2.3.2 include 지시자

현재 페이지에 다른 HTML문서나 JSP 페이지의 내용을 삽입할 때 사용

include는 file이 유일한 속성

- file 속성 : 삽입할 파일의 URL을 기술

웹 사이트에서 공통적으로 보여주어야 할 내용을 include 지시자를 통해 표현한다

```
<%@ include file="파일명" %>
```

2.3.3 taglib 지시자

JSP에서 사용 가능한 태그 라이브러리를 지정

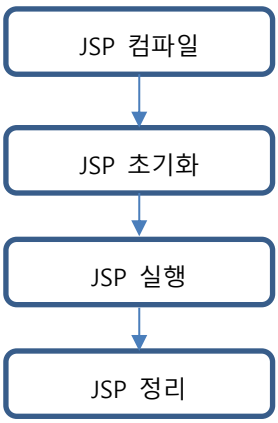
JSP 내의 불필요한 자바 코드를 줄일 수 있다

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

JSP 라이프 사이클

JSP의 라이프 사이클(생명 주기)는 서블릿과 유사

컴파일 — 초기화 — 실행 — 정리, 4단계를 거친다



2.4 JSP 컴파일

클라이언트(브라우저)에서 JSP를 요구할 경우 JSP엔진이 먼저 페이지를 컴파일 할 필요가 있는지 확인한다

JSP가 수정 된 일이 있다면 컴파일 수행

컴파일은 JSP구문분석, 서블릿으로 JSP 변환, 서블릿 컴파일 3단계를 거친다

2.5 JSP 초기화

컨테이너가 JSP 파일을 로드한 후 서비스를 제공하기 전에 요청 jspInit( )메소드를 호출

2.6 JSP 실행

JSP 페이지 초기화가 완료되면, JSP 엔진은 _jspService( ) 메소드를 호출

2.7 JSP 정리

JSP 컨테이너에 의해 제거될 때 발생

jspDestroy( )메소드가 수행되며 서블릿의 destroy( )에 해당한다

- JSP 기본 문법 표현을 위한 소스 코드

myWeb/web/testJSP/testJsp1.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%> <!—JSP 지시자 태그 →
03 <%! // JSP 선언 태그
04             int max = 10;
05             int sum = 0; /* 자바 소스 코드 주석 */
06 %>
07 <!DOCTYPE html>
08 <html>
09   <head>
10    <meta charset="UTF-8">
11    <title>Hello JSP!!</title>
12   </head>
13   <body>
14    <h3>Hello JSP!!</h3>
15    <hr>
16    <h3>1부터 10까지 합계 출력</h3>
17 <% for(- 95 -n ti = 0; i < max; i++){
18             sum += i; // 스크립트릿 태그
19 } %>
20 정답 : <%= sum %> <!—표현식 태그 →
21    <br><br><br>
22    <a href="../index.html">뒤로가기</a>
23 <%@ include file="footer.jsp" %>  <!—include 지시자로 footer.jsp 삽입 →
24   </body>
25 </html>
```

myWeb/web/testJSP/footer.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02 pageEncoding="UTF-8"%>
03 <hr><footer>
04         include 지시자로 추가된 footer.jsp 파일입니다.<br>
05         Copyright by KH. All right reserved.
06 </footer>
```

Chapter 11. JSP 내장 객체

1 내장 객체

JSP 내장 객체는 JSP 페이지에서 객체를 생성하는 과정 없이 바로 사용할 수 있는 객체들을 뜻한다. JSP가 서블릿 파일로 변환될 때 서블릿 컨테이너가 자동으로 생성을 해준다.

myWeb/web/testJSP/testJsp2.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>JSP 내장 객체 Test</title>
08 </head>
09 <body>
10 <h3>JSP 내장 객체 Test</h3>
11 <hr>
12 <h4>1. out.print()</h4>
13 result =
14 <%
15             out.print("Hello JSP");
16 %>
17 <br><br><br>
18 <a href="../index.html">뒤로가기</a>
19 </body>
20 </html>
```

test.jsp 소스의 15번째 줄 처럼 out객체를 사용할 때 별도의 객체 생성과정 없이 사용 가능한 것이 내장 객체이다. 반면, 서블릿에서 out객체를 사용하기 위해서는

```
PrintWriter out=response.getWriter( );
```

와 같은 객체 생성이 필요하다

- 내장 객체의 용도에 따른 4가지 분류

내장 객체 분류	내장 객체
입출력 관련 객체	resquest response out
서블릿 관련 객체	page config
외부 환경 정보를 제공하는 객체	session application pageContext
예외 관련 객체	exception

2 out 내장 객체

서버에서 클라이언트로 열려있는 출력 스트림

JSP의 실행결과를 클라이언트의 브라우저로 출력할 때 효율적으로 사용할 수 있다

출력 전담 내장 객체

3 request 내장 객체

클라이언트의 요청과 관련된 정보가 저장되는 내장 객체

request 객체를 파악하면 클라이언트에서 서버로 전송된 데이터를 알 수 있다

- 클라이언트의 요청이 있을 때 관련된 정보들을 알 수 있는 메소드

반환타입	메소드 명	설명
String	getRemoteAddr( )	웹 서버에 연결한 클라이언트의 IP주소를 구함
long	getContentLength( )	클라이어트가 전송한 요청정보의 길이를 구함
String	getCharacterEncoding( )	클라이언트가 요청 정보를 전송할 때 사용한 캐릭터셋을 구함
String	getContentType( )	클라이언트가 요청 정보를 전송할 때 사용한 컨텐트 타입을 구함
String	getProtocol( )	클라이언트가 요청한 프로토콜을 구함
String	getMethod( )	데이터 전송 방식을 구함
String	getRequestURL( )	요청 URL을 구함
String	getContextPath( )	JSP페이지가 속한 웹어플리케이션의 컨텍스트패스 반환
String	getServerName( )	연결할 때 사용한 서버이름을 구함
int	getServerPort( )	사용중인 프로토콜을 구함

- request 객체의 요청 파라미터 관련 메소드

반환타입	메소드 명	설명
String	getParameter(String name)	지정한 이름의 파라미터가 있을 경우 첫 번째 파라미터의 값을 구함
String[ ]	getParameterValues(String name)	지정한 이름의 파라미터가 있을 경우 지정한 이름을 가진 파라미터의 모든 값을 String[ ]으로 구함
Enumeration	getParameterNames( )	모든 파라미터의 이름을 구함
Map	getParameterMap( )	전송한 파라미터를 맵 형식으로 구함

4 response 내장 객체

클라이언트에 대한 응답을 처리하는 내장 객체

실행 결과를 브라우저로 되돌려 줄 때 사용

리다이렉트 기능을 많이 사용

*** JSP 페이지 이동 방법 1. 리다이렉트**

브라우저의 URL을 변경하도록 하여 페이지를 이동하는 방식

request와 response 객체가 유지되지 않는다

response 객체의 sendRedirect( ) 메소드를 사용하여 페이지 이동

*** JSP 페이지 이동 방법 2. 포워딩**

브라우저의 URL을 유지하며 문서의 내용(페이지)만 이동하는 방식

request 객체가 유지된다

requestDispatcher 객체로 접근한 후 forward( )메소드를 호출하여 페이지 이동

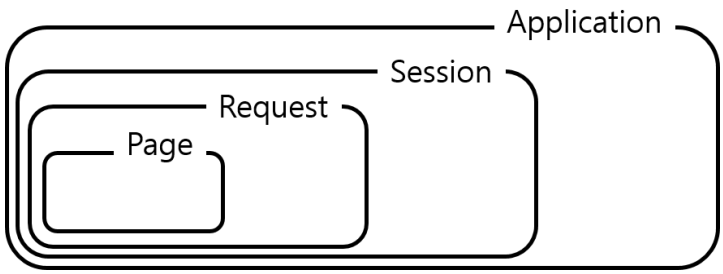
5 내장 객체의 영역(범위, Scope)

내장 객체의 영역은 객체의 유효범위(유효기간)을 뜻한다

해당 객체가 얼마나 유지되는 지 지정해 주는 것

영역은 page, request, session, application으로 4가지로 분류된다

영역	설명
page	하나의 JSP페이지를 처리할 때 사용되는 영역
request	하나의 요청을 처리할 때 사용되는 영역
session	하나의 브라우저와 관련된 영역
application	하나의 웹 어플리케이션과 관련된 영역



** 영역의 구분을 그림으로 표현하자면 다음과 같겠다.

5.1 page 영역

한 번의 클라이언트 요청에 하나의 JSP 페이지를 범위로 갖는다

클라이언트의 요청을 처리하는 JSP 페이지는 요청에 따라 새로운 page영역을 갖게 되고 pageContext 내장 객체를 할당 받으며, 해당 페이지 내에서만 사용할 수 있다

5.2 request 영역

클라이언트로부터 오는 한 번의 요청과 관련된 영역으로, 클라이언트가 요청한 페이지와 요청을 받은 페이지 사이에서 request 내장 객체에 정보를 저장할 수 있다. 브라우저가 결과를 받으면 그 요청과 관련된 request 내장 객체는 사라진다. page 영역은 JSP 페이지만을 포함하며 request 영역은 하나의 요청을 처리하는데 사용되는 모든 JSP 페이지를 포함한다.

request 영역에 저장된 정보를 얻기 위해서는 request 객체가 필요하다

5.3 session 영역

웹 브라우저를 닫기 전까지 페이지를 이동하더라도 유지되며 웹 서버에서 제공하는 것. 세션을 통해 웹 어플리케이션의 로그인 상태를 유지한다

5.4 application 영역

웹 어플리케이션에 속한 모든 페이지, 페이지에 대한 요청, 세션 모두 하나의 application 영역에 속한다. 모든 JSP 페이지는 하나의 application 내장 객체를 공유하고 있으며 이 application 내장 객체가 application 영역에 속한다.

myWeb/web/testJSP/testJsp2.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02 pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>JSP 내장 객체 Test</title>
08 </head>
09 <body>
10 <h3>JSP 내장 객체 Test</h3>
11 <hr>
12 <h4>1. out.print()</h4>
13 result = <% out.print("Hello JSP"); %>          // Hello JSP
14 <h4>2. Request의 내장 객체</h4>
15 <%
16 out.print("프로토콜 : "+request.getProtocol()+"<br>");      // HTTP/1.1
17 out.print("서버 명 : "+request.getServerName()+"<br>");      // localhost
18 out.print("웹루트 경로 : "+request.getContextPath()+"<br>");    // /test
19 out.print("현재 경로 : "+request.getRequestURI()+"<br>");   //
20 /test/testJSP/testJsp2.jsp
21 %>
22 <h4>3. Response 내장 객체</h4>
23 response.sendRedirect() 주석을 제거하면, 접속 시 바로 index.html로 간다.
24 <%--  response.sendRedirect("../index.html");  --%>
25 <br><br><br>
26 <a href="../index.html">뒤로가기</a>
27 </body>
   </html>
```

Chapter 12. 파일 업/다운로드

1 COS 라이브러리

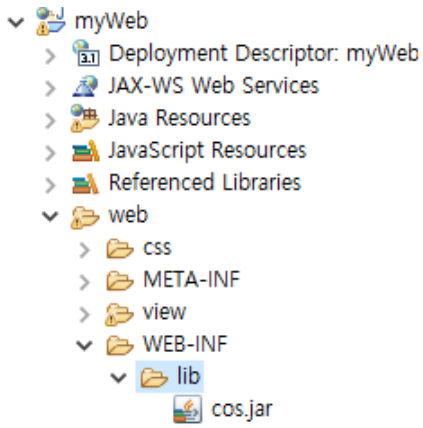
JSP에서 자주 사용되는 기능인 파일 업로드는 Java에서 기본으로 지원하지 않지만, cos.jar 오픈 라이브러리를 이용하면 쉽게 구현 가능할 수 있다.

- COS 라이브러리를 얻을 수 있는 주소

```
http://servlets.com/cos/
```

웹 페이지 하단에서 볼 수 있는 Download 항목에서 cos 파일을 다운 받는다.

압축 파일 내 lib폴더의 cos.jar 파일을 복사하여 자신의 프로젝트의 web/WEB-INF/lib 폴더에 붙여 넣는다. ** lib폴더가 없다면 새 폴더로 생성하여 이름을 lib로 짓자.



- 파일 업로드를 위한 form 태그

```
<form name = "사용자정의" method = "post" enctype = "multipart/form-data">
  <input type = "file" name = "사용자정의">
</form
```

method를 post로 지정, enctype 속성을 multipart/form-data로 추가 input 태그의 type을 file로 지정한다.

enctype 속성의 값인 multipart/form-data는 서버로 데이터가 전송될 때 일반 텍스트뿐만 아니라 파일 형식의 데이터도 전송됨을 알리기 위해 지정하는 것이다. 이를 추가하지 않으면 file 선택 박스에서 선택된 파일의 이름만 텍스트로 전송되고 파일 객체가 서버로

전송되지 않는다.

또한 파일을 전송할 경우 form으로 전송하는 데이터의 크기가 커지므로 get 메소드 방식으로 전송하는 것이 불가능해진다.

2 파일 업로드 하기

- 파일 전송 폼 화면

web/view/fileUpload/upload.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>파일 업로드 테스트</title>
08 </head>
09 <body>
10
11 <form action="upload.do" method="post" enctype="multipart/form-data">
12
13 글쓴이 : <input type="text" name="name"/><br/>
14 제  목 : <input type="text" name="title"/><br/>
15 첨부파일 : <input type="file" name="uploadFile"/><br/>
16 <input type="submit" value="전송"/>
17 </form>
18
19 </body>
20 </html>
```

- 파일 업로드를 위한 서블릿 클래스

first.fileUpload.UploadServlet.java

```
01 package first.fileUpload;
02
```

```
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import javax.servlet.ServletContext;
06  import javax.servlet.ServletException;
07  import javax.servlet.annotation.WebServlet;
08  import javax.servlet.http.HttpServlet;
09  import javax.servlet.http.HttpServletRequest;
10  import javax.servlet.http.HttpServletResponse;
11
12  import org.apache.tomcat.util.http.fileupload.servlet.ServletFileUpload;
13
14  import com.oreilly.servlet.MultipartRequest;
15  import com.oreilly.servlet.multipart.DefaultFileRenamePolicy;
16
17  @WebServlet("/upload.do")
18  public class UploadServlet extends HttpServlet {
19    private static final long serialVersionUID = 1L;
20
21    protected void doPost(HttpServletRequest request, HttpServletResponse response)
22        throws ServletException, IOException {
23
24      request.setCharacterEncoding("UTF-8");
25      response.setContentType("text/html; charset=UTF-8");
26
27      PrintWriter out = response.getWriter();
28
29      // 파일 저장 경로 (web 경로 밑에 해당 폴더를 생성해 주어야 한다)
30      String fileSavePath = "upload";
31
32      // 파일 크기 10M 제한
33      int uploadSizeLimit = 10 * 1024 * 1024;
34      String encType = "UTF-8";
35      //enctype="multipart/form-data" 로 전송되었는지 확인
```

```
36      if(!ServletFileUpload.isMultipartContent(request))
37        response.sendRedirect("view/error/Error.jsp");
38      ServletContext context = getServletContext();
39      String uploadPath = context.getRealPath(fileSavePath);
40      System.out.println(uploadPath);
41
42      MultipartRequest multi = new MultipartRequest(request, // request 객체
43          uploadPath, // 서버 상 업로드 될 디렉토리
44          uploadSizeLimit, // 업로드 파일 크기 제한
45          encType, // 인코딩 방법
46          new DefaultFileRenamePolicy() // 동일 이름 존재 시 새로운 이름 부여 방식
47      );
48
49      // 업로드 된 파일 이름 얻어오기
50      String file = multi.getFilesystemName("uploadFile");
51
52      if (file == null) {
53        // 업로드 실패 시
54        System.out.println("업로드 실패");
55      } else {
56        // 업로드 성공 시
57        out.println("<br> 글쓴이 : " + multi.getParameter("name"));
58        out.println("<br> 제목 : " + multi.getParameter("title"));
59        out.println("<br> 첨부파일명 : " + file);
60        out.println("<br> 업로드 성공!!!");
61      }
62    }
63  }
```

업로드된 파일은 서버의 지정 폴더에 저장된다.

eclipse에서 서버를 생성 후 옵션 설정 시 serve modules without publishing을 체크하지 않고 구동하여 구현한 경우, 업로드한 파일은 하기 경로에서 확인할 수 있다.

```
{workspace}₩.metadata₩.plugins₩org.eclipse.wst.server.core₩tmp0₩wtpwebapps₩FileUpload₩upload
```

MultipartRequest 클래스는 cos.jar 라이브러리에서 제공되는 파일 업로드를 담당하는 클래스이다. 파일 업로드를 담당하는 생성자와 메소드를 포함한다.

- 파일 업로드를 하기 위한 import

```
import com.oreilly.servlet.MultipartRequest;
import com.oreilly.servlet.multipart.DefaultFileRenamePolicy;
```

- MultipartRequest 생성자

매개 변수	설명
request	MultipartRequest 객체와 연결될 request 객체
saveDirectory	서버 측에 저장될 경로
maxPostSize	업로드 최대 크기
encoding	파일 인코딩 방식 (파일 이름이 한글일 경우 UTF-8 지정)
policy	파일 중복 처리를 어떻게 할지 결정 (new DefaultFileRenamePolicy( ) 를 이용해 간단히 지정할 수 있다. 파일 명 뒤에 숫자를 붙인다.)

- MultipartRequest의 메소드

메소드 명	설명
getParameterNames( )	폼에서 전송된 파라미터 열거체로 리턴
getParameterValues( )	폼에서 전송된 파라미터를 배열로 리턴
getParameter( )	객체의 해당 파라미터 값을 가져옴
getFileName( )	파일을 여러 개 업로드 할 경우 값들을 열거체로 리턴
getFilesystemName( )	서버에 실제로 업로드된 파일 이름
getOriginalFileName( )	클라이언트가 업로드한 원본 파일 이름
getContentType( )	업로드 파일의 컨텐트 타입 확인
getFile( )	서버에 업로드 된 파일 정보를 객체로 얻어옴

3 한꺼번에 여러 파일 업로드하기

- 파일 전송 폼 화면

web/view/fileUpload/multiUpload.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
```

```
04  <html>
05  <head>
06  <meta charset="UTF-8">
07  <title>여러 파일 업로드 테스트</title>
08  </head>
09  <body>
10
11  <form action="multiUpload.do" method="post" enctype="multipart/form-
12  data">
13  첨부파일1 : <input type="file" name="uploadFile1"/><br/>
14  첨부파일2 : <input type="file" name="uploadFile2"/><br/>
15  첨부파일3 : <input type="file" name="uploadFile3"/><br/>
16  <input type="submit" value="전송"/>
17  </form>
18
19  </body>
    </html>
```

- 파일 업로드를 위한 서블릿 클래스

first.fileUpload.MultiUploadServlet.java

```
01  package first.fileUpload;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import java.util.Enumeration;
06
07  import javax.servlet.ServletContext;
08  import javax.servlet.ServletException;
09  import javax.servlet.annotation.WebServlet;
10  import javax.servlet.http.HttpServlet;
11  import javax.servlet.http.HttpServletRequest;
12  import javax.servlet.http.HttpServletResponse;
13
```

```
14  import com.oreilly.servlet.MultipartRequest;
15  import com.oreilly.servlet.multipart.DefaultFileRenamePolicy;
16
17  @WebServlet("/multiUpload.do")
18  public class MultiUploadServlet extends HttpServlet {
19    private static final long serialVersionUID = 1L;
20
21    protected void doPost(HttpServletRequest request, HttpServletResponse response)
22        throws ServletException, IOException {
23
24      request.setCharacterEncoding("UTF-8");
25      response.setContentType("text/html; charset=UTF-8");
26
27      PrintWriter out = response.getWriter();
28
29      // 파일 저장 경로
30      String fileSavePath = "upload";
31      // 파일 크기 10M 제한
32      int uploadSizeLimit = 10 * 1024 * 1024;
33      String encType = "UTF-8";
34
35      ServletContext context = getServletContext();
36      String uploadPath = context.getRealPath(fileSavePath);
37       MultipartRequest multi = new MultipartRequest(request, // request 객체
38          uploadPath, // 서버 상 업로드 될 디렉토리
39          uploadSizeLimit, // 업로드 파일 크기 제한
40          encType, // 인코딩 방법
41          new DefaultFileRenamePolicy() // 동일 이름 존재 시 새로운 이름 부여 방식
42      );
43
44      Enumeration files = multi.getFileNames();
45      out.println("<h2>멀티 파일 업로드 구현하기</h2>");
46      while (files.hasMoreElements()) {
```

```
47          // 업로드 된 파일 이름 얻어오기
48          String file = (String) files.nextElement();
49          String fileName = multi.getFilesystemName(file);
50
51          out.println("<br> 첨부파일명 : " + fileName);
52       }
53    }
54 }
```

4. 파일 다운로드 하기

뷰 페이지에서 파일명을 클릭하면 해당 파일이 다운되게 구현할 경우

```
01 <a href="/test/filedown?file=sample.txt">sample.txt</a>
```

first.fileDownServlet.java

```
01 package first;
02 import java.io.BufferedInputStream;
03 import java.io.File;
04 import java.io.FileInputStream;
05 import java.io.IOException;
06
07 import javax.servlet.ServletException;
08 import javax.servlet.ServletOutputStream;
09 import javax.servlet.annotation.WebServlet;
10 import javax.servlet.http.HttpServlet;
11 import javax.servlet.http.HttpServletRequest;
12 import javax.servlet.http.HttpServletResponse;
13
14 /**
15  * Servlet implementation class BoardFileDownServlet
16  */
17 @WebServlet("/bfdown")
```

```
18  public class BoardFileDownServlet extends HttpServlet {
19          private static final long serialVersionUID = 1L;
20
21      /**
22       * @see HttpServlet#HttpServlet()
23       */
24      public BoardFileDownServlet() {
25          super();
26          // TODO Auto-generated constructor stub
27      }
28
29          /**
30           *          @see          HttpServlet#doGet(HttpServletRequest          request,
31  HttpServletResponse response)
32           */
33          protected void doGet(HttpServletRequest request, HttpServletResponse
34  response) throws ServletException, IOException {
35                  //게시글 첨부파일 다운 처리용 컨트롤러
36                  request.setCharacterEncoding("utf-8");
37
38                  //프로젝트 내에 파일이 저장된 폴더의 경로정보 얻어옴
39                  String readFolder = request.getSession()
40
41          .getServletContext().getRealPath("/buploadFiles");
42
43                  String - 110 -oolean- 110 - = request.getParameter("file");
44
45                  //클라이언트로 내보낼 출력 스트림 생성
46                  ServletOutputStream downOut =
47                                  response.getOutputStream();
48
49                  File    downFile    =    new    File(readFolder    +    "/"    +
50  - 110 -oolean- 110 -);
```

```
51
52                      response.setContentType("text/plain; charset=utf-8");
53                      //한글 파일명 인코딩 처리함
54                      response.addHeader("Content-Disposition",
55                                              "attachment; filename=₩"" +
56                                              new     String(originalFileName.getBytes("UTF-
57 8"),
58                                                                      "ISO-8859-1") + "₩"");
59                      response.setContentLength((int)downFile.length());
60
61                      BufferedInputStream bin = new BufferedInputStream(
62                                              new FileInputStream(downFile));
63                      int read = -1;
64                      while((read = bin.read()) != -1){
65                              downOut.write(read);
66                              downOut.flush();
67                      }
68                      downOut.close();
69                      bin.close();
70              }
71              /**
72               *      @see        HttpServlet#doPost(HttpServletRequest        request,
73 HttpServletResponse response)
74               */
75              protected void doPost(HttpServletRequest request, HttpServletResponse
76 response) throws ServletException, IOException {
77                      // TODO Auto-generated method stub
                        doGet(request, response);
                }
}
```

Chapter 13. Ajax

1 Ajax, Asynchronous JavaScript + XML

XML기반의 기술로 비동기적 자바스크립트 XML의 준말이다. 자바 스크립트로 HTTP 요청을 보내 XML로 응답을 받는 기술이다.

표준HTML과 CSS로 작성된 화면과 DOM(Document Object Model)을 이용한 상호작용, 서버와의 데이터 교환은 XML형식을 이용하며 **비동기 통신**으로 웹브라우저에 내장된 XMLHttpRequest 객체를 이용하는 자바스크립트 기술이다.

기존의 방식의 웹 사이트는 웹 브라우저가 웹 서버에 요청을 전송하면 웹 서버는 JSP/PHP/ASP등의 언어로 이루어진 서버 측 어플리케이션을 사용해 요청을 처리한 뒤, 처리 결과를 HTML로 생성해 웹 브라우저에 응답한다. 웹 브라우저는 응답받은 HTML을 분석한 뒤 그 내용을 화면에 그려주게 된다. 즉, 웹 브라우저가 웹 서버와 통신을 하고 요청 결과는 HTML로 생성되며 사용자 입장에서 페이지 이동이 발생하는 것을 확인할 수 있게 된다.

Ajax 방식의 웹 사이트는 별도의 웹 페이지 호출 없이 웹 어플리케이션과 비동기적으로 데이터를 주고 받은 후 클라이언트에서 응답 받은 데이터를 처리할 수 있게 된다. 즉, 웹 페이지의 이동없이 화면을 유지한 채 웹 서버와 데이터 교환을 할 수 있게 된다.

Ajax 방식의 웹 사이트는 기존의 방식과는 다르게 XMLHttpRequest 객체가 웹 서버와 통신을 하며, 응답 결과가 HTML이 아닌 XML 또는 단순 텍스트로 처리된다.

1.1 Ajax 방식 웹 사이트의 장점

- 비동기 통신 기반으로 데이터 전송외 사용자는 다른 작업을 수행할 수 있다
- 데이터만을 응답으로 받으므로 서버 측 처리 속도가 빠르며 전송 데이터의 크기도 적어진다
- 불필요한 데이터 요청을 최소화할 수 있다
- 많은 부분을 웹 브라우저에서 처리하므로 서버에 부하를 줄일 수 있다

2 AJAX요청 처리 객체 (XMLHttpRequest)

2.1 XMLHttpRequest 객체의 메소드

2.1.1 void open(String method, String url, - 112 -oolean asynch)

요청을 초기화 한다.

method : HTTP 메소드 지정

url : 접속할 URL 지정한다. 접속할 URL은 현재페이지와 같은 도메인이어야 한다

asynch : 동기(false), 비동기(true) 지정 * 동기, 비동기를 설정할 수 있다.

2.1.2 void send( )

서버에 요청을 보낸다

2.2 XMLHttpRequest 객체 속성

2.2.1 onreadystatechange

자바스크립트 콜백 함수를 저장 / 저장함수는 readyState 값이 변경될 때마다 호출된다

2.2.2 readyState

요청의 상태를 의미하는 값

- 0 : UNINITIALIZED, 객체만 생성되고 초기화되지 않은 상태(open( ) 메소드가 호출 전)
- 1 : LOADING, open메서드가 호출되고 아직 send 메소드가 호출 되지 않은 상태
- 2 : LOADED, send( )는 호출했지만 states와 header는 도착하지 않은 상태
- 3 : INTERACTIVE, 데이터의 일부만 받은 상태
- 4 : COMPLETED, 데이터를 전부 받은 상태

2.2.3 responseText

서버의 응답을 String 형식으로 나타냄

2.2.4 responseXML

서버의 응답을 XML 형식으로 나타냄

2.2.5 status

서버로부터 받은 HTTP 응답 상태 코드

2.2.6 statusText

HTTP 상태코드에 대한 텍스트

3 XHR 객체 생성하기

Ajax 통신을 하기 위해 XMLHttpRequest 객체를 생성해야 한다. IE 7.0 이전에는 모질라 웹 브라우저와 IE 브라우저가 각각 XHR 객체를 생성하는 방식이 달랐지만 IE 7.0 부터는

동일하게 new XMLHttpRequest( )를 통해 객체를 생성할 수 있다.

```
function getXMLHttpRequest() {
  // IE에서 XMLHttpRequest 객체 구하기
  if( window.ActiveXObject ) {
    try {
      // IE 최신 버전
      return new ActiveXObject("Msxml2.XMLHTTP");
    } catch(e) {
      try {
        // IE 이전 버전
        return new ActiveXObject("Microsoft.XMLHTTP");
      } catch(e1) {
        return null;
      }
    }
  } else if(window.XMLHttpRequest) {
    //IE외 브라우저에서 XMLHttpRequest 객체구하기
    return new XMLHttpRequest();
  } else {
    return null;
  }
}
```

4 웹 서버에 요청

4.1 GET 방식

```
httpRequest = getXMLHttpRequest( );
httpRequest.open("GET", "/test.jsp?id=admin&pw=1234", true);
httpRequest.send(null);
```

4.2 POST 방식

```
httpRequest = getXMLHttpRequest( );
httpRequest.open("POST", url, true);
```

```
httpRequest.send("id=admin&pw=1234");
```

5 웹 서버 응답 반영하기

XMLHttpRequest 객체의 속성인 onreadystatechange를 사용하여 콜백 함수를 지정하고 웹 서버로부터 응답이 도착하면 해당 콜백 함수가 불러와지도록 설정할 수 있다.

- GET 방식의 요청에 응답이 왔을 때 콜백 함수 호출되도록 하기

```
httpRequest = getXMLHttpRequest( );
httpRequest.onreadystatechange = callbackFunc;
httpRequest.open("GET", "/test.jsp?id=admin&pw=1234", true);
httpRequest.send(null);
```

```
function callbackFunc( ) {
  // 서버의 응답에 알맞은 작업 수행
}
```

콜백함수의 작성은 XHR 객체의 status 속성을 확인해 정상적으로 수행하거나 에러메시지를 보여주도록 작성할 수 있다.

6 XMLHttpRequest 모듈 만들기

XMLHttpRequest 객체를 이용하여 Ajax 방식으로 코드를 구현하려면 매 페이지마다 반복되는 코드가 존재하게 된다. 이를 보다 편하게 사용하기 위해 .js 파일을 만들어 모듈화 해보도록 한다.

- XMLHttpRequest 모듈 : httpRequest.js

myWeb/web/js/httpRequest.js

```
01 //XMLHttpRequst 객체를 생성
02 function getXMLHttpRequest() {
03    if (window.ActiveXObject) {
04       try {
05          return new ActiveXObject("Msxml2.XMLHTTP");
06       } catch(e) {
07          try {
08             return new ActiveXObject("Microsoft.XMLHTTP");
```

```
09            } catch(e1) { return null; }
10        }
11    } else if (window.XMLHttpRequest) {
12        return new XMLHttpRequest();
13    } else {
14        return null;
15    }
16 }
17
18 //생성된 XMLHttpRequst 객체 전역변수
19 var httpRequest = null;
20
21 //지정 메소드(GET/POST), 지정 URL, 파라미터 값을 사용하여 웹 서버에 요청을
22 전송
23 function sendRequest(method, url, params, callback) {
24    httpRequest = getXMLHttpRequest();
25
26    var httpMethod = method ? method : 'GET';
27    if (httpMethod != 'GET' && httpMethod != 'POST') {
28        httpMethod = 'GET';
29    }
30
31    var httpParams = (params == null || params == '') ? null : params;
32    var httpUrl = url;
33
34    // GET 메소드면 URL 뒤에 파라미터를 붙임
35    if (httpMethod == 'GET' && httpParams != null) {
36        httpUrl = httpUrl + "?" + httpParams;
37    }
38
39    // 비동기식으로 XMLHttpRequest 객체 사용
40    httpRequest.open(httpMethod, httpUrl, true);
41    // 웹 서버에 전송할 컨텐트 타입 지정
42    httpRequest.setRequestHeader('Content-Type',                'application/x-www-form-
43 urlencoded');
44    // 콜백 함수 지정
```

```
45    httpRequest.onreadystatechange = callback;
46    // HTTP 요청 방식이 'POST'면 send() 함수를 통해 파라미터 전송
47    httpRequest.send(httpMethod == 'POST' ? httpParams : null);
48  }
```

화면담당 웹 페이지 : ajaxTest.jsp

myWeb/web/view/ajax/ajaxTest.jsp

```
01  <%@ page language="java" contentType="text/html; charset=UTF-8"
02              pageEncoding="UTF-8"%>
03  <!DOCTYPE html>
04  <html>
05  <head>
06  <meta charset="UTF-8">
07  <title>안녕하세요</title>
08  <!-- XMLHttpRequest 모듈 로딩 -->
09  <script type="text/javascript" src="/web/js/httpRequest.js"></script>
10  <script type="text/javascript">
11  // [입력]버튼 클릭시 호출.
12  // sendRequest() 함수를 사용하여 폼에 입력한 값을 hello.jsp에 전송
13  function helloToServer() {
14    var params = "name="+encodeURIComponent(document.f.name.value);
15    sendRequest("POST", "hello.jsp", params, helloFromServer);
16  }
17  //웹 서버로부터 올바른 응답 결과가 도착하면 hello.jsp가 생성한 텍스트 출력
18  function helloFromServer() {
19    if (httpRequest.readyState == 4) {
20      if (httpRequest.status == 200) {
21        alert("서버응답:"+httpRequest.responseText);
22      }
23    }
24  }
25  </script>
26  </head>
27  <body>
28
29    <form name="f">
```

```
30      <input type="text" name="name">
31      <input type="button" value="입력" onclick="helloToServer()">
32    </form>
33
34  </body>
35  </html>
```

- 요청/응답 페이지 : hello.jsp

myWeb/web/view/ajax/hello.jsp

```
01  <%@ page contentType="text/html; charset=UTF-8"%>
02  <%
03  // UTF-8 인코딩으로 파라미터 읽음
04  request.setCharacterEncoding("UTF-8");
05  String name = request.getParameter("name");
06  %>
07  안녕하세요, <%=name %>님!
```

7 jQuery Ajax

Ajax는 브라우저마다 다른 코드를 가지게 되는 단점이 있다. 하지만 jQuery가 제공해주는 Ajax기능을 사용하면 크로스브라우징을 제공하기 때문에 한가지 코드로 다양한 브라우저에 대응할 수 있게 된다.

7.1 jQuery의 Ajax API

Ajax 통신에 필요한 여러가지 설정에 관련된 기능을 제공하는 Global Ajax Event Headers, 통신에는 직접적으로 관여하지 않지만 도움이 되는 Helper Functions, POST와 GET 메소드 통신을 할 수 있도록 하는 Shorthand Methods, 가장 기본적인 통신을 제공하는 Low-Level Interface 등을 제공한다. 이처럼 jQuery는 Ajax 관련 다양한 API를 제공하지만 그 중 Low-Level Interface API를 많이 사용한다.

- jQuery Ajax API 참조 문서 사이트

http://api.jquery.com/category/ajax/

jQuery Ajax Low-Level Interface API

- jQuery.ajax API 의 기본 형태

```
jQuery.ajax( {url [, settings ]} )
```

객체로 매개변수 대입, url : Ajax 통신 요청을 보낼 url / settings : 통신을 위한 여러 설정

- jQuery.ajax API 기본 예제

```
$.ajax({
   url : "ajaxData.jsp?type=1",          // url : ajax 요청 주소
   dataType : "text",                    // data : 서버로 데이터를 전송할 옵션
   type : "GET",                         // dataType : 서버측에서 데이터를 해석할 형
   success : function(data){             식
      alert("통신데이터 값 : " +         //          ( xml, json, script, html 등 )
data) ;                                  // type : HTTP 전송 메소드 지정 ( get, post )
      $("#resultArea").html(data) ;      // success : 요청에 대한 처리가 성공했을 때
   }                                     //          호출할 콜백
   error : function( ){                  // error : 만약 ajsx통신에 에러가 발생한다면
      alert('실패!!');                   //          호출할 콜백 함수
   },
});
```

7.2 ajax 메소드를 이용한 간단한 계산기 구현

전달한 인자를 계산한 후 response에 담기 위한 방법은 크게 HTML, XML, JSON 세가지가 있는데, 여기서는 JSON을 활용한 응답 전송에 대해 다뤄보겠다.

먼저 하기 사이트에서 제공하는 라이브러리를 받아 web/WEB-INF/lib 경로에 넣는다.

```
https://mvnrepository.com/artifact/com.googlecode.json-simple/json-simple/1.1.1
```

- 계산기 화면

myWeb/web/view/ajax/ajaxTestCalc.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02    pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>Ajax Test Calculate Page</title>
08 <script type="text/javascript"
```

```
09  src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
10   // JQuery 를 사용하기 위해, CDN 방식을 통한 JQuery API 코드 작성
11  <script type="text/javascript">
12    $(function() {
13      $("#btnSend").on('click', function() {
14        var num1 = $("#num1").val();
15        var num2 = $("#num2").val();
16        var oper = $("#oper").val();
17
18        var url = "/test/calc.do";
19        var query = "n1=" + num1 + "&n2=" + num2;
20        query += "&oper=" + oper;
21
22        $.ajax({  // JQuery 를 통한 ajax 호출 방식 사용
23          type : "POST",
24          url : url,
25          data : query,
26          dataType : "json",  // 전달받을 객체는 JSON 이다.
27          success : function(data) {
28            $("#result").html(data.result);   // 전달받은 JSON 객체의 Key인 result로
29  value값 접근
30            $("#num1").val("");
31            $("#num2").val("");
32            $("#oper").val("add");
33          },
34          error : function(e) {
35            alert(e.responseText);
36          }
37        });
38      });
39    });
40  </script>
41  </head>
```

```
42  <body>
43    <input type="text" id="num1">
44    <select id="oper">
45      <option value="add">더하기</option>
46      <option value="sub">빼기</option>
47      <option value="mul">곱하기</option>
48      <option value="div">나누기</option>
49    </select>
50    <input type="text" id="num2">
51    <input type="button" value=" = " id="btnSend">
52    <br>
53    <hr>
54    <div id="result"></div>
55  </body>
56  </html>
```

- 계산 기능 서블릿

first.ajax.controller.CalcAjaxServlet.java

```
01  package first.ajax.controller;
02
03  import java.io.IOException;
04  import java.io.PrintWriter;
05  import javax.servlet.ServletException;
06  import javax.servlet.annotation.WebServlet;
07  import javax.servlet.http.HttpServlet;
08  import javax.servlet.http.HttpServletRequest;
09  import javax.servlet.http.HttpServletResponse;
10  import org.json.simple.JSONObject;
11
12  @WebServlet("/calc.do")
13  public class CalcAjaxServlet extends HttpServlet {
14    private static final long serialVersionUID = 1L;
15    public CalcAjaxServlet() { }
```

```
16    protected void doGet(HttpServletRequest request, HttpServletResponse response)
17          throws ServletException, IOException {
18       response.setContentType("application/json; charset=UTF-8");
19       // 기존"text/html"이 아닌 'application/json'으로 ContentType을 선언
20       //JSONObject 로 하여 JSON 객체 생성
21       JSONObject jobj = new JSONObject();
22       try {
23          int num1 = Integer.parseInt(request.getParameter("n1"));
24          int num2 = Integer.parseInt(request.getParameter("n2"));
25          String oper = request.getParameter("oper");
26          if (oper.equals("add")) {
27          // jobj.put ( Key, Value ) 를 통해 응답하고자 하는 결과를 Map에 담는다.
28             jobj.put("result",String.format("%d+%d=%d", num1, num2, num1 + num2));
29          } else if (oper.equals("sub")) {
30             jobj.put("result",String.format("%d-%d=%d", num1, num2, num1 - num2));
31          } else if (oper.equals("mul")) {
32             jobj.put("result",String.format("%d*%d=%d", num1, num2, num1 * num2));
33          } else if (oper.equals("div")) {
34             jobj.put("result",String.format("%d/%d=%d", num1, num2, num1 / num2));
35          }
36       } catch (ArithmeticException e) {
37          jobj.put("result","0으로는 나눌 수 없습니다.");
38       } catch (NumberFormatException e) {
39          jobj.put("result","계산 값이 숫자가 아닙니다.");
40       } finally {
41        PrintWriter out = response.getWriter();
42        out.print(jobj.toJSONString());  // JSON객체의 toString() 메소드를 사용하여 응
43 답값 전달
44        out.flush();
45        out.close();
46       }
47    }
48 }
```

Part 4. 배치 프로그램 구현하기

Chapter 14. JSP 액션 태그

1 액션 태그

클라이언트 혹은 서버에게 어떤 행동을 하도록 지시하는 태그를 뜻한다

액션 태그는 스크립트릿, 주석, 디렉티브와 함께 JSP 페이지를 이루는 요소 중 하나이다

태그 종류	설명
<jsp:forward>	다른 사이트로 이동할 때 사용 페이지의 흐름을 제어
<jsp:include>	정적 혹은 동적인 자원을 현재 페이지에 포함시킴 페이지를 모듈화할 때 사용
<jsp:param>	<jsp:forward>, <jsp:include>, <jsp:plugin>과 같이 사용됨 파라미터를 추가할 때 사용
<jsp:useBean>	빈(Bean)을 생성하고 사용하기 위한 환경을 정의
<jsp:setProperty>	빈에서 속성 값을 할당
<jsp:getProperty>	빈에서 속성 값을 얻어올 때 사용

2 <jsp:param> 액션 태그

파라미터 값을 전달할 때 사용

단독으로 사용하지 않고 <jsp:include>나 <jsp:forward> 태그의 내부에 기술하여 사용

- 형식

```
<jsp:param name="파라미터 변수" value="값"/>
```

3 <jsp:forward> 액션 태그

페이지를 이동할 때 사용

포워드 방식으로 페이지를 이동

기존 request 연결을 유지하면서 서버상의 url로 request 정보를 전달

- 형식

```
<jsp:forward page="이동할페이지">
  <jsp:param name="변수명" value="값" />
</jsp:forward>
```

4 <jsp:include> 액션 태그

모든 페이지에서 공통적으로 사용되어야 하는 내용에 대해서 코드를 복사 붙여넣기 하는 것보다 각각 JSP 페이지에 구성을 하고 액션태그를 이용해 페이지를 포함시키는 것이 좋다

공통적으로 적용된 부분을 한꺼번에 수정할 수 있게 되어 유지보수가 수월해진다

- 형식1

```
<jsp:forward page="포함할 페이지 이름" flush="true" 또는 "false" />
```

- 형식2

```
<jsp:forward page="포함할 페이지 이름" flush="true">
  <jsp:param name="변수명" value="값" />
</jsp:forward>
```

5 <jsp:plugin> 액션 태그

자바 애플릿 또는 자바빈즈 컴포넌트를 클라이언트로 다운받아 사용할 수 있도록 브라우저에 맞는 HTML 코드를 생성해주는 역할을 한다. 서버 측에서 사용되는 컴포넌트의 경우 서버에 부하를 주게 되는데 plugin 액션 태그를 이용해 서버의 부하를 줄여 줄 수 있다

- 형식

```
<jsp:plugin type="플러그인 타입" codebase="클래스 파일 위치"
        code="불러올 클래스 파일" width="가로" height="세로">
  <jsp:params>
    <jsp:param name="파라미터이름" value="파라미터값"/>
  <jsp:params>
</jsp:plugin〉
```

- plugin 태그의 속성

속성	사용법	설명
type	type="appletbean"	타입 설정
code	code="code.class"	사용할 클래스 파일 설정
width	width="800"	플러그인 표시 영역 너비 지정
height	height="600"	플러그인 표시 영역 높이 지정
codebase	codebase="./"	플러그인 파일 위치 지정
name	name="pluginOne"	플러그인 식별을 위한 이름 지정
align	align="left"	정렬방식 지정
vspace	vspace="10"	수직여백 설정
hspace	hspace="10"	수평여백 설정
jreversion	jreversion="1.8"	플러그인을 실행할 JRE 버전 입력
iepluginurl	iepluginurl="url"	IE전용 플러그인 다운 URL 입력
nspluginulr	nspluginurl="url"	넷스케이프 전용 플러그인 다운 URL 입력

6 <jsp:useBean> 액션 태그

객체의 이름과 사용범위, 빈의 저장위치 등을 통해서 객체가 생성된다. JSP의 자바 코드에서는 action의 id 특성에 지정된 값을 통해서 객체를 참조한다.

- 형식

```
<jsp:useBean id="빈 이름" scope="범위" class="빈의 저장위치"/>
```

id : 객체 인스턴스를 식별하는 이름

scope : 객체 참조의 유효범위(default : page)

class : 완전한 형태의 클래스 이름

7 <jsp:setProperty> 액션 태그

빈의 속성에 값을 설정하는 태그

- 형식

```
<jsp:setProperty name = "빈 이름" property="프로퍼티 명" value="저장할 값"/>
```

name : <jsp:useBean> 태그에 정의된 빈 인스턴스 이름

property : 값을 설정하고자 하는 빈 속성의 이름 , 설정 시 servletRequest안의 모든 인자들 중 빈 속성과 데이터 형이 일치하는 것을 찾아 각각의 속성들을 각각의 인자 값으로 설정한다.

value : 빈 속성에 설정할 값

8 <jsp:getProperty> 액션 태그

빈의 속성값을 얻는데 사용

- 형식

```
<jsp:getProperty name = "빈 이름" property="프로퍼티 명"/>
```

name : 속성을 얻고자 하는 빈 인스턴스의 이름

property : 얻고자 하는 속성의 이름

Chapter 15. EL과 JSTL

1 표현언어, Expression Language

표현 언어는 값을 표현하는데 사용되는 새로운 스크립트 언어로서 JSP2.0 규약에서 새롭게 추가된 기능이다. JSP에서 출력을 위해서 표현식을 사용한다.

- 표현 언어에서 제공하는 기능
  ① JSP의 네 가지 기본 객체가 제공하는 영역의 속성 사용
  ② 집합 객체에 대한 접근 방법 제공
  ③ 수치연산 , 관계연산 , 논리연산자 제공
  ④ 자바 클래스 메소드 호출 기능 제공
  ⑤ 표현 언어만의 기본 객체 제공

JSP에서는 스크립트 요소를 제공하며 그 중 표현식 스크립트는 계산식, 함수 호출 결과를 문자열 형태로 출력해주는 역할을 한다. 표현언어는 표현식보다 사용 방법이 간단하고 문법 체계가 직관적이어서 더욱 쉽게 사용 가능하다.

- 표현식

```
<%=표현식%>
```

- 표현 언어

```
${표현언어}
```

- "Hello EL"을 세가지 방법으로 출력한 예제

myWeb/web/testJSP/testEl.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02 pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
```

```
07  <title>EL Test Page</title>
08  </head>
09  <body>
10
11  EL 표현 : ${"Hello EL" } <br> <!—표현 언어 →
12  JSP 표현식 : <%= "Hello EL" %> <br> <!—표현식 →
13  JSP 스크립트릿 : <% out.println("Hello EL"); %> <br> <!—스크립트릿 →
14
15  </body>
16  </html>
```

1.1 표현 언어에서 사용되는 자료형

표현 언어에서는 정수형, 실수형, 논리형, null 등의 데이터 타입을 사용할 수 있다

myWeb/web/testJSP/testElDatatype.jsp

```
01  <%@ page language="java" contentType="text/html; charset=UTF-8"
02  pageEncoding="UTF-8"%>
03  <!DOCTYPE html>
04  <html>
05  <head>
06  <meta charset="UTF-8">
07  <title>EL 자료형 연습하기</title>
08  </head>
09  <body>
10
11  정수형 : ${15 } <br>              <!—정수형 : 15 →
12  실수형 : ${6.7 } <br>             <!—실수형 : 6.7 →
13  문자열 : ${"안녕하세요" } <br>    <!—문자열 : 안녕하세요 →
14  논리형 : ${false } <br>           <!—논리형 : false →
15  null : ${null } <br>              <!—null :    →
16
17  </body>
18  </html>
```

1.2 표현 언어에서 사용되는 연산자

표현 언어로 접근된 데이터를 조작 및 비교, 연산을 할 수 있도록 연산자의 사용이 가능하다

종류	연산자
산술	+ - * /(or div) %(or mod)
관계	==(or eq) !=(or ne) <(or lt) >(or gt) <=(or le) >=(or ge)
조건	a ? b : c
논리	&&(or and) ||(or or) !(or not)
null 검사	empty

2 표현 언어의 기본객체

EL에서는 11개의 기본객체를 제공하고 있다

기본 객체	설명
pageContext	JSP의 page 기본 객체와 동일
pageScope	pageCntext 기본 객체에 저장된 속성의 <속성, 값> 매핑을 지정한 Map객체
requestScope	request 기본 객체에 저장된 속성의 <속성, 값> 매핑을 지정한 Map 객체
sessionScope	session 기본 객체에 저장된 속성의 <속성, 값> 매핑을 지정한 mMap 객체
applicationScope	application 기본 객체에 저장된 속성의 <속성, 값> 매핑을 지정한 Map객체
param	요청 파라미터의 <파라미터이름, 값> 매핑을 저장한 Map객체 파라미터 값의 타입은 String으로서, request.Parameter(이름)의 결과와 동일
paramValues	요청 파라미터의 <파라미터이름, 값> 매핑을 저장한 Map객체 파라미터 값의 타입은 String으로서, request.ParameterValues(이름)의 결과와 동일
header	요청 정보의 <헤더이름, 값> 매핑을 지정한 Map객체 request.getHeader(이름)의 결과와 동일

headerValues	요청 정보의 <헤더이름, 값> 매핑을 지정한 Map객체 request.getHeaders(이름)의 결과와 동일
cookie	<쿠키이름, Cookie> 매핑을 저장한 Map객체 request.getCookie( )로 구한 Cookie 배열로부터 매핑을 생성
initParam	초기화 파라미터의 <이름, 값> 매핑을 저장한 Map객체 application.getInitParameter( )결과와 동일

2.1 표현언어로 로그인 폼 정보 활용하기

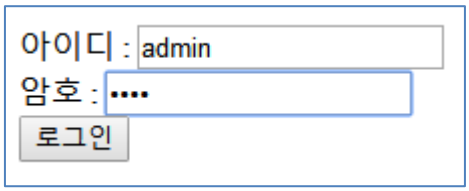
myWeb/web/view/login/login.jsp

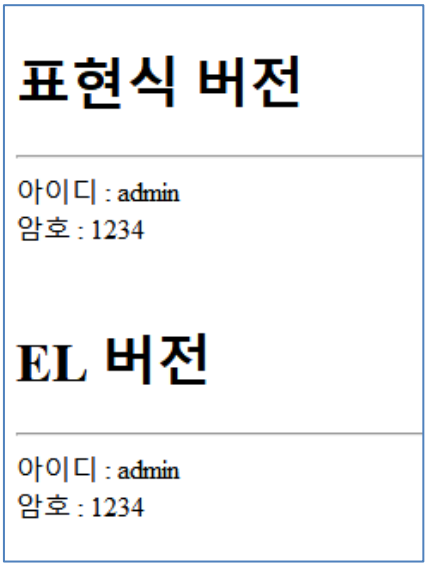
```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02         pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>Login Service</title>
08 </head>
09 <body>
10 // form 태그를 통해 input 태그의 value를 파라미터 값으로 loginEx.jsp에 전송
11 <form method="get" action="loginEx.jsp">
12             <label for="id">아이디 : </label> <input type="text" name="id" id="id"
13 /><br>
14             <label for="pass"> 암호 : </label> <input type="password" name="pass"
15                     id="pass" /><br> <input type="submit" value="로그인" />
16 </form>
17
18 </body>
   </html>
```

myWeb/web/view/login/loginEx.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02         pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>Login 전달 확인</title>
08 </head>
09 <body>
```

```
10
11  <h1>표현식 버전</h1><hr>
12  아이디 : <%=request.getParameter("id") %><br>
13  암호 :  <%=request.getParameter("pass") %><br><br>
14
15  <h1>EL 버전</h1><hr>
16  아이디 : ${param.id }<br>
17  암호 :  ${param.pass }<br><br>
18
19  </body>
20  </html>
```





3 표현 언어에서 내장객체 사용하기

웹 어플리케이션을 JSP로 구현하는데 필요한 정보를 JSP 내장 객체에 속성 값으로 저장하여 사용한다. 속성에 저장된 값을 표현 언어에서도 접근 가능하다.

범위	내장 객체	설명
page	pageScope	page 객체에 저장된 속성의 <속성, 값>매핑을 저장한 Map 객체
request	requestScope	request 객체에 저장된 속성의 <속성, 값>매핑을 저장한 Map 객체
session	sessionScope	session 객체에 저장된 속성의 <속성, 값>매핑을 저장한 Map 객체
application	applicationScope	application 객체에 저장된 속성의 <속성, 값>매핑을 저장한 Map 객체

- JSP 내장 객체의 정보를 주고 받기 위한 메소드

반환타입	메소드	설명
void	setAttribute(String name, Object value)	name에 값을 설정
Object	getAttribute(String name)	name에 설정된 값을 얻음
Object	getAttributeNames( )	현재 객체에 관련된 모든 속성 이름을 얻어냄
void	removeAttribute(String name)	name에 설정된 값을 제거

4 JSTL, JSP Standard Tag Library

JSP에서 사용 가능한 표준 태그 라이브러리

JSTL 태그를 사용하면 JSP 코드가 깔끔해지고 가독성이 좋아진다

JSP 페이지에서 논리적인 판단, 반복문의 처리, 데이터베이스 처리 등의 처리를 하는 코드를 깔끔하게 작성하기 위해서 커스텀 태그를 많이 작성하게 되는데 이런 중복된 노력을 제거하기 위해서 나온 것이다.

4.1 JSTL에서 제공해주는 기능

- 간단한 프로그램 로직 구현(자바의 변수 선언, if문, for문 등)
- 다른 JSP 페이지 호출
- 날짜, 시간, 숫자의 포맷
- 데이터베이스로의 입력, 수정, 삭제, 조회(CRUD)
- XML 문서의 처리
- 문자열 처리 함수 호출

4.2 JSTL에서 제공하는 5가지 커스텀 태그

커스텀 태그	설명
기본 기능(core)	변수 선언, 실행 흐름 제어, 다른 JSP 페이지로 제어 이동
형식화(format)	숫자, 날짜, 시간의 포맷을 지정 국제화, 다국어 지원 기능
데이터베이스(sql)	데이터베이스 CRUD 지원
XML 처리(xml)	XML 문서를 처리할 때 필요한 기능 지원
함수 처리(functions)	문자열 처리 함수 제공

4.3 JSTL taglib 지시자

JSTL이 제공해 주는 기능을 사용하기 위해서는 taglib 지시자를 추가해주어야 한다

uri로 사용하려는 라이브러리를 지정하고 prefix 속성으로 접두사를 지정한다

- 형식

```
<%@ taglib uri="라이브러리 식별자" prefix="접두사" %>
```

- 태그 라이브러리의 URI 식별자와 접두사

기능	prefix	기본 URI
기본 기능	c	http://java.sun.com/jsp/jstl/core
형식화	fmt	http://java.sun.com/jsp/fmt
데이터베이스 작업	sql	http://java.sun.com/jsp/sql
XML 처리	x	http://java.sun.com/jsp/xml
함수 처리	fn	http://java.sun.com/jsp/jstl/functions

4.4 JSTL Core 태그

5 종류의 라이브러리들 중에서 가장 자주 사용되는 태그

태그	설명
<c:set>	변수 값 설정
<c:remove>	변수 값 제거
<c:if>	조건문
<c:choose>	여러 조건을 처리할 때
<c:forEach>	반복 처리
<c:forTokens>	구분자로 분리된 각각의 토큰 처리
<c:import>	외부 자원을 url을 지정하여 가져와 사용
<c:redirect>	지정한 경로로 이동
<c:url>	url 재 작성
<c:out>	데이터 출력
<c:catch>	예외 처리

- JSTL core 태그를 사용하기 위한 taglib 지시자

```
<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core"%>
```

4.4.1 <c:set>과 <c:remove> 태그

<c:set> 태그는 해당 범위 내에 속성을 생성하고 속성 값을 지정하는 데 사용

- <c:set> 태그 형식

```
<c:set var="변수명" value="저장할값" [scope="{page | request | session | application}"]>
```

var : 변수 이름을 String 형으로 지정
value : 변수에 저장할 값을 지정
scope : 변수의 효력이 발휘되는 영역 지정. 기본 값은 page

<c:remove> 태그는 해당 scope에 있는 변수를 제거하는 데 사용

- <c:remove> 태그 형식

```
<c:remove var="변수명" [scope="{page | request | session | application}"]>
```

var와 scope가 <c:set> 했을 때와 같아야 변수 제거 가능

4.4.2 흐름 제어 태그

<c:if> 태그는 자바의 if문과 비슷한 기능
else 문이 없는 단순 if문의 구조와 같다

- <c:if> 태그 형식

```
<c:if test="조건식">
  참일 경우 수행
</c:if>
```

<c:choose> 태그는 자바의 if – else문과 비슷한 기능
순서대로 test 조건을 검사하며 참일 경우 해당 몸체 수행

- <c:choose> 태그 형식

```
<c:choose>
  <c:when test="조건1"> 몸체1 </c:when>
  <c:when test="조건2"> 몸체2 </c:when>
  <c:when test="조건3"> 몸체3 </c:when>
</c:choose>
```

<c:forEach> 태그는 반복 수행을 할 때 사용
배열이나 컬렉션, 맵 등과 같은 데이터 집합체에 저장되어 있는 값들을 순차적으로 처리할 때 사용하는 태그

- <c:forEach> 태그 형식1

```
<c:forEach [var="변수명"] items="집합체">
  몸체
</c:forEach>
```

또한, 인덱스를 사용하여 원하는 구간만큼 반복횟수를 지정할 수 있다

- <c:forEach> 태그 형식2

```
<c:forEach [var="변수명"] begin="시작값" end="끝감" [step="증가치"]>
  몸체
</c:forEach>
```

<c:forTokens> 태그는 문자열을 구분자로 분리해서 하나씩 추출할 때 사용

- <c:forTokens> 태그 형식

```
<c:forTokens var="토큰을 저장할 변수" items="토큰으로 나눈 문자열" delims="구분자">
  몸체
</c:forTokens>
```

4.4.3 <c:import>

다른 페이지의 내용을 포함시키기 위해 사용
페이지의 내용을 변수에 저장할 수 있음

- <c:import> 태그 형식

```
<c:import url="URL" [var="변수명"] [scope="영역"] [charEncoding="charEncoding"]>
</c:import>
```

4.4.4 <c:url>

이후에 여러 번 반복되어 사용될 수 있는 주소를 변수에 담아

- <c:url> 태그 형식

```
<c:url value="URL" [var="변수명"] [scope="영역"]>
</c:url>
```

4.4.5 <c:redirect>

지정한 페이지로 이동시키기 위해 사용

- <c:redirect> 태그 형식

```
<c:redirect url="URL" [context="경로명"]>
```

4.4.6 <c:out>

value 속성에 지정한 문자열 혹은 변수의 내용을 출력할 때 사용하는 태그

- <c:out> 태그 형식

```
<c:out [var="변수명"] value="value" [default="기본값"]>
```

변수명을 선언해 값을 담을 수도 있으며, 지정한 출력값이 없을 경우 출력될 기본값을 지정할 수 있다.

4.4.7 <c:catch>

예외처리를 위한 태그

- <c:catch> 태그 형식

```
<c:catch var="변수 이름" >
  예외가 발생할 수 있는 코드
</c:catch>
```

var : 예외의 내용이 들어가는 변수

4.5 JSTL fmt 태그

JSTL을 국제화 하기 위한 목적으로 JSP 페이지에서 다양한 언어를 지원 받을 수 있도록 한다

또한, 날짜와 숫자의 출력 형식을 다루는 데 사용된다

기능	태그	설명

숫자 날짜 형식	formatNumber	포맷에 맞는 숫자를 출력
	formatDate	날짜 정보를 포맷팅하여 출력
	parseDate	문자열을 날짜 형식으로 파싱
	parseNumber	문자열을 숫자 형식으로 파싱
	setTimeZone	시간대를 지정
	timeZone	시간대를 지정
로케일 지정	setLocale	국제화 태그들의 로케일을 지정
	requestEncoding	요청 파라미터의 인코딩 지정
메시지 처리	bundle	태그 몸체에서 사용하는 리소스 번들 지정
	message(param)	메시지 출력
	setBundle	특정 리소스 번들을 사용할 수 있도록 로딩

- JSTL fmt 태그를 사용하기 위한 taglib 지시자

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jstl/fmt"%>
```

숫자 날짜 형식 지정 태그 1 <fmt:formatNumber>

숫자 형식을 지정하기 위한 태그로 사용

- <fmt:formatNumber> 태그 형식

```
<fmt:formatNumber value="수치 데이터"
  [type="{number | currency | percent}"]
  [pattern="패턴"]
  [currencySymbor="화폐 단위"]
  [groupingUsed="{true | false}"]
  [var="변수 이름"]
  [scope="page | request | session | application]"] />
```

value : 형식화할 수치 데이터

type : 숫자, 통화 퍼센트 중 표시될 형식 지정

pattern : 지정한 형식 패턴

currencySymbol : 통화 기호, 통화 형식일 때만 적용

groupingUsed : 콤마처럼 단위를 구분할 때 사용하는 기호의 표시 여부 결정
var : 형식 출력 결과 문자열을 담는 scope에 해당하는 변수를 지정
scope : var 속성에 지정한 변수가 효력을 발휘하는 영역 지정

구분 기호를 표시할 지 결정하는 groupingUsed가 기본적으로 true로 지정되어 있어 value 속성에 지정한 수치 데이터를 출력할 때 단위를 구분하기 위해 콤마를 세자리 마다 한 번씩 찍게된다. 표시하지 않으려면 false로 지정한다.

type 속성에 percent를 지정하면 수치 데이터를 퍼센트로 표시하게 된다. 이 때 0.5를 데이터로 사용하면 50%로 출력하므로 수치 지정에 주의를 기울여야 한다.
통화 형식으로 출력하기 위해서는 type에서 currency를 지정한다.

숫자 날짜 형식 지정 태그 2 <fmt:formatDate>

날짜 형식을 지정하기 위한 태그로 사용

- <fmt:formatDate> 태그 형식

```
<fmt: formatDate value="date"
  [type="{time | date | both}"]
  [dateStyle="{default | short | medium | long | full}"]
  [timeStyle="{default | short | medium | long | full}"]
  [pattern="customPattern"]
  [timeZone="timeZone"]
  [var="변수 이름"]
  [scope="page | request | session | application]"] />
```

value : 형식화될 Date와 time (java.util.Date 타입 사용)
type : 시간, 날짜, 시간날짜 모두 중 형식화할 데이터의 형태를 지정
dateStyle : 미리 정의된 날짜 형식 중 선택
timeStyle : 미리 정의된 시간 형식 중 선택
pattern : 사용자가 지정한 형식을 사용
timeZone : 형식화 시간에 나타날 타임존
var : 형식 출력 결과 문자열을 담는 scope에 해당하는 변수를 지정

scope : var 속성에 지정한 변수가 효력을 발휘하는 영역 지정

날짜만 출력하고 싶다면 type에 date를 지정하고, 시간만 출력하고 싶다면 time을 지정한다. 둘 모두를 출력하고자 한다면 both를 지정하면 된다. 기본적으로 제공하는 형식을 사용하지 않고 사용자가 직접 형식을 바꾸고 싶다면 pattern에 yy, MM, dd, hh, mm, ss 등의 날짜 시간 형식 지정자를 사용하여 표현할 수 있다.

4.6 JSTL Function 태그

EL에서 사용될 수 있는 함수 태그를 제공한다. 일반적으로 태그형식으로 사용하는 것이 아닌, EL내에서 사용하여 함수식에 맞게 값을 변환 후 출력하므로 헷갈리지 않길 바란다.

함수	설명
length(obj)	obj가 Conllection인 경우 저장된 항목의 개수를, 문자인 경우 문자열의 길이를 반환
toUpperCase(str)	str을 대문자로 변환
toLowerCase(str)	str을 소문자로 변환
substring(str, idx1, idx2)	str.substring(idx1, idx2)의 결과를 반환, idx2가 -1일 경우 str.substring(idx1)과 동일
substringBefore(str1, str2)	str1에 포함되어 있는 str2 이전의 문자열을 구함
substringAfter(str1, str2)	str1에 포함되어 있는 str2 이후의 문자열을 구함
trim(str)	str 좌우의 공백 문자를 제거
replace(str, src, dest)	str에 있는 src를 dest로 변환
indexOf(str1, str2)	str1에서 str2가 위치한 인덱스를 구함
startsWith(str1, str2)	str1이 str2로 시작할 경우 true, 그렇지 않을 경우 false
endsWith(str1, str2)	str1이 str2로 끝나는 경우 true, 그렇지 안을 경우 false
contains(str1, str2)	st1이 str2를 포함하고 있을 경우 true를 반환
split(str1, str2)	str2로 명시한 글자를 기준으로 str1을 분리해서 배열로 반환

- JSTL Function 태그를 사용하기 위한 taglib 지시자

```
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions"%>
```

함수들의 쓰임은 java, javascript의 함수와 동일하므로, 사용법만 간단하게 알아보자.

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions"%>
<c:set var="str" value="Hello World"/>
length(str) : ${ fn:length(str) }<br>
replace(str, src, dest) : ${ fn: replace(str, "H", "h") } <br>
toUpperCase(str) : ${ fn: toUpperCase(str) }<br>
```

이렇듯 일반적인 태그 선언 방식이 아닌 EL 이내에 'fn: 함수명() '으로 함수를 호출하여 사용하면 되겠다.

Chapter 16. 공공데이터 활용

Server프로그래밍을 활용하여 정부에서 제공하는 공공데이터를 활용하는 방법에 대해서 알아보자. 공공데이터는 공공의 목적을 위해 정부에서 제공하는 공공기관이 수집한 데이터로 대기 오염, 코로나 현황, 지역별 날씨에 대한 정보, 상점 상권분석 등의 데이터를 정부의 각 기관에서 제공하고 있다. 이 수많은 공공데이터를 이용하면 훨씬 폭넓고 다양한 정보를 제공하는 웹애플리케이션을 개발할 수 있다.

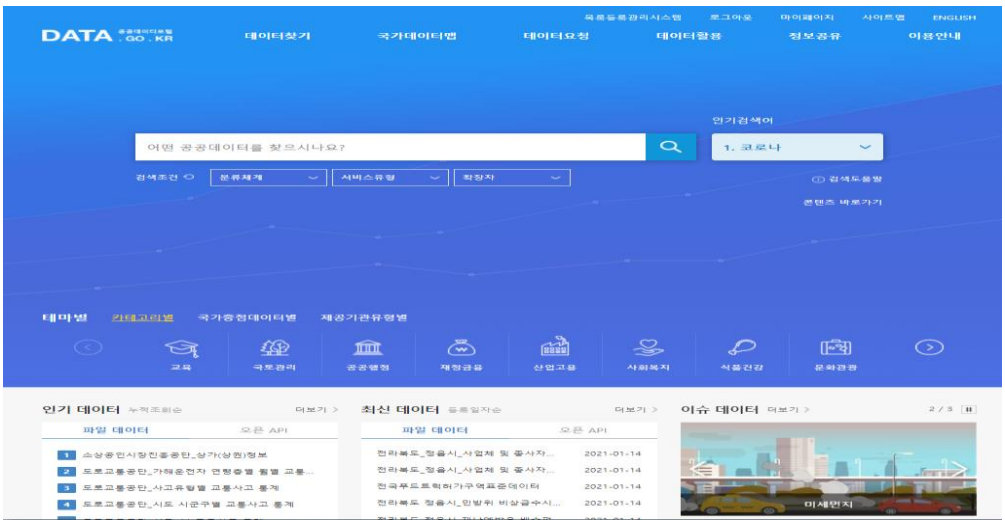
서버에서 공공데이터를 활용하는 방법은 공공데이터를 제공하는 공공데이터 포털에서 오픈API를 이용하여 요청을 보내 데이터를 받아와 java로 변환하는 방식으로 이용할 수 있다. 오픈 API를 이용해서 공공데이터를 받아와서 처리하는 방법에 대해 알아보자.

1 공공데이터 포털이용하기

먼저 공공데이터를 이용해서 웹 애플리케이션을 개발하려면 내가 개발하려는 앱 애플리케이션에서 필요한 공공데이터가 무엇있고 어떻게 구성이 되어있는 확인을 하고 필요한 데이터를 선정해야한다.

필요한 데이터를 선정하기 위해서는 공공데이터 정보를 제공하는 공공데이터 포털에 접속하여 조회하면 간단하게 어떤한 데이터가 있는지 확인이 가능한다.

* https://www.data.go.kr/index.do



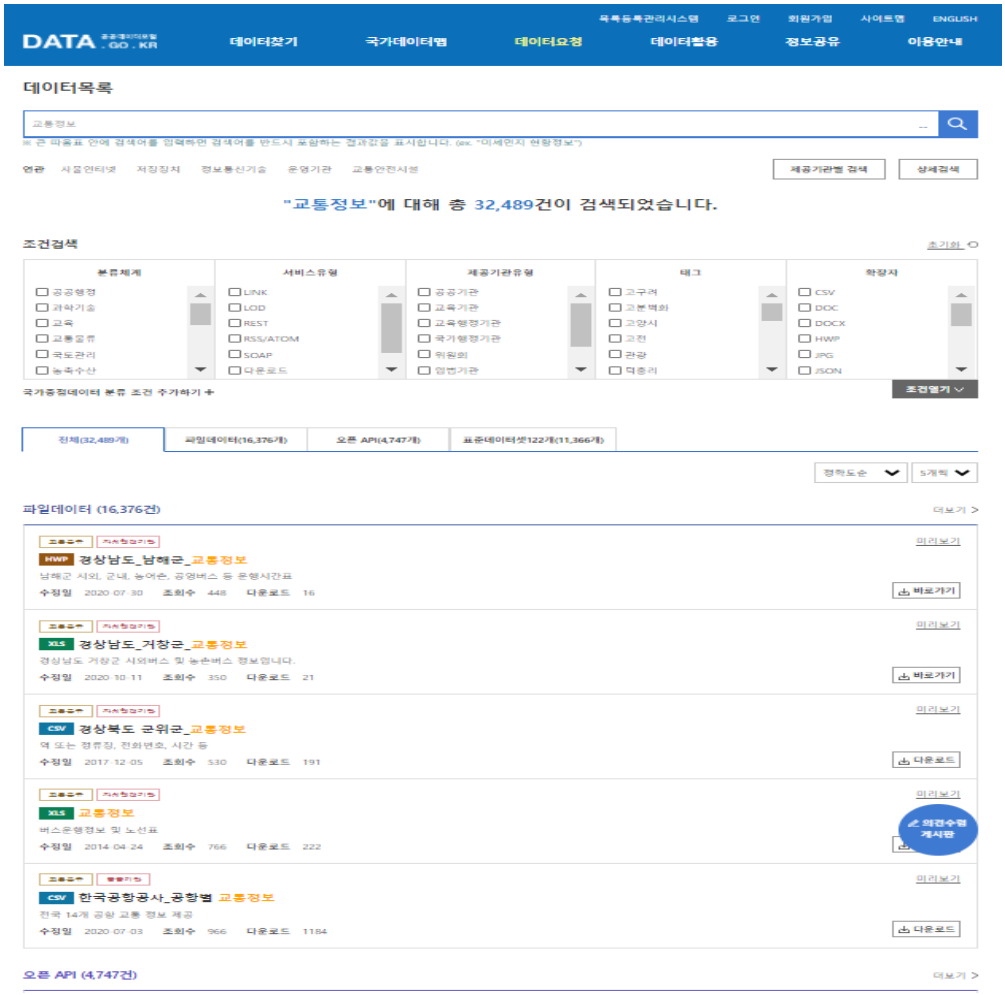
메인화면을 보면 통합으로 공공데이터를 조회할 수 있는 화면이 나오고 위 검색창에 원하는 데이터를 keyword로 검색하여 데이터를 조회할 수 있음. 또한 아래 인기데이터나 신규 데이터에 대한 순위를 링크로 제공하고 있다.

공공데이터 포털에서 제공하는 데이터는 크게 두가지로 나눌 수 있는데 첫번째는 파일 형식으로 데이터를 제공하는것이고 두번째는 오픈 API로 제공하는 것이다.

파일로 제공하는 형태는 파일을 다운로드하여 공공데이터를 이용하는 것이고, 오픈API로 제공되는 것은 서버에서 지정된 URL주소로 요청을 보내 해당 공공데이터를 받아와 이용을 하는 것이다.

기본적인 데이터 조회를 해서 그 결과가 어떻게 나오는지 확인해보겠다.

공공데이터 포털 검색창에 "교통정보"를 검색해 보자.

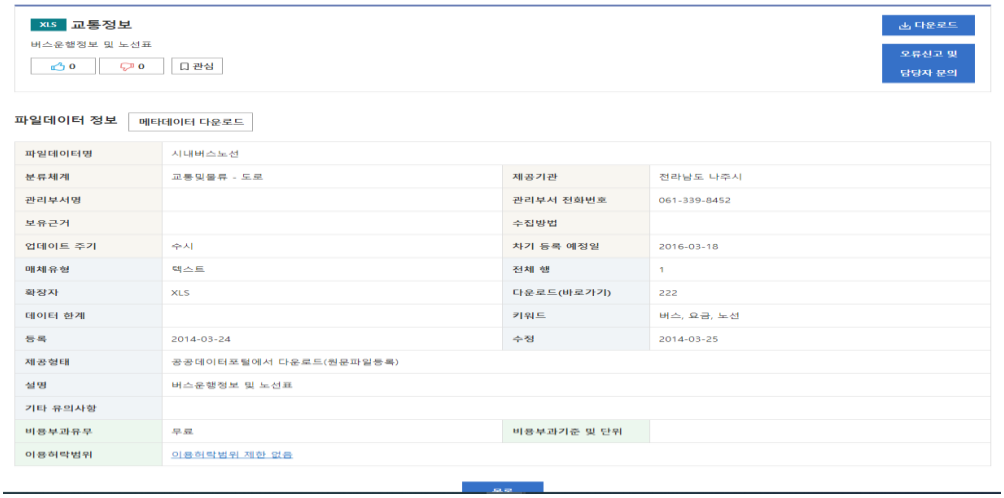


검색창에 입력을 하고 조회를 하면 교통정보와 관련된 데이터가 조회되어 나오게 된다. 그리고 검색결과 창에는 상세필터를 할 수 있게 되어 있고, 데이터는 유형별(파일, 오픈API등) 나누어서 조회되는 것을 알 수 있다.

또 검색결과에 대해 조건을 탭으로 제공하여 쉽게 원하는 데이터를 찾을 수 있다.

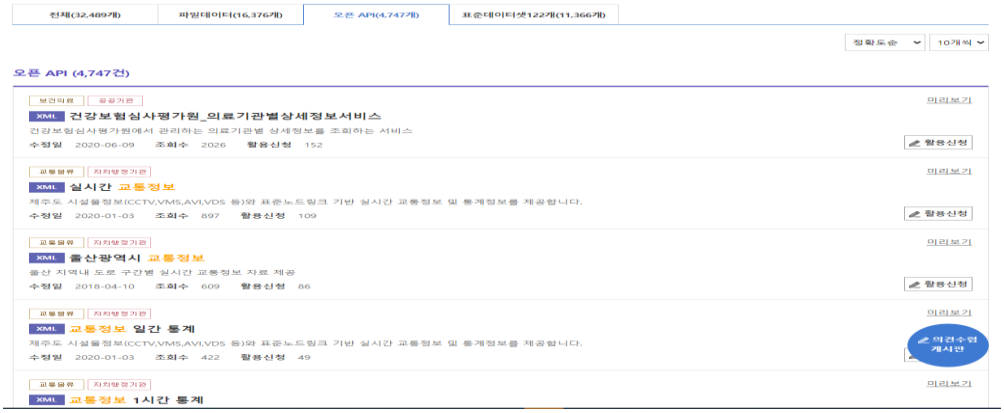
검색된 결과를 클릭을 하면 데이터의 상세내용으로 들어갈 수 있다.

검색된 내용 중 "교통정보" 데이터를 클릭해보자.



파일로 제공되는 데이터로 제공되는 파일의 형식, 내용, 다운로드 버튼 등이 있는 것을 확인할 수 있다. 파일로 제공되는 것은 위에서 설명한것과 같이 파일을 받아서 서버로 읽어와서 사용할 수 있다. 지금 파일은 xls로 엑셀파일로 데이터를 제공하고 있다. 이번 실습에서는 오픈 API를 이용하기 때문에 더 자세한 내용은 생략하겠다.

이번엔 오픈API데이터를 확인해 보도록 하겠다. 탭에서 오픈API를 선택하면 오픈API형식으로 제공하는 데이터만 볼 수 있고 그 중 "실시간 교통정보"데이터를 클릭해보자.



상세화면은 파일로 제공하는 화면과는 다른다. 일단 오픈API로 데이터를 이용하려면 활용신청을 하여 API이용 인증키를 받아야한다. 인증키를 받는 방법에 대해서는 차후 설명하도록 하겠다.



XML 실시간 교통정보

제주도 시설물정보(CCTV,VMS,AVI,VDS 등)와 표준노드링크 기반 실시간 교통정보 및 통계정보를 제공합니다.

0 0 관심

활용신청

오류신고 및 담당자 문의

OpenAPI 정보 메타데이터 다운로드

분류체계	교통및물류 - 도로	제공기관	제주특별자치도
관리부서명	자치경찰단 교통정보센터	관리부서 전화번호	064-710-6411
API 유형	REST	데이터포맷	XML
활용신청	109	키워드	교통정보, 시설물, 신호
등록	2015-03-25	수정	2015-03-30
심의유형	개발단계 : 허용 / 운영단계 : 허용		
비용부과유무	무료		
이용허락범위	이용허락범위 제한 없음		
참고문서	제주도 실시간교통정보 OpenAPI 활용가이드(Use ServiceKey) v1.1.doc		

상세기능 활용사례

상세기능

목록 실시간 교통정보 조회

표준링크 기반으로 제주도 실시간 소통정보를 제공한다.

상세화면 아래부분으로 가면 오픈API에 요청을 보내는 방법과 요청에 대한 응답 방식이 상세하게 나와있다.



표준링크 기반으로 제주도 실시간 소통정보를 제공한다.

· 활용승인 절차 개발단계 : 허용 / 운영단계 : 허용
· 신청가능 트래픽 10000000 / 운영계정은 활용사례 등록시 신청하면 트래픽 증가 가능
· 요청주소
· 서비스URL http://openapi.jejuits.go.kr/rfcapi/rest/jejuits/getTrafficInfo

활용신청

요청변수(Request Parameter)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터	항목설명
N/A	N/A	N/A	N/A	N/A	N/A

출력결과(Response Element)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터	항목설명
구간 ID	LINK_ID	10	1	4055393300	표준링크ID
가공일시	PRCN_DT	14	1	20150103142221	정보가공일시
교통량	TFVL	6	1	26	교통량(대)
평균속도	SPED	3	1	53	평균속도(km/h)
점유율	OCPY_RATE	5	1	30	평균점유율(%)
통행시간	TRVL_HH	6	1	120	통행시간(초)

이용하는 방법에 대한 참고용 소스코드도 제공하고 있어 서비스를 이용하는데 편리하다.

샘플코드

Java | Javascript | C++ | PHP | Curl | Objective-C | Python | Nodejs

```
/* Java 샘플 코드 */


import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.io.BufferedReader;
import java.io.IOException;

public class ApiExplorer {
    public static void main(String[] args) throws IOException {
        StringBuilder urlBuilder = new StringBuilder("http://openapi.jejuits.go.kr/rfcapi/rest/jejuits/getTrafficInfo"); /*URL*/
        urlBuilder.append("?" + URLEncoder.encode("ServiceKey","UTF-8") + "=서비스키"); /*Service Key*/
        urlBuilder.append("&" + URLEncoder.encode("N/A","UTF-8") + "=" + URLEncoder.encode("N/A", "UTF-8")); /*N/A*/
        URL url = new URL(urlBuilder.toString());
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        conn.setRequestProperty("Content-type", "application/json");
        System.out.println("Response code: " + conn.getResponseCode());
        BufferedReader rd;
        if(conn.getResponseCode() >= 200 && conn.getResponseCode() <= 300) {
            rd = new BufferedReader(new InputStreamReader(conn.getInputStream()));
        } else {
            rd = new BufferedReader(new InputStreamReader(conn.getErrorStream()));
        }
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = rd.readLine()) != null) {
            sb.append(line);
        }
        rd.close();
        conn.disconnect();
        System.out.println(sb.toString());
    }
}
```

* 샘플로 java 예제를 살펴보았다.

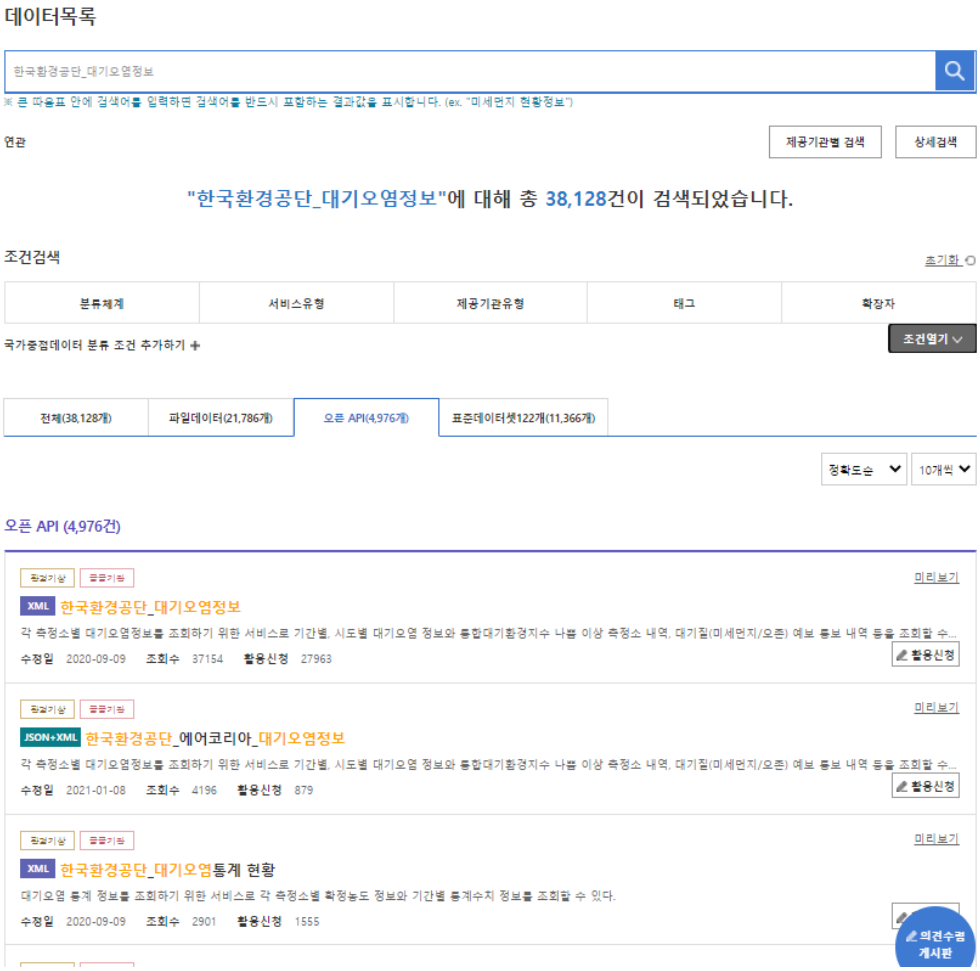
2 공공데이터를 이용해서 서비스 구현하기

2.1 대기오염정보 제공 서비스 구축하기

2.1.1 공공포털에서 제공하는 대기오염현황 조회하기

1. 공공포탈에 접속하여 검색창에 "대기오염정보" 검색하기
2. 오픈 API탭 선택
3. "한국환경공단_대기오염정보" 클릭

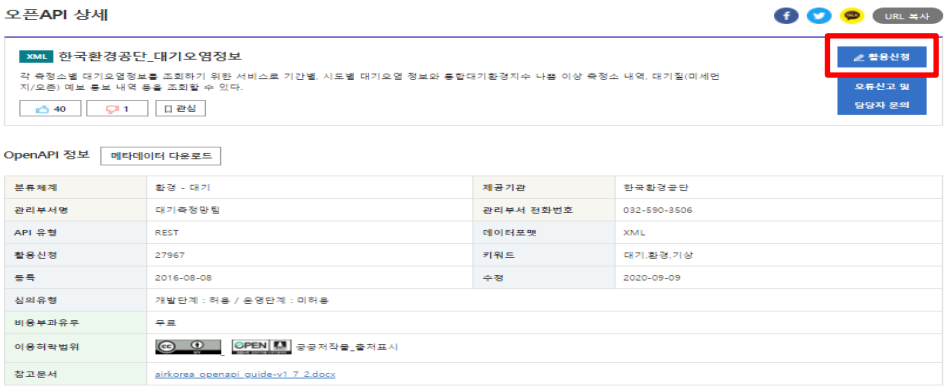
   * 한국환경공단_대기오염정보를 바로 검색하면 첫번째 데이터로 출력



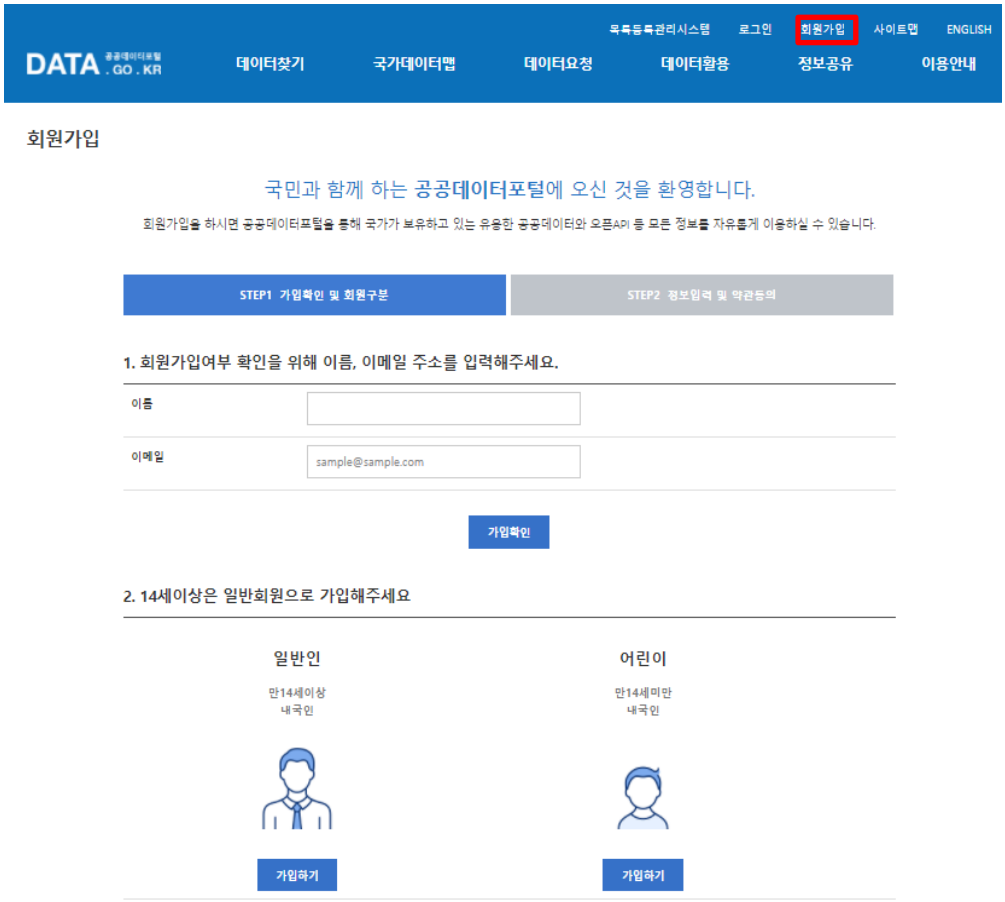
2.1.2 오픈API이 활용신청하기

오픈 API를 사용하려면 활용신청 후 인증키를 부여받은 후 사용이 가능하다.

상세화면에서 활용신청 버튼을 클릭한다.



활용신청을 하기 위해서는 회원가입이 필요하다.



이름과 이메일을 입력하고 가입확인 후 가입을 진행한다. 이메일 인증과 핸드폰인증이 있으니 이메일을 반드시 본인이 쓰고 있는 메일주소로 작성해야한다. 세부정보를 입력하고 가입을 완료한다.



STEP1 가입확인 및 회원구분 | STEP2 정보입력 및 약관동의

기본정보 입력 *표시는 필수 입력항목입니다.

* 아이디 [ ] 중복확인
4~20자 이내 영문 소문자, 숫자만 가능합니다.

* 비밀번호 [ ]
반복, 연속문자, 개인정보 관련은 사용불가
숫자, 영문자, 특수문자(!, @, #, $, %, ^, &, +, =, ~, *) 조합 9자이상 ~ 20자 이하

* 비밀번호 확인 [ ]

개인정보입력

* 이름 [유병승]

* 이메일 [prince215312@gamil.com] 이메일 인증 ☐ 이메일 수신 동의
수신거부 시 일부 서비스에 제한이 있을 수 있습니다.
비밀번호 초기화 메일 수신 등에 반드시 필요한 정보이므로 정확히 이메일 주소를 입력해주세요.

전화번호 [ ]

* 휴대전화번호 [ ]
☐ SMS 수신 동의
수신거부 시 일부 서비스에 제한이 있을 수 있습니다.

* 일반회원 인증 휴대폰 인증 본인 인증하기

* 자동등록방지 yrew7 새로고침
보안문자를 입력하세요.

☐ 공공데이터포털 서비스 이용약관에 동의합니다.

제1조 목적
본 약관은 공공데이터 포털(이하 '포털'이라 합니다)에서 제공되는 서비스의 이용과 관련하여, 이용조건 및 절차, 회원과 운영기관의 권리·의무 및 책임, 그 외 필요한 사항을 규정함을 목적으로 합니다.

제2조 용어의 정의
본 약관에서 사용하는 용어의 정의는 다음과 같습니다.

1. "회원"이란 포털에서 제공하는 소정의 회원가입절차에 따라 이용약관에 동의하고 가입을 완료한 법인 또는 개인을 말합니다.
2. "이용자"란 포털에 회원으로 가입하고 포털이 제공하는 서비스를 이용하는 자를 말하며, 공공기관을 제외합니다.

☐ (필수) 개인정보 수집 및 이용에 동의합니다. ☐ (선택) 개인정보 수집 및 이용에 동의합니다.

가. 수집하는 개인정보의 항목
공공데이터포털은 다음의 개인정보 항목을 수집하고 있습니다.
○. 공공데이터포털 회원정보
• 회원가입 시 입력해야 하는 필수항목과 선택항목이 있으며, 선택항목은 입력하지 않으시더라도 서비스 이용에는 제한이 없습니다.
• 일반회원가입
- 필수항목 : 이름, 아이디, 비밀번호, 이메일, 휴대전화번호
- 선택항목 : 전화번호
• 서비스 이용과정에서 아래와 같은 정보들이 자동으로 생성되어 수집될 수 있습니다.
- 항목 : 방문 일시, 서비스 이용기록, IP 주소

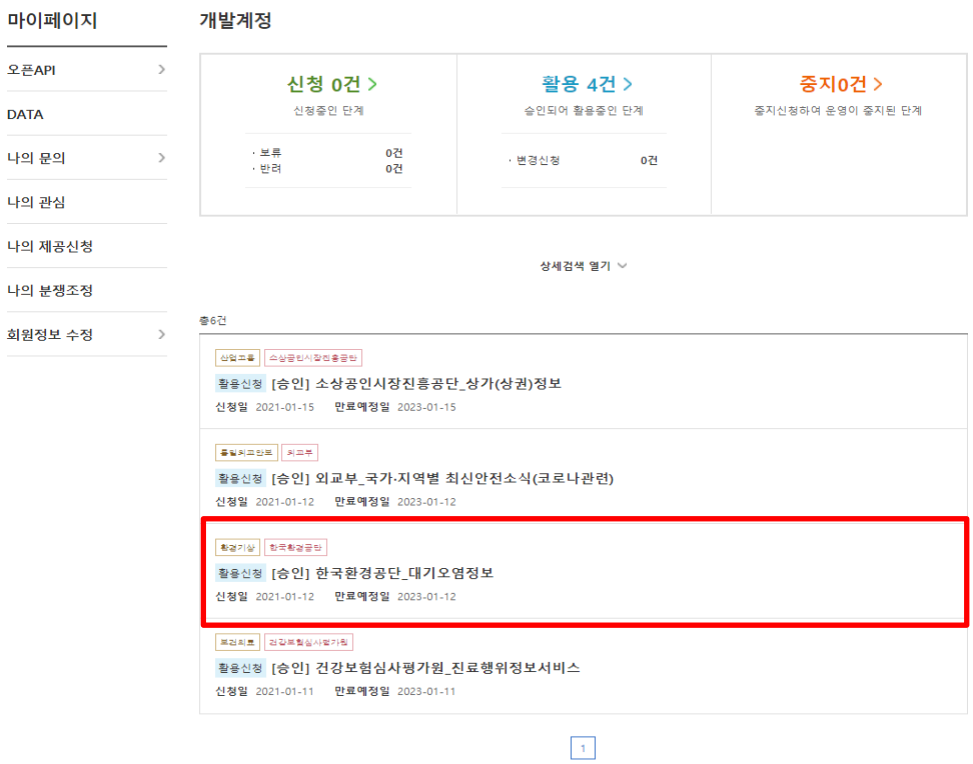
☐ 전체 약관에 동의합니다.

취소 작성 완료

세부정보를 입력하고 가입을 완료 후 로그인하고 활용신청을 다시 진행한다. 활용신청을 하면 활용 목적을 작성하는데 참고자료선택 "직업훈련교육 참고자료"로 작성하자.



활용신청을 하고 나면 마이페이지에서 신청한 데이터에 대한 정보가 확인이 가능하다. 신청후 발급된 인증키로 데이터를 이용하려면 1~2 시간이 소요된다.



승인된 데이터의 상세내용에 들어가면 승인받은 공공데이터에 대한 상세정보와 인증키를 확인할 수 있다. 대기오염정보 중 여러가지 조회기능을 제공하는것도 확인할 수 있음 이중 내가 원하는 데이터를 활용하면 된다. 서비스 정보부분을 보면 참고 문서가 있는데 참고문서에는 데이터활용에 대한 더 상세한 정보가 있기 때문에 공공데이터를 이용할때는 이 문서를 이용하는 것이 좋다.



그리고 활용신청 상세기능 정보 테이블을 보면 미리보기 기능을 제공하고 있다. 우리는 다양한 정보 중 시도별 실시간 측정정보 조회데이터를 이용해보도록 하겠다. 이용하기 전에 미리보기로 어떠한 데이터가 넘어오는 지 확인해보자. 확인버튼을 클릭하면 요청시 보내야하는 파라미터값이 보이고 default 값을 설정된 것이 확인되고 미리 보기 버튼을 클릭하면 API 가 제공하는 데이터가 페이지에 반환되는 것을 확인할 수 있다. 반환값을 확인해 보자.

활용신청 상세기능정보

NO	상세기능	설명	일일 트래픽	미리보기
1	측정소별 실시간 측정정보 조회	측정소명과 측정데이터 기간(일, 한달, 3개월)으로 해당 측정소의 일반항목 측정정보를 제공하는 측정소별 실시간 측정정보조회	500	확인
2	통합대기환경지수 나쁨 이상 측정소 목록조회	통합대기환경지수가 나쁨 등급 이상인 측정소명과 주소 목록 정보를 제공하는 통합대기환경지수 나쁨 이상 측정소 목록조회	500	확인
3	시도별 실시간 측정정보 조회	시도명을 검색조건으로 하여 시도별 측정소목록에 대한 일반 항목과 CAI 최종 실시간 측정값과 지수정보 조회 기능을 제공하는 시도별 실시간 측정정보 조회	500	확인

요청변수(Request Parameter) 닫기

항목명	샘플데이터	설명
numOfRows	10	한 페이지 결과 수
pageNo	1	페이지 번호
sidoName	인천	시도 이름 (서울, 부산, 대구, 인천, 광주, 대전, 울산, 경기, 강원, 충북, 충남, 전북, 전남, 경북, 경남, 제주, 세종)
ver	1.3	버전별 상세 결과 참고문서 참조

미리보기

4	대기질 예보통보 조회	통보코드와 통보시간으로 예보정보와 발생 원인정보를 조회하는 대기질(미세먼지/오존) 예보통보 조회	500	확인
5	시도별 실시간 평균정보 조회	시도별 측정소목록에 대한 일반 항목의 시간 및 일평균 자료 및 지역 평균 정보를 제공하는 시도별 실시간 평균정보 조회	500	확인
6	시군구별 실시간 평균정보 조회	시도의 각 시군구별 측정소목록의 일반 항목에 대한 시간대별 평균농도를 제공하는 시군구별 실시간 평균정보 조회	500	확인

활용정보

활용목적	참고자료
활용내용	직업훈련을 위한 참고자료

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
<response>
  <header>
    <resultCode>00</resultCode>
    <resultMsg>NORMAL SERVICE.</resultMsg>
  </header>
  <body>
    <items>
      <item>
        <stationName>신흥</stationName>
        <mangName>도시대기</mangName>
        <dataTime>2021-01-16 16:00</dataTime>
        <so2Value>0.005</so2Value>
        <coValue>0.5</coValue>
        <o3Value>0.037</o3Value>
        <no2Value>0.011</no2Value>
        <pm10Value>41</pm10Value>
        <pm10Value24>56</pm10Value24>
        <pm25Value>26</pm25Value>
        <pm25Value24>40</pm25Value24>
        <khaiValue>116</khaiValue>
        <khaiGrade>3</khaiGrade>
        <so2Grade>1</so2Grade>
        <coGrade>1</coGrade>
        <o3Grade>2</o3Grade>
        <no2Grade>1</no2Grade>
        <pm10Grade>2</pm10Grade>
        <pm25Grade>3</pm25Grade>
        <pm10Gradelh>2</pm10Gradelh>
        <pm25Gradelh>2</pm25Gradelh>
      </item>
      <item>
        <stationName>서해</stationName>
        <mangName>도로변대기</mangName>
        <dataTime>2021-01-16 16:00</dataTime>
        <so2Value>0.001</so2Value>
        <coValue>0.4</coValue>
        <o3Value>0.037</o3Value>
        <no2Value>0.009</no2Value>
        <pm10Value>48</pm10Value>
        <pm10Value24>55</pm10Value24>
        <pm25Value>22</pm25Value>
        <pm25Value24>37</pm25Value24>
        <khaiValue>105</khaiValue>
        <khaiGrade>3</khaiGrade>
        <so2Grade>1</so2Grade>
        <coGrade>1</coGrade>
        <o3Grade>2</o3Grade>
        <no2Grade>1</no2Grade>
        <pm10Grade>2</pm10Grade>
        <pm25Grade>3</pm25Grade>
        <pm10Gradelh>2</pm10Gradelh>
        <pm25Gradelh>2</pm25Gradelh>
      </item>
      <item>
        <stationName>영흥</stationName>
        <mangName>도시대기</mangName>
        <dataTime>2021-01-16 16:00</dataTime>
        <so2Value>0.003</so2Value>
        <coValue>0.4</coValue>
        <o3Value>0.020</o3Value>
        <no2Value>0.005</no2Value>
```


데이터 형식이 xml 로 되어있어 xml 파일 형식으로 반환되는 것을 볼 수 있다. 참고로 서버에서 해당 데이터를 가져오는 서비스를 구현할 때 위 주소를 이용하면 동일하게 데이터를 가져올 수 있다.

2.1.3 공공데이터 분석하기

공공데이터에서 반환되는 데이터에 대한 정보를 분석하는 과정으로 데이터를로 서버에서 해당를 서버에서 받아서 클라이언트에게 응답을 해주기 위해 필요한 데이터로 정제화하는 작업이라고 보면 된다.

반환되어오는 데이터의 상세 정보를 확인하려면 참고문서를 확인하면 된다. 참고문서를 열어보면 각 서비스에 대한 상세한 정보가 나온다.

대기오염정보 조회 서비스를 웹애플리케이션에 제공할 예정으로 관련 내용을에 대해 살펴보자.

▪ 가. 서비스 개요

서비스 정보	서비스 ID	SC-OA-09-02	
	서비스명(국문)	대기오염정보 조회 서비스	
	서비스명(영문)	ArpltnInforInqireSvc	
	서비스 설명	각 측정소별 대기오염정보를 조회하기 위한 서비스로 기간별, 시도별 대기오염 정보와 통합대기환경지수 나쁨 이상 측정소 내역, 대기질(미세먼지/오존) 예보 통보 내역 등을 조회할 수 있다.	
서비스 보안	서비스 인증/권한	[O] 서비스 Key [ ] 인증서 (GPKI) [ ] Basic (ID/PW) [ ] 없음	[ ]WS-Security
	메시지 레벨 암호화	[ ] 전자서명 [ ] 암호화 [O] 없음	
	전송 레벨 암호화	[ ] SSL [O] 없음	
적용 기술 수준	인터페이스 표준	[ ] SOAP 1.2(RPC-Encoded, Document Literal, Document Literal Wrapped) [O] REST (GET) [ ] RSS 1.0 [ ] RSS 2.0 [ ] Atom 1.0 [ ] 기타	
	교환 데이터 표준	[O] XML [O] JSON [ ] MIME [ ] MTOM	
서비스 URL	개발환경	http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc	
	운영환경	http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc	
서비스 배포 정보	서비스 버전	1.1	
	서비스 시작일	2013-12-05	배포 일자 2013-12-05
	서비스 이력	(갱신) 2016-08-08 : 시도별 실시간 평균정보 조회 및 시군구별 실시간 평균정보 조회 오퍼레이션 추가	
메시지 교환 유형		[O] Request-Response [ ] Publish-Subscribe [ ] Fire-and-Forgot [ ] Notification	
메시지 로깅 수준		성공 [O] Header [ ] Body	실패 [O] Header [O] Body
사용 제약 사항 (비고)		N/A	
데이터 갱신주기		- 실시간 정보 : 10분(매 시간 시간자료 갱신은 20분 전후로 반영됨) - 대기질 예보 정보 : 매 시간 22분, 57분	

※ JSON 방식 호출 방법 : 호출 시 다음의 파라미터(&_returnType=json)를 추가하여 호출

먼저 서비스 개요가 나오는데 서비스에 대한 주요 정보들이 나온다. 여기서

중요한 것은 서비스에 대한 주요 내용, 데이터 반환 방식(JSON, XML 등), 데이터 갱신주기 등을 확인해 보면 된다. 관련정보를 숙지해서 서비스는 제공하는 시기(갱시주기)와 어떤 정보를 제공할 수 있는지 파악할 수 있다.

오퍼레이션 목록은 말그대로 조회되는 서비스에 대한 이름을 의미한다고 보면 된다. *상세화면에서 미리보기 샘플이 있었던 내용들이다.

▪ 나. 오퍼레이션 목록

일련번호	서비스명(국문)	오퍼레이션명(영문)	오퍼레이션명(국문)	메시지명(영문)
1	대기오염 정보 조회 서비스	getMsrstnAcctoRltmMesureDnsty	측정소별 실시간 측정정보 조회	N/A
2		getUnityAirEnvrnIdexSnstveAboveMsrstnList	통합대기환경지수 나쁨 이상 측정소 목록조회	N/A
3		getCtprvnRltmMesureDnsty	시도별 실시간 측정정보 조회	N/A
4		getMinuDustFrcstDspth	대기질 예보통보 조회	N/A
5		getCtprvnMesureLIst	시도별 실시간 평균정보 조회	N/A
6		getCtprvnMesureSidoLIst	시군구별 실시간 평균정보 조회	N/A

이 오퍼레이션 중 서비스할 오퍼레이션을 선택하여 분석하면 된다. 이중 시도별 실시간 측정정보조회 정보확인 하고 제공해보자. 아래와 그림과 같이 각 서비스별 오퍼레이션 명세가 있는데 이것을 확인하여 데이터를 확인하고 분석할 수 있다.

▪ (3) 시도별 실시간 측정정보 조회 오퍼레이션 명세

오퍼레이션 정보	오퍼레이션 번호	3	오퍼레이션명(국문)	시도별 실시간 측정정보 조회
	오퍼레이션 유형	조회	오퍼레이션명(영문)	getCtprvnRltmMesureDnsty
	오퍼레이션 설명	시도명을 검색조건으로 하여 시도별 측정소목록에 대한 일반 항목과 CAI 최종 실시간 측정값과 지수 정보 조회 기능을 제공하는 시도별 실시간 측정정보 조회		
	URL end-point	http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty		
	최대 메시지 사이즈	[1000K bytes]		
	평균 응답 시간	[500 ms]	초당 최대 트랜잭션	[30 tps]

▪ 요청 메시지 명세

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
ServiceKey	서비스키	-	1	aAb12.....%30	서비스키
numOfRows	한 페이지 결과 수	4	0	10	한 페이지 결과 수
pageNo	페이지 번호	4	0	1	페이지 번호
sidoName	시도 명	10	1	서울	시도 이름 (서울, 부산, 대구, 인천, 광주, 대전, 울산, 경기, 강원, 충북, 충남, 전북, 전남, 경북, 경남, 제주, 세종)
ver	오퍼레이션 버전	4	0	1.0	버전별 상세 결과 아래쪽 참고

서비스별 오퍼레이션 명세가 있는데 이것을 확인하여 데이터를 확인하고 분석할 수 있다. 오퍼레이션 정보에서는 요청을 하기 위한 URL 주소, 요청에 대한 서비스 주요내용이 있으며 요청하기 위한 Parameter(요청메세지 명세)들이 명시가 되어있다.

요청주소 : http://openapi.airkorea□.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty

ServiceKey : 활용신청을 했을 때 부여 받은 인증키을 값으로 넣어 보내면 된다. 필수값

numOfRows : 한페이지 결과값 선택값 기본 10

pageNo : 페이지번호 선택값 기본 1

sidoName : 조회한 지역에 대한 이름, 항목설명에 있는 지명으로 작성 필수값

* 파라미터에는 필수적으로 넣어야하는 값과 선택하는 값이 있다. 선택은 설정하지않으면 기본값으로 설정됨

필수적인 값을 파라미터값에 넣고 HttpRequest를 보내면 응답하는 메시지 명세도 상세하게 나열되어있다. 데이터 분석을 위해서는 이 명세서를 잘 분석하여 DB테이블 및 DTO(VO)를 설계할 수 있다. 간단하게 보면 응답메세지는 요청시에 넘어오는 데이터들에 대한 key값과 value값을 의미하고, 데이터는 JSON방식과 XML방식으로 데이터가 반환된다. JSON에서는 항목명(영문)은 key값이 되고 value는 샘플데이터에서 있는 내용들이다. XML에서는 items태그안에 항목명(영문)이 자식 태그가되고 각 자식태그안의 값이 샘플데이터에 있는 값이 된다.

항목별 설명을 보고 웹애플리케이션 구현에 필요한 내용을 추려내면 되겠다. 추려진 데이터의 key값과 샘플데이터의 내용을 확인해서 데이터를 받을 DTO(VO)클래스를 설계할 때 참조하면 되겠다. 아래 내용을 분석하여 DTO(VO)를 설계해보자.

▪ 응답 메시지 명세

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
resultCode	결과코드	2	1	00	결과코드
resultMsg	결과메세지	50	1	[illegible]	결과메세지
numOfRows	한 페이지 결과 수	4	1	10	한 페이지 결과 수
pageNo	페이지 번호	4	1	1	페이지 번호
totalCount	전체 결과 수	4	1	40	전체 결과 수
items	목록	-	0..n	-	목록
stationName	측정소명	30	1	중구	측정소명
mangName	측정망 정보	10	1	도시대기	측정망 정보 (국가배경, 교외대기, 도시대기, 도로변대기)
dataTime	측정일시	20	1	2016-04-20 14:00	오염도 측정 연-월-일 시간 : 분
so2Value	아황산가스 농도	10	1	0.007	아황산가스 농도(단위 : ppm)
coValue	일산화탄소 농도	10	1	0.7	일산화탄소 농도(단위 : ppm)
o3Value	오존 농도	10	1	0.043	오존 농도(단위 : ppm)
no2Value	이산화질소 농도	10	1	0.043	이산화질소 농도(단위 : ppm)
pm10Value	미세먼지(PM_{10}) 농도	10	1	68	미세먼지(PM_{10}) 농도 (단위 : ㎍/㎥)
pm10Value24	미세먼지(PM_{10}) 24시간예측이동농도	10	1	56	미세먼지(PM_{10}) 24시간예측 이동농도(단위 : ㎍/㎥)
pm25Value	미세먼지($PM_{2.5}$) 농도	10	1	39	미세먼지($PM_{2.5}$) 농도 (단위 : ㎍/㎥)
pm25Value24	미세먼지($PM_{2.5}$) 24시간예측이동농도	10	1	26	미세먼지($PM_{2.5}$) 24시간예측 이동농도(단위 : ㎍/㎥)
khaiValue	통합대기환경수치	10	1	76	통합대기환경수치
khaiGrade	통합대기환경지수	10	1	2	통합대기환경지수
so2Grade	아황산가스 지수	10	1	1	아황산가스 지수
coGrade	일산화탄소 지수	10	1	1	일산화탄소 지수
o3Grade	오존 지수	10	1	2	오존 지수
no2Grade	이산화질소 지수	10	1	2	이산화질소 지수
pm10Grade	미세먼지(PM_{10}) 24시간등급자료	10	1	2	미세먼지(PM_{10}) 24시간 등급자료
pm25Grade	미세먼지($PM_{2.5}$) 24시간등급자료	10	1	2	미세먼지($PM_{2.5}$) 24시간 등급자료
pm10Grade1h	미세먼지(PM_{10}) 1시간등급자료	10	1	2	미세먼지(PM_{10}) 1시간 등급자료
pm25Grade1h	미세먼지($PM_{2.5}$) 1시간등급자료	10	1	2	미세먼지($PM_{2.5}$) 1시간 등급자료

※ 항목구분 : 필수(1), 옵션(0), 1건 이상 복수건(1..n), 0건 또는 복수건(0..n)

※ 항목별 Grade 값의 의미

- 적용 항목명 : khaiGrade, so2Grade, coGrade, o3Grade, no2Grade, pm10Grade, pm25Grade, pm25Grade1h, pm25Grade1h

등급	좋음	보통	나쁨	매우나쁨
Grade 값	1	2	3	4

위 자료의 내용을 확인하고 서비스 제공에 필요한 데이터를 추릴 수 있고 샘플데이터의 내용을 보고 DTO(VO)의 필드의 변수명 자료형을 결정할 수 있다.

전체 데이터를 모두 가져와 활용한다고 했을 때 클래스를 설계하면 다음과 같이 설계할 수 있을 것이다.

```
public class AirDto {

	private String stationName;
	private String mangName;
	private String dataTime;
	private int khaiGrade;
	private int khaiValue;
	private int pm10Grade;
	private int pm10Grade1h;
	private int pm10Value;
	private int pm10Value24;
	private int pm25Grade;
	private int pm25Grade1h;
	private int pm25Value;
	private int pm25Value24;
	private int coGrade;
	private int no2Grade;
	private int o3Grade;
	private int so2Grade;
	private double coValue;
	private double no2Value;
	private double o3Value;
	private double so2Value;
```

샘플자료를 보고 문자열형식이면 String, 정수 int, 실수형이면 double 로 설계하면 된다. 여기서 문제되는 것이 Date 를 표현하는 dataTime 인데 서버상에서 요청과 응답을 하게 되면 문자열로 처리가 되기 때문에 다른 파싱(Date 로 변환)하는 작업없이 그냥 문자열로 표시하는것으로 하겠다. 기본적으로 기본생성자와 매개변수가 있는 생성자 캡슐화에 의한 setter, getter 는 필드에 맞게 작성해주면 되겠다.

만약 받아온 데이터를 DB 에 저장하려면 테이블도 마찬가지 방법으로 설계하면 되겠다.

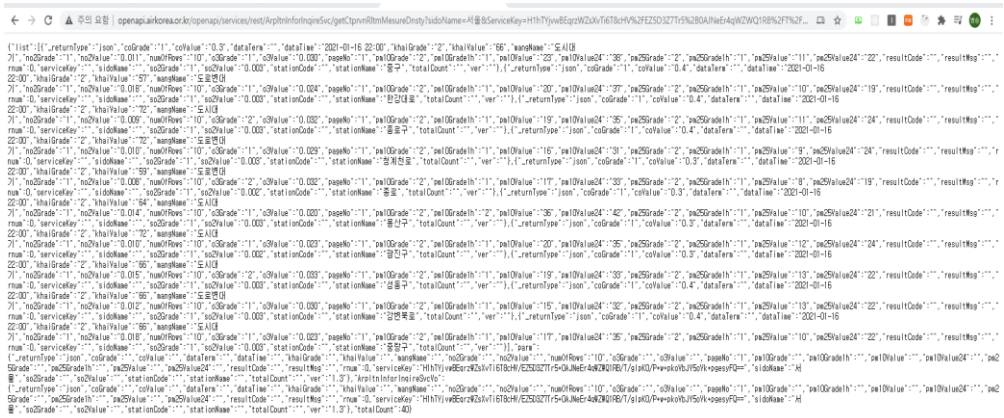
```
create table airsatus(
	idx number primary key,
	stationName varchar2(200),
	mangName varchar2(50),
	dataTime date,
	khaiGrade number,
	khaiValue number,
	pm10Grade number,
	pm10Grade1h number,
	pm10Value number,
	pm10Value24 number,
	pm25Grade number,
	pm25Grade1h number,
	pm25Value number,
	pm25Value24 number,
	coGrade number,
	no2Grade number,
	o3Grade number,
	so2Grade number,
	coValue number,
	no2Value number,
	o3Value number,
	so2Value number
);
```

웹브라우저에서 요청 URL 로 파라미터와 함께 보내면 아래와 같이 브라우저에서 반환되는 데이터를 직접확인할 수 있다.

브라우저 URL

http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty?sidoName=%EC%84%9C%EC%9A%B8&ServiceKey=H1hTYjvwBEqrzWZsXvTi6T8cHV%2FEZ5D3Z7Tr5%2B0AJNeEr4qWZWQ1RB%2FT%2FgIpK0%2FP%2Bw%2BpkoVbJY5oYk%2BpgesyFQ%3D%3D&ver=1.3&_returnType=json

- json 으로 데이터를 받는 방식, service 키는 본인이 받은 키로 입력할 것



2.1.4 웹서비스 구축하기

공공데이터에 대한 기본적인 분석을 통한 클래스 설계 및 DB설계 등이 완료가 되었으면 서버프로그래밍을 통해서 웹서비스를 구축해보도록 하자. 웹서비스는 두개의 데이터에 대해서 구성하도록하겠다. JSON데이터을 받아 서비스를 구성하는 방법과 XML데이터를 받아서 서비스를 구성하는 방법을 해보겠다.

2.1.4.1 JSON데이터 처리하는 웹서비스 구성하기

서비스 구성방법

1. **Index.jsp 페이지에서 지역명을 입력하여 버튼으로 대기 정보 요청**

요청주소 : "/air.do"

2. 공공데이터 분석결과 기준으로 AirDto클래스 생성
3. air.do주소에 매핑되어 있는 AirController구성
   - 클라이언트 요청을 받으면 지역명을 받아 저장
   - HttpURLConnection객체를 이용해서 오픈API에 데이터 요청
   - 반환결과인 json파일을 Gson을 이용하여 airDto클래스에 저장
   - AirDto를 List로 저장하고 Request에 저장후 result.jsp페이지 반환
4. result.jsp 구성
   - request데이터를 항목에 맞게 테이블로 구성하여 출력

index.jsp 구성하기

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
        <h2>OpenAPI JSON</h2>
        <form action="<%=request.getContextPath() %>/air.do">
                <input type="text" name="location">
                <input type="hidden" name="param" value="json">
                <input type="submit" value="json요청">
        </form>
</body>
</html>
```

<form>태그를 이용해서 location 값과 json 데이터로 받이 위해 두개의 파라미터를 서버로 보내고 있다.

AirDto.java 구성하기

com.kh.dto.AirDto.java

```
public class AirDto {

	private String stationName;
	private String mangName;
	private String dataTime;
	private int khaiGrade;
	private int khaiValue;
	private int pm10Grade;
	private int pm10Grade1h;
	private int pm10Value;
	private int pm10Value24;
	private int pm25Grade;
	private int pm25Grade1h;
	private int pm25Value;
	private int pm25Value24;
	private int coGrade;
	private int no2Grade;
	private int o3Grade;
	private int so2Grade;
	private double coValue;
	private double no2Value;
	private double o3Value;
	private double so2Value;

	public AirDto() {
	}
	public AirDto(String stationName, String mangName,
			String dataTime, int khaiGrade,
			int khaiValue, int pm10Grade,
			int pm10Grade1h, int pm10Value,
			int pm10Value24, int pm25Grade,
			int pm25Grade1h, int pm25Value,
			int pm25Value24, int coGrade, int no2Grade,
			int o3Grade, int so2Grade, double coValue,
			double no2Value, double o3Value,
			double so2Value) {
		this.stationName = stationName;
		this.mangName = mangName;
		this.dataTime = dataTime;
		this.khaiGrade = khaiGrade;
		this.khaiValue = khaiValue;
		this.pm10Grade = pm10Grade;
		this.pm10Grade1h = pm10Grade1h;
		this.pm10Value = pm10Value;
		this.pm10Value24 = pm10Value24;
		this.pm25Grade = pm25Grade;
		this.pm25Grade1h = pm25Grade1h;
		this.pm25Value = pm25Value;
		this.pm25Value24 = pm25Value24;
```

```
                this.coGrade = coGrade;
                this.no2Grade = no2Grade;
                this.o3Grade = o3Grade;
                this.so2Grade = so2Grade;
                this.coValue = coValue;
                this.no2Value = no2Value;
                this.o3Value = o3Value;
                this.so2Value = so2Value;
        }

        public String getStationName() {
                return stationName;
        }
        public void setStationName(String stationName) {
                this.stationName = stationName;
        }
        public String getMangName() {
                return mangName;
        }
        public void setMangName(String mangName) {
                this.mangName = mangName;
        }
        public String getDataTime() {
                return dataTime;
        }
        public void setDataTime(String dataTime) {
                this.dataTime = dataTime;
        }
        public int getKhaiGrade() {
                return khaiGrade;
        }
        public void setKhaiGrade(int khaiGrade) {
                this.khaiGrade = khaiGrade;
        }
        public int getKhaiValue() {
                return khaiValue;
        }
        public void setKhaiValue(int khaiValue) {
                this.khaiValue = khaiValue;
        }
        public int getPm10Grade() {
                return pm10Grade;
        }
        public void setPm10Grade(int pm10Grade) {
                this.pm10Grade = pm10Grade;
        }
        public int getPm10Grade1h() {
                return pm10Grade1h;
        }
```

```
	public void setPm10Grade1h(int pm10Grade1h) {
		this.pm10Grade1h = pm10Grade1h;
	}
	public int getPm10Value() {
		return pm10Value;
	}
	public void setPm10Value(int pm10Value) {
		this.pm10Value = pm10Value;
	}
	public int getPm10Value24() {
		return pm10Value24;
	}
	public void setPm10Value24(int pm10Value24) {
		this.pm10Value24 = pm10Value24;
	}
	public int getPm25Grade() {
		return pm25Grade;
	}
	public void setPm25Grade(int pm25Grade) {
		this.pm25Grade = pm25Grade;
	}
	public int getPm25Grade1h() {
		return pm25Grade1h;
	}
	public void setPm25Grade1h(int pm25Grade1h) {
		this.pm25Grade1h = pm25Grade1h;
	}
	public int getPm25Value() {
		return pm25Value;
	}
	public void setPm25Value(int pm25Value) {
		this.pm25Value = pm25Value;
	}
	public int getPm25Value24() {
		return pm25Value24;
	}
	public void setPm25Value24(int pm25Value24) {
		this.pm25Value24 = pm25Value24;
	}
	public int getCoGrade() {
		return coGrade;
	}
	public void setCoGrade(int coGrade) {
		this.coGrade = coGrade;
	}
	public int getNo2Grade() {
		return no2Grade;
	}
	public void setNo2Grade(int no2Grade) {
```

```
                this.no2Grade = no2Grade;
        }
        public int getO3Grade() {
                return o3Grade;
        }
        public void setO3Grade(int o3Grade) {
                this.o3Grade = o3Grade;
        }
        public int getSo2Grade() {
                return so2Grade;
        }
        public void setSo2Grade(int so2Grade) {
                this.so2Grade = so2Grade;
        }
        public double getCoValue() {
                return coValue;
        }
        public void setCoValue(double coValue) {
                this.coValue = coValue;
        }
        public double getNo2Value() {
                return no2Value;
        }
        public void setNo2Value(double no2Value) {
                this.no2Value = no2Value;
        }
        public double getO3Value() {
                return o3Value;
        }
        public void setO3Value(double o3Value) {
                this.o3Value = o3Value;
        }
        public double getSo2Value() {
                return so2Value;
        }
        public void setSo2Value(double so2Value) {
                this.so2Value = so2Value;
        }
}
```

공공데이터에 대한 분석틀 토대로 반환된 데이터를 저장할 수 있는 DTO(VO)를 구성했다.

AirCotroller.java 구성하기 * Sevlet

JSON데이터 파싱을 위한 라이브러리 Gson을 이용하도록하겠다, Gson을 이용하기 위해 프로젝트에 Gson라이브러리를 추가해야 한다.

*Gson추가하기

https://mvnrepository.com/artifact/com.google.code.gson/gson/2.8.6

jar파일 다운로드 후 WEB-INF/lib에 다운로드한 jar파일을 넣어준다.

com.kh.controller.AirController.java

```
package com.kh.controller;

import java.io.IOException;
import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.MalformedURLException;
import java.net.URL;
import java.net.URLEncoder;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.kh.dto.AirDto;

@WebServlet("/air.do")
public class AirController extends HttpServlet {
	private static final long serialVersionUID = 1L;

	private static final String serviceKey =
	"H1hTYjvwBEqrzWZsXvTi6T8cHV%2FEZ5D3Z7Tr5%2B0AJNeEr4qWZWQ1R
	B%2FT%2FgIpK0%2FP%2Bw%2BpkoVbJY5oYk%2BpgesyFQ%3D%3D";
	//활용신청해서 받은 인증키를 넣으면 된다.
	protected void doGet(HttpServletRequest request,
```

```
HttpServletResponse response)
                        throws ServletException, IOException {

                List<AirDto> list =
request.getParameter("param").equals("json")?openAPIJson(request.
getParameter("location")):"");//XML방식으로 확장하기 위해 작성
                request.setAttribute("list", list);
                //공공데이터에 요청하는 내용은 별도의 openAPIJson()
                //매소드를 구현하여 처리하도록 하겠다.
                RequestDispatcher dispatch =
request.getRequestDispatcher("result.jsp");
                dispatch.forward(request, response);

        }

        protected void doPost(HttpServletRequest request,
HttpServletResponse response)
                        throws ServletException, IOException {
                doGet(request, response);
        }

//공공데이터 요청하는 openAPIJson 매소드
private List<AirDto> openAPIJson(String location) {
                List<AirDto> list = new ArrayList<AirDto>();

                Scanner sc = null;

                try {

                        String url =
"http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforI
nqireSvc/getCtprvnRltmMesureDnsty?serviceKey="
                                                + serviceKey
                                                + "&sidoName="
                                                +
URLEncoder.encode(location, "UTF-8")//한글로 들어오는 내용은 UTF-8로
따로 인코딩을 해줘야한다.
                                                + "&_returnType=json";
//_returnType=json을 보내면 json방식으로 데이터를 받을 수 있다.

                        HttpURLConnection urlConnection =
(HttpURLConnection) new URL(url).openConnection();
                        InputStream resp =
urlConnection.getInputStream();
                        sc = new Scanner(resp,"UTF-8");
```

```
                        StringBuffer responseText = new
StringBuffer();
                        while (sc.hasNext()) {
                                responseText.append(sc.next());
                        }

                        JsonElement toJson =
JsonParser.parseString(responseText.toString());
                        JsonArray jsonArray =
toJson.getAsJsonObject().get("list").getAsJsonArray();

                        for (JsonElement item : jsonArray) {
                                JsonObject object =
item.getAsJsonObject();
                                AirDto dto = new AirDto();


        dto.setStationName(object.get("stationName").getAsString()
);

        dto.setMangName(object.get("mangName").getAsString());

        dto.setDataTime(object.get("dataTime").getAsString());

        dto.setKhaiGrade(object.get("khaiGrade").getAsInt());

        dto.setKhaiValue(object.get("khaiValue").getAsInt());

        dto.setPm10Grade(object.get("pm10Grade").getAsInt());

        dto.setPm10Grade1h(object.get("pm10Grade1h").getAsInt());

        dto.setPm10Value(object.get("pm10Value").getAsInt());

        dto.setPm10Value24(object.get("pm10Value24").getAsInt());

        dto.setPm25Grade(object.get("pm25Grade").getAsInt());

        dto.setPm25Grade1h(object.get("pm25Grade1h").getAsInt());

        dto.setPm25Value(object.get("pm25Value").getAsInt());

        dto.setPm25Value24(object.get("pm25Value24").getAsInt());

        dto.setCoGrade(object.get("coGrade").getAsInt());

        dto.setNo2Grade(object.get("no2Grade").getAsInt());
                                try {
```

```
        dto.setO3Grade(object.get("o3Grade").getAsInt());
                                }catch(NumberFormatException e) {
                                        dto.setO3Grade(0);
                                }

        dto.setSo2Grade(object.get("so2Grade").getAsInt());

        dto.setCoValue(object.get("coValue").getAsDouble());

        dto.setNo2Value(object.get("no2Value").getAsDouble());
                                try {

        dto.setO3Value(object.get("o3Value").getAsDouble());
                                }catch(NumberFormatException e) {
                                        dto.setO3Value(0);
                                }

        dto.setSo2Value(object.get("so2Value").getAsDouble());

                                list.add(dto);
                        }

                } catch (MalformedURLException e) {
                        e.printStackTrace();
                } catch (IOException e) {
                        e.printStackTrace();
                } finally {
                        sc.close();
                }

                return list;
        }
}
```

openAPIJson 매소드는 공공데이터를 자바의 HttpURLConnection 객체를 이용해서 요청을 보내고 JSON 형태로 반환된 데이터를 AirDTO.java 객체로 반환해서 List 에 담아 처리하는 기능을 한다.

HttpUrlConnection 객체는 java 에서 web 서버에 HttpRequet 요청을 보내고 그 응답을 스트림으로 받아 올 수 있는 객체이다.

요청을 보낼때는 URL 객체를 생성해서 openConnection()매소드를 이용을한다. URL 객체는 생성지 매개변수로 URL 주소를 작성하면 된다. openConnection() 매소드가 실행되면 URLConnection 객체가 반환이 되는데 Web 에서 데이터를

받아오기 때문에 최적화되어 있는 HttpURLConnection 객체를 이용해야해서 형변환을 해주고 HttpURLConnection 객체에 대입한다.

HttpURLConnection 객체를 이용해서 응답데이터를 받아오려면 getInputStream()메소드를 이용하면 서버에서 응답한 내용을 String 으로 받아올 수 있다. 스트림을 받아와서 StringBuffer 에 저장한 다음 Gson 으로 파싱하고 AriDto 클래스에 값을 넣어주고 있다.

result.jsp 구성하기 응답페이지 구성하기

jsp 에서는 JSTL core 태그를 사용하기 위해 라이브러리에 해당 jar 파일을 추가해준다. 간단하게 controller 에서 넣어준 데이터를 테이블 형식으로 출력하는 페이지를 구성하겠다.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

        <table border="1">
                <tr>
                        <th>측정소명</th>
                        <th>측정망 정보</th>
                        <th>측정일시</th>
                        <th>통합대기환경지수</th>
                        <th>통합대기환경수치</th>
                        <th>미세먼지(PM10) 24시간 등급자료</th>
                        <th>미세먼지(  PM10) 1시간 등급자료</th>
                        <th>미세먼지(PM10) 농도</th>
                        <th>미세먼지(PM10) 24시간 예측이동농도</th>
```

```
                        <th>미세먼지(PM2.5) 24시간 등급자료</th>
                        <th>미세먼지(PM2.5) 1시간 등급자료</th>
                        <th>미세먼지(PM2.5) 농도</th>
                        <th>미세먼지(PM2.5) 24시간 예측이동농도</th>
                        <th>일산화탄소 지수</th>
                        <th>이산화질소 지수</th>
                        <th>오존 지수</th>
                        <th>아황산가스 지수</th>
                        <th>일산화탄소 농도</th>
                        <th>이산화질소 농도</th>
                        <th>오존 농도</th>
                        <th>아황산가스 농도</th>
                </tr>
                <c:forEach items="${list }" var="dto">
                        <tr>
                                <td>${dto.stationName }</td>
                                <td>${dto.mangName }</td>
                                <td>${dto.dataTime }</td>
                                <td>${dto.khaiGrade }</td>
                                <td>${dto.khaiValue }</td>
                                <td>${dto.pm10Grade }</td>
                                <td>${dto.pm10Grade1h }</td>
                                <td>${dto.pm10Value }</td>
                                <td>${dto.pm10Value24 }</td>
                                <td>${dto.pm25Grade }</td>
                                <td>${dto.pm25Grade1h }</td>
                                <td>${dto.pm25Value }</td>
                                <td>${dto.pm25Value24 }</td>
                                <td>${dto.coGrade }</td>
                                <td>${dto.no2Grade }</td>
                                <td>${dto.o3Grade }</td>
                                <td>${dto.so2Grade }</td>
                                <td>${dto.coValue }</td>
                                <td>${dto.no2Value }</td>
                                <td>${dto.o3Value }</td>
                                <td>${dto.so2Value }</td>
                        </tr>
                </c:forEach>
        </table>

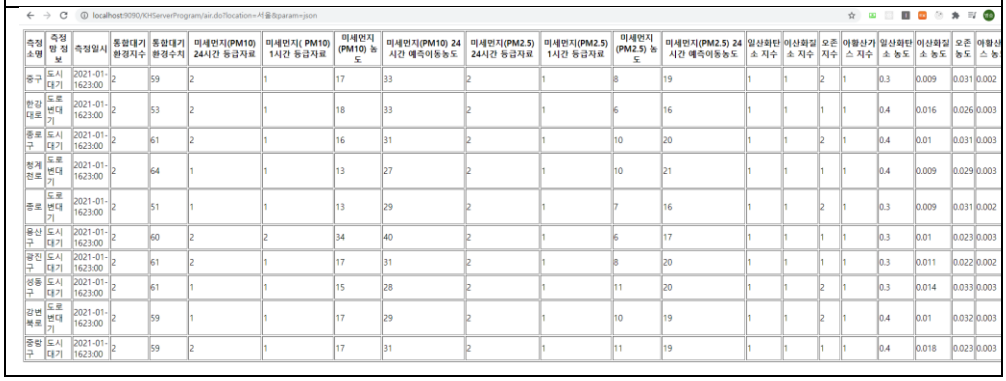
</body>
</html>
```

실행화면

요청화면



결과화면



2.1.4.2 XML데이터 처리하는 웹서비스 구성하기

이전에 구성한 내용에서 XML로 데이터처리하는 매소드를 추가해서 분기로 처리하겠다.

index.jsp XML데이터 요청 추가

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
```

```
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
	<h2>OpenAPI JSON</h2>
	<form action="<%=request.getContextPath() %>/air.do">
		<input type="text" name="location">
		<input type="hidden" name="param" value="json">
		<input type="submit" value="json요청">
	</form>
	<h2>OpenAPI XML</h2>
	<form action="<%=request.getContextPath() %>/air.do">
		<input type="text" name="location">
		<input type="hidden" name="param" value="xml">
		<input type="submit" value="xml요청">
	</form>
</body>
</html>
```

이전에 xml로 데이터를 요청하는 form태그를 추가하고, param값을 기준으로 json, xml요청을 구분하여 처리하겠다.

AirController.java 수정하기

com.kh.controller.AirController

```
package com.kh.controller;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.UnsupportedEncodingException;
import java.net.HttpURLConnection;
import java.net.MalformedURLException;
import java.net.URL;
import java.net.URLEncoder;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
```

```
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;

import org.w3c.dom.Document;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.kh.dto.AirDto;

@WebServlet("/air.do")
public class AirController extends HttpServlet {
	private static final long serialVersionUID = 1L;

	private static final String serviceKey =
"H1hTYjvwBEqrzWZsXvTi6T8cHV%2FEZ5D3Z7Tr5%2B0AJNeEr4qWZWQ1RB%2FT%2
FgIpK0%2FP%2Bw%2BpkoVbJY5oYk%2BpgesyFQ%3D%3D";

	protected void doGet(HttpServletRequest request,
HttpServletResponse response)
			throws ServletException, IOException {

		List<AirDto> list =
request.getParameter("param").equals("json")?openAPIJson(request.
getParameter("location")):openAPIXml(request.getParameter("locati
on"));//삼항연산자로 param값을 기준으로 xml,json실행 메소드를 분기함.
		request.setAttribute("list", list);

		RequestDispatcher dispatch =
request.getRequestDispatcher("result.jsp");
		dispatch.forward(request, response);

	}

	protected void doPost(HttpServletRequest request,
HttpServletResponse response)
			throws ServletException, IOException {
		doGet(request, response);
	}
```

```
//추가메소드 xml데이터 파싱처리
private List<AirDto> openAPIXml(String location){
		List<AirDto> list = new ArrayList<AirDto>();

		Scanner sc = null;

		try {
			String url =
"http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty?serviceKey="
					+ serviceKey
					+ "&sidoName="
					+
URLEncoder.encode(location, "UTF-8");

			HttpURLConnection urlConnection =
(HttpURLConnection) new URL(url).openConnection();
			InputStream inputStream =
urlConnection.getInputStream();
			InputStreamReader inputStreamReader = new
InputStreamReader(inputStream);
		BufferedReader br = new
BufferedReader(inputStreamReader);

		StringBuffer xml = new StringBuffer();
		String
path=AirController.class.getResource("/").getPath();//임시저장경로를 지정
		File f=new File(path+"sample.xml");//가져온 데이터를 xml파일로 저장.
		FileOutputStream write=new
FileOutputStream(f);//BufferedWriter로 하면 UTF-8이 인식이 안되므로 FileOutputStream으로 해야함.

		int data=-1;
		// StringBuffer sb=new StringBuffer();
		while((data=inputStream.read())!=-1) {
			write.write(data);
			//sb.append((char)data);
		}//end while
			write.close();
			//System.out.println(sb.toString());
			// System.out.println(xml.toString());
			DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
			DocumentBuilder documentBuilder =
```

```
factory.newDocumentBuilder();
                        Document document =
documentBuilder.parse(f);
                        NodeList nodelist  =
document.getElementsByTagName("item");
                        for(int i=0;i<nodelist.getLength();i++) {
                                Node n=nodelist.item(i);
                                NodeList
itemChild=n.getChildNodes();//아이템에 자식들...
                                AirDto dto=new AirDto();
                                for(int
j=0;j<itemChild.getLength();j++) {
                                        Node
itemch=itemChild.item(j);

        if(!itemch.getNodeName().equals("#text")) {
                                                String
nodeName=itemch.getNodeName();

        //System.out.println(itemch.getFirstChild().getNodeValue()
);
                                                switch(nodeName) {
                                                        case
"stationName":
dto.setStationName(itemch.getFirstChild().getNodeValue());break;
                                                        case
"dataTime" :
dto.setDataTime(itemch.getFirstChild().getNodeValue());break;
                                                        case
"so2Value" :
dto.setSo2Value(Double.parseDouble(itemch.getFirstChild().getNode
Value()));break;
                                                        case
"coValue" :
dto.setCoValue(Double.parseDouble(itemch.getFirstChild().getNodeV
alue()));break;
                                                        case
"o3Value" : try{


        dto.setO3Value(Double.parseDouble(itemch.getFirstChild().g
etNodeValue()));break;

                }catch(Exception e) {

                        dto.setO3Value(0);break;

                }
                                                        case
```

```
"no2Value" :
dto.setNo2Value(Double.parseDouble(itemch.getFirstChild().getNode
Value()));break;
                                                        case
"pm10Value" :
dto.setPm10Value(Integer.parseInt(itemch.getFirstChild().getNodeV
alue()));break;
                                                        case
"khaiValue" :
dto.setKhaiValue(Integer.parseInt(itemch.getFirstChild().getNodeV
alue()));break;
                                                        case
"khaiGrade" :
dto.setKhaiGrade(Integer.parseInt(itemch.getFirstChild().getNodeV
alue()));break;
                                                        case
"so2Grade" :
dto.setSo2Grade(Integer.parseInt(itemch.getFirstChild().getNodeVa
lue()));break;
                                                        case
"coGrade" :
dto.setCoGrade(Integer.parseInt(itemch.getFirstChild().getNodeVal
ue()));break;
                                                        case
"o3Grade" : try{


       dto.setO3Grade(Integer.parseInt(itemch.getFirstChild().get
NodeValue()));break;

               }catch(Exception e) {

                       dto.setO3Grade(0);break;

               }
                                                        case
"no2Grade" :
dto.setNo2Grade(Integer.parseInt(itemch.getFirstChild().getNodeVa
lue()));break;
                                                        case
"pm10Grade" :
dto.setPm10Grade(Integer.parseInt(itemch.getFirstChild().getNodeV
alue()));break;
                                                    }
                                                }
                                        }
                                        list.add(dto);
                                }
                                f.delete();
```

```
            } catch (UnsupportedEncodingException e) {
                  e.printStackTrace();
            } catch (MalformedURLException e) {
                  e.printStackTrace();
            } catch (IOException e) {
                  e.printStackTrace();
            } catch (ParserConfigurationException e) {
                  // TODO Auto-generated catch block
                  e.printStackTrace();
            } catch (SAXException e) {
                  // TODO Auto-generated catch block
                  e.printStackTrace();
            }

            return list;
      }
```

xml 페이지를 파싱하기 위해서는 org.w3.dom 패키지에서 제공하는 Docment 객체를 이용해서 원하는 데이터를 출력할 수가 있다. Xml 은 태그형태로 제공이 되기 때문에 그 태그를 분기해서 데이터를 가져와야해서 약간 복잡할 수 있다.

일단 오픈 API 에서 데이터를 가져오는 방식은 JSON 으로 가져오는 방식이 동일하나 요청주소 매개변수에 _returnType=json 의 내용을 빼고 요청을 보낸다. 그렇게 보내면 XML 형태로 반환받을 수 있다.

XML 형태로 받으면 구조를 잘 파악해야한다. 태그는 트리구조로 계층화되어있어 그 계층과 태그명을 알아야지 파싱하여 데이터를 가져올 수 있다.

XML 구조

root 태그 : response

자식태그

header : 해당서비스 코드, 메시지 정보

body : items 태그를 자식으로 가지고 items 실제 데이터이다.

numOfRows : 페이지당 데이터 수

pageNo : 페이지번호

totalCount : 전체 데이터수

아래 그림과 같이 데이터를 가져오기 위해서는 items 에 있는 item 태그를 가져와서 파싱하면 된다. 태그명은 공공데이터 분석이에 확인한 항목명(영문)인 것을 볼 수 있다. 그값으로 파싱하여 AirDto 객체에 각 item 태그의 값을 넣어주면 되겠다.

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<response>
  ▼<header>
    <resultCode>00</resultCode>
    <resultMsg>NORMAL SERVICE.</resultMsg>
  </header>
  ▼<body>
    ▼<items>
      ▼<item>
        <stationName>중구</stationName>
        <mangName>도시대기</mangName>
        <dataTime>2021-01-16 24:00</dataTime>
        <so2Value>0.003</so2Value>
        <coValue>0.4</coValue>
        <o3Value>0.029</o3Value>
        <no2Value>0.009</no2Value>
        <pm10Value>17</pm10Value>
        <pm10Value24>29</pm10Value24>
        <pm25Value>10</pm25Value>
        <pm25Value24>16</pm25Value24>
        <khaiValue>51</khaiValue>
        <khaiGrade>2</khaiGrade>
        <so2Grade>1</so2Grade>
        <coGrade>1</coGrade>
        <o3Grade>1</o3Grade>
        <no2Grade>1</no2Grade>
        <pm10Grade>1</pm10Grade>
        <pm25Grade>2</pm25Grade>
        <pm10Grade1h>1</pm10Grade1h>
        <pm25Grade1h>1</pm25Grade1h>
      </item>
      ▼<item>
        <stationName>한강대로</stationName>
        <mangName>도로변대기</mangName>
        <dataTime>2021-01-16 24:00</dataTime>
        <so2Value>0.003</so2Value>
        <coValue>0.4</coValue>
        <o3Value>0.028</o3Value>
        <no2Value>0.013</no2Value>
        <pm10Value>14</pm10Value>
        <pm10Value24>28</pm10Value24>
        <pm25Value>8</pm25Value>
        <pm25Value24>14</pm25Value24>
        <khaiValue>47</khaiValue>
        <khaiGrade>1</khaiGrade>
        <so2Grade>1</so2Grade>
        <coGrade>1</coGrade>
        <o3Grade>1</o3Grade>
        <no2Grade>1</no2Grade>
        <pm10Grade>1</pm10Grade>
        <pm25Grade>1</pm25Grade>
        <pm10Grade1h>1</pm10Grade1h>
        <pm25Grade1h>1</pm25Grade1h>
      </item>
      ▼<item>
```

DocumentBuilder 객체의 parse()매소드를 이용해서 String 으로 넘어노는 xml 태그를 파싱을 하여 데이터를 가져와야 한다. 여기서원래 parse()매소드의 매개변수에 inputStream 을 이용해서 바로 xml 데이터를 바로 받아와서 처리할 수 있지만, 공공데이터에 대한 xml 파일을 바로 받아오는 경우 parse 에러가 발생하여 HttpURLConnection 으로 가져오는 데이터를 임시로 sample.xml 파일 저장하고 그 파일을 DocumentBuilder.parse()매소드를 이용해서 불러와 파싱처리를 했다.

그리고 불러올 때 UTF-8 방식의 XML 파일은 Writer 를 이용해서 파일에 저장하는 경우 한글이 깨지는 현상이 발생하므로 FileInputStream 을 이용해서 처리애햐 한다.

Document 객체에서 각 태그에 접근하려면 getElementByTagName("태그명") 매소드를 이용하면 태그에 접근할 수 있고, NodeList 객체로 반환이 되면 for 문을 이용해서 순회애서 각 node 에 접근 후 데이터를 getFirstChild(). getNodeValue()매소드를 이용하면 가져올 수 있다.

매소드를 이용해서 가져온 데이터는 getNodeName()으로 태그명을 확인 후 태그명과 일치하는 AirDto 객체 필드에 넣어주면 된다.

Json 과 달리 XML 형식은 데이터의 내용이 상당부분 짤리는 현상이 발생하는 것이 있어 JSON 방식으로 받아오는 것이 더 좋다.

실행화면

요청화면



실행결과



localhost:9090/KHServerProgram/air.do?location=서울&param=xml

측정소명	측정망 정보	측정일시	통합대기환경지수	통합대기환경수치	미세먼지(PM10) 24시간 등급자료	미세먼지(PM10) 1시간 등급자료	미세먼지(PM10) 농도	미세먼지(PM10) 24시간 예측이동농도	미세먼지(PM2.5) 24시간 등급자료	미세먼지(PM2.5) 1시간 등급자료	미세먼지(PM2.5) 농도	미세먼지(PM2.5) 24시간 예측이동농도	일산화탄소 지수	이산화질소 지수	오존지수	아황산가스 지수	일산화탄소 농도	이산화질소 농도	오존농도	아황산가스
중구		2021-01-16 24:00	2	51	1	0	17	0	0	0	0	0	1	1	1	1	0.4	0.009	0.029	0.003
한강대로		2021-01-16 24:00	1	47	1	0	14	0	0	0	0	0	1	1	1	1	0.4	0.013	0.028	0.003
종로구		2021-01-16 24:00	2	56	1	0	17	0	0	0	0	0	1	1	1	1	0.4	0.01	0.03	0.003
청계천로		2021-01-16 24:00	2	54	1	0	13	0	0	0	0	0	1	1	1	1	0.4	0.011	0.027	0.003
종로		2021-01-16 24:00	1	47	1	0	13	0	0	0	0	0	1	1	1	1	0.4	0.01	0.028	0.003
용산구		2021-01-16 24:00	2	56	2	0	18	0	0	0	0	0	1	1	1	1	0.4	0.009	0.023	0.003
광진구		2021-01-16 24:00	2	54	1	0	16	0	0	0	0	0	1	1	1	1	0.3	0.011	0.02	0.002
성동구		2021-01-16 24:00	2	54	1	0	14	0	0	0	0	0	1	1	2	1	0.4	0.014	0.032	0.003
강변북로		2021-01-16 24:00	2	54	1	0	13	0	0	0	0	0	1	1	1	1	0.5	0.014	0.027	0.003
중랑구		2021-01-16 24:00	2	54	1	0	17	0	0	0	0	0	1	1	1	1	0.4	0.019	0.022	0.003
동대문구		2021-01-16 24:00	2	54	1	0	16	0	0	0	0	0	1	1	1	1	0.4	0.012	0.027	0.003

공공데이터를 이용하는 방법에 대해 알아봤다. 다른 공공데이터도 유사한방식으로 URL 요청주소와 parameter 값만 알고있다면 손쉽게 다양한 정보를 가지고 더 다양한 웹애플리케이션을 구현할 수 있다.